ICPC QUICKSHEET - KACTL-STYLE VI

Bản tra cứu nhanh C++: ưu tiên code ngắn, dễ chép, mô tả tiếng Việt. Các mục KACTL/CP3 được chắt lọc, viết lại ngắn gọn theo phong cách global array + function.

BẢNG TRA NHANH - KHI NÀO DÙNG THUẬT TOÁN NÀO?

Trang 1: dấu hiệu đề bài, giới hạn thường gặp, điều kiện áp dụng và độ phức tạp

1. Nhìn N trước khi chọn thuật toán

Quy mô	Gợi ý khả thi
$N \le 10$	Thử hoán vị $O(N!)$; $10! \approx 3.6$ triệu.
$N \le 20$	**Bitmask DP** $O(N2^N)$ nếu state là tập.
$N \le 15$	Submask toàn bộ $O(3^N)$: $3^{15} \approx 14$ triệu.
$N \approx 40$	**Meet-in-the-middle** $O(N2^{N/2})$.
$N \lesssim 400$	Có thể thử $O(N^3)$ nếu hằng số nhỏ.
$N \lesssim 4000$	Có thể thử $O(N^2)$, tùy TL.
$N \sim 10^5$–10^6	Ưu tiên $O(N \log N)$ / $O(N)$.
$V \lesssim 400$	Floyd $O(V^3)$ nếu cần mọi cặp.
Bề rộng $m \le 10$–12	Profile DP $O(N \cdot \text{poly}(m) 2^m)$.
W nhỏ, N lớn	Knapsack $O(NW)$: phụ thuộc *W*, không chỉ N.

Không phải $N < 20$ là luôn dùng QHD. Chỉ dùng **bitmask DP** khi mỗi trạng thái biểu diễn được bằng tập con, và tổng trạng thái/chuyển trạng thái đủ nhỏ. Giới hạn trên chỉ là *ước lượng*, còn phụ thuộc time limit, số test, memory và hằng số.

Ước lượng bộ nhớ bitmask

2^{20} trạng thái $\times 8$ byte ≈ 8 MiB.
2^{24} trạng thái $\times 8$ byte ≈ 128 MiB.
2^{26} trạng thái $\times 8$ byte ≈ 512 MiB.
Với `dp[mask][last]`, còn nhân thêm chiều `last`.

2. Chọn dạng QHD (DP)

0/1 Knapsack: mỗi vật tối đa 1 lần ⇒ `item ngoài, capacity giảm`.
Complete Knapsack: dùng vật vô hạn ⇒ `item ngoài, capacity tăng`.
Coin combinations: không phân biệt thứ tự ⇒ `coin ngoài, sum trong`.
Coin sequences: phân biệt thứ tự ⇒ `sum ngoài, coin trong`.
Interval DP: ghép đoạn nhỏ ⇒ `length tăng, left, split`.
Bitmask DP: đã chọn những gì ⇒ `mask, next choice`; thường $N \le 20$.
Digit DP: đếm số trong $[L, R]$ theo chữ số; state `pos/tight/started/...`.
Profile DP: lưới dài nhưng bề rộng nhỏ; mask biểu diễn biên.
Tree DP: con trước cha (postorder); cần mọi root ⇒ rerooting.
D&C/Knuth DP: chỉ khi chứng minh được vị trí tối ưu đơn điệu (và điều kiện tương ứng).

3. Đồ thị: nhận biết qua trọng số

Điều kiện	Chọn
Không trọng số	BFS $O(V+E)$
Trọng số chỉ 0/1	0-1 BFS, deque
Mọi trọng số ≥ 0	Dijkstra $O(E \log V)$
Có cạnh âm	Bellman-Ford; kiểm tra chu trình âm
Đồ thị DAG	Topo + DP, kể cả cạnh âm
Mọi cặp, V nhỏ	Floyd-Warshall $O(V^3)$
Nhiều trạng thái + mỗi bước 1	BFS trên đồ thị trạng thái
Đúng K cạnh, K lớn	Matrix power; min-plus nếu cần min cost

Điều kiện cứng: Dijkstra không dùng trực tiếp khi có cạnh âm. Topological DP cần DAG. BFS cần các bước đồng chi phí.

4. Liên thông, cây, ghép cặp

Connectivity tĩnh: DFS/BFS. Có `union` và query nối nhau ⇒ DSU.
Add/delete edge offline: rollback DSU + segment tree theo thời gian.
MST vô hướng: Kruskal + DSU hoặc Prim.
Minimax path: MST + LCA lấy max edge trên path.
Directed cycles: SCC; gom SCC rồi topo DP nếu bài cho phép gom state.
Nhiều LCA/k-th ancestor: binary lifting $O(\log N)$ mỗi query.
Nhiều query đường trên cây: HLD + Segment Tree $O(\log^2 N)$.
Bipartite matching không trọng số: Hopcroft-Karp.
Assignment có cost: Hungarian; có capacity tổng quát ⇒ MCMF.
Capacity / disjoint paths / min-cut: Dinic.
Matching không bipartite: không dùng Hopcroft-Karp; thử Blossom hoặc bitmask DP nếu số đỉnh nhỏ.

5. Cấu trúc dữ liệu cho query

Nhu cầu	Chọn
Tổng tĩnh	Prefix sum
Min/max/gcd tĩnh	Sparse Table
Point add, range sum	Fenwick (BIT)
Range update/query	Lazy Segment Tree
k-th/order statistics	PBDS / frequency SegTree
Min/max theo cửa sổ	Monotonic deque
Median/cost cửa sổ	2 multiset / Fenwick
Sequence insert/reverse	Implicit Treap
Nhiều đoạn offline	Mo, khi add/del rẻ
Min/max các hàm $mx+b$	CHT / Li Chao

Cẩn thận: Sparse Table $O(1)$ kiểu overlap cần phép idempotent (min/max/gcd). Segment Tree cần phép gộp kết hợp (associative).

6. Chuỗi: câu hỏi nào, thuật toán đó

Một pattern trong text: KMP/Z, không collision.
Border / chu kỳ của chuỗi: prefix-function hoặc Z.
Palindrome dài nhất: Manacher $O(N)$.
So sánh nhiều substring: Rolling Hash; chấp nhận rủi ro collision hoặc dùng double hash.
Nhiều pattern: Aho-Corasick.
Suffix theo thứ tự từ điển/LCP: Suffix Array + Kasai.
Đếm distinct substring/append online: Suffix Automaton.
Đếm palindrome khác nhau: Eertree.
Xoay vòng nhỏ nhất: Booth.

7. Số học: giới hạn giá trị

Nhiều truy vấn factor, giá trị $\lesssim 10^7$**:** sàng SPF / Linear Sieve (xem memory).
Primality 64-bit: Miller-Rabin deterministic.
Factor 64-bit lớn: Pollard-Rho + Miller-Rabin.
GCD bằng 1 / đúng bằng d: Möbius hoặc đếm theo bội.
Chia hết ít nhất một prime trong $K \le 20$**:** Inclusion-Exclusion $O(2^K)$.
Tổng có $\lfloor N/i \rfloor$**:** gom thương $O(\sqrt{N})$.
$\binom{n}{k}$ mod p**:** factorial/inverse nếu $n < p$; Lucas nếu p prime nhỏ, n rất lớn.
Đối xứng quay/lật: Burnside; không tự ý chia cho số phép đối xứng.
Recurrence, chỉ số cực lớn: Fast doubling (Fibonacci), matrix/Kitamasa; chưa biết recurrence ⇒ BM.
Convolution hệ số lớn: NTT nếu modulus phù hợp.
Phương trình tuyến tính: Gaussian; modulo prime thì dùng inverse modular.

8. Geometry / search / số thực

Điểm nguyên: Cross/Orient, dùng `__int128` nếu tọa độ lớn.
Bao lồi: Convex Hull $O(N \log N)$; cặp xa nhất trên hull ⇒ calipers.
Cặp điểm gần nhất: Closest Pair $O(N \log N)$.
Circle nhỏ nhất phủ tất cả: MEC randomized, expected $O(N)$.
Circle nhỏ nhất phủ $\ge K$ **điểm:** fix 2 điểm biên + 1D sweep, $O(N^3 \log N)$; hợp khi $N \lesssim 300$ và TL đủ.
Tìm min/max đáp án: Binary Search Answer nếu checker monotone.
Hàm một đỉnh/cực trị: Ternary/Golden Section nếu unimodal.
Bruteforce subset $N \approx 40$**:** Meet-in-the-Middle, chỉ nếu hai nửa ghép được.
Số thực: EPS theo scale, clamp trước `acos`, dùng `atan2(cross,dot)`.

Quy tắc chốt: tính số trạng thái $\times$ số chuyển $\times$ số test; kiểm tra memory trước. *Dấu hiệu dạng bài và điều kiện đúng quan trọng hơn chỉ riêng N.*

Mẫu / quy ước

```
#include <bits/stdc++.h>
using namespace std;
#define fast_io ios_base::sync_with_stdio(false)
     ;cin.tie(nullptr);cout.tie(nullptr)
#define ll long long
#define ld long double
#define int128 __int128
#define vt vector
#define ar array
#define ii pair<int,int>
#define pll pair<ll,ll>
#define vi vector<int>
#define vll vector<ll>
#define ft first
#define sd second
#define pb push_back
#define eb emplace_back
#define all(x) (x).begin(),(x).end()
#define rall(x) (x).rbegin(),(x).rend()
#define sz(x) (int)(x).size()
```

Quy ước: mảng toàn cục + hàm; mã ngắn nhưng vẫn dễ đọc; dùng ll cho khoảng cách/tổng; kiểm tra đánh số từ 0/1; khởi tạo lại dữ liệu mỗi bộ test.

Nhận diện dạng bài nhanh

Tham lam một lượt: sau khi xử lý vị trí i, giữ tiền tố đã hợp lệ.

$1+2+\cdots+n=n(n+1)/2$. Đếm bù: **đáp án = tất cả - trường hợp xấu**.

Trước khi mô phỏng, kiểm tra bất biến về chẵn lẻ / modulo / gcd / tính đơn điệu.

Dựng nghiệm: nếu đề chỉ cần *một nghiệm hợp lệ*, hãy dựng cấu hình đơn giản thay vì tối ưu.

Mẹo tần suất: nếu cấm hai giá trị bằng nhau đứng cạnh nhau, điều kiện cần thường là max $freq \le \lceil n/2 \rceil$.

```
// đuyt ậtp con
for(int mask=0;mask<(1<<n);mask++){
  for(int i=0;i<n;i++) if(mask>>i&1){ }
}
// mã Gray
int g=i^(i>>1);
```

Gợi ý: $n \le 20$ -> nghĩ đến bitmask; $n \approx 40$ -> chia đôi tập.

Khung quay lui: loại trạng thái sai -> đánh dấu -> thử lựa chọn -> bỏ đánh dấu. Cắt tỉa mạnh có thể biến vét cạn lớn thành chạy được.

Chia theo khối chữ số: tìm khối chứa vị trí k thay vì sinh toàn bộ dãy.

$n!$ có số chữ số 0 tận cùng là: $\sum_{k\ge1}\lfloor n/5^k \rfloor$. XOR: $x\oplus x=0$, $x\oplus 0=x$.

BFS / Dijkstra / 0-1 BFS

Ngữ cảnh dùng: Bài hỏi đường đi ngắn trên graph. BFS khi mọi cạnh cùng chi phí; 0-1 BFS khi trọng số chỉ 0/1; Dijkstra khi mọi trọng số không âm.

BFS - đường đi ngắn nhất không trọng số

```
vector<int> d(n+1,-1);
queue<int> q;
d[s]=0;q.push(s);
while(!q.empty()){
  int u=q.front();q.pop();
  for(int v:adj[u]) if(d[v]==-1){
    d[v]=d[u]+1;q.push(v);
  }
}
```

$O(V+E)$. Dùng khi mọi cạnh có cùng chi phí. BFS trên lưới/trạng thái dùng cùng ý tưởng với dx,dy.

Dijkstra

```
vector<pair<int,int>> adj[N];
vector<ll> dijkstra(int s){
  vector<ll>d(n+1,LINF);
  priority_queue<pair<ll,int>,
    vector<pair<ll,int>>,
    greater<pair<ll,int>>>q;
  d[s]=0;q.push({0,s});
  while(!q.empty()){
    auto [du,u]=q.top();q.pop();
    if(du>d[u])continue;
    for(auto [v,w]:adj[u])
      if(d[v]>du+w)
        d[v]=du+w,q.push({d[v],v});
  }
  return d;
}
```

Chỉ dùng khi $w \ge 0$. $O((V+E)\log V)$. Vô hướng: thêm cả 2 chiều.

0-1 BFS

```
vector<int>d(n+1,INF);
deque<int>q;d[s]=0;q.push_front(s);
while(!q.empty()){
  int u=q.front();q.pop_front();
  for(auto [v,w]:adj[u]) if(d[v]>d[u]+w){
    d[v]=d[u]+w;
    if(w==0)q.push_front(v);
    else q.push_back(v);
  }
}
```

Chỉ dùng trọng số 0/1. $O(V+E)$.

Bellman-Ford / Floyd-Warshall

Ngữ cảnh dùng: Có cạnh âm hoặc cần mọi cặp. Bellman-Ford chịu được cạnh âm và phát hiện chu trình âm; Floyd phù hợp khi số đỉnh nhỏ và cần khoảng cách giữa nhiều cặp.

Bellman-Ford - có cạnh âm

```
vector<ll>d(n+1,LINF);d[s]=0;
for(int it=1;it<n;it++){
  bool ok=0;
  for(auto [u,v,w]:edges) if(d[u]!=LINF&&d[v]>d[
      u]+w)
    d[v]=d[u]+w,ok=1;
  if(!ok)break;
}
// ặlp thêm 1 vòng mà còn relax => có chu trình
     âm reachable
```

$O(VE)$. Dùng khi có trọng số âm. Muốn phát hiện chu trình âm: kiểm tra còn relax ở vòng thứ n.

Floyd-Warshall - mọi cặp

```
for(int k=1;k<=n;k++)
 for(int i=1;i<=n;i++)
  for(int j=1;j<=n;j++)
   if(d[i][k]<LINF&&d[k][j]<LINF)
     d[i][j]=min(d[i][j],d[i][k]+d[k][j]);
```

$O(N^3)$. Khởi tạo $d[i][i]=0$, cạnh song song lấy min. Có chu trình âm nếu sau chạy tồn tại $d[i][i]<0$.

DSU / Kruskal

Ngữ cảnh dùng: Cần gộp thành phần liên thông hoặc dựng MST từ danh sách cạnh. Kruskal áp dụng cho graph vô hướng có trọng số; DSU rất hợp với xử lý offline theo ngưỡng.

DSU / Union-Find

```
int p[N],sz[N];
void init(int n){
  for(int i=1;i<=n;i++)p[i]=i,sz[i]=1;
}
int Find(int u){
  return u==p[u]?u:p[u]=Find(p[u]);
}
bool Union(int u,int v){
  u=Find(u);v=Find(v);
  if(u==v)return 0;
  if(sz[u]<sz[v])swap(u,v);
  p[v]=u;sz[u]+=sz[v];
  return 1;
}
```

Nén đường đi + gộp theo kích thước. Trung bình $O(\alpha(N)) \approx O(1)$.

Kruskal

```
vector<array<int,3>> edges; // {w,u,v}
sort(all(edges));
init(n);ll mst=0;int cnt=0;
for(auto [w,u,v]:edges){
  if(Union(u,v)) mst+=w,cnt++;
}
if(cnt!=n-1){ /* no MST */ }
```

Sắp xếp theo trọng số; lấy cạnh khi hai đầu thuộc hai thành phần khác nhau. $O(E\log E)$.

Topo / SCC / LCA

Ngữ cảnh dùng: Topo cho DAG; SCC cho graph có hướng có chu trình; LCA cho truy vấn quan hệ tổ tiên/khoảng cách trên cây đã cố định gốc.

Sắp xếp topo - Kahn

```
queue<int>q;
for(int i=1;i<=n;i++)if(indeg[i]==0)q.push(i);
vector<int>ord;
while(!q.empty()){
  int u=q.front();q.pop();ord.pb(u);
  for(int v:adj[u])if(--indeg[v]==0)q.push(v);
}
// là DAG khi và ỉch khi sz(ord)==n
```

SCC - Kosaraju rút gọn

```
vector<int>g[N],rg[N],ord;
int vis[N],comp[N];
void dfs1(int u){
  vis[u]=1;for(int v:g[u])if(!vis[v])dfs1(v);
  ord.pb(u);
}
void dfs2(int u,int c){
  comp[u]=c;
  for(int v:rg[u])if(!comp[v])dfs2(v,c);
}
// ạchy dfs1 cho ọmi đỉnh; đảo ord; ạchy dfs2
     theo ứth ựt đó
```

LCA - nâng nhị phân

```
int up[N][LOG],dep[N];
void dfs(int u,int p){
  up[u][0]=p;
  for(int j=1;j<LOG;j++)up[u][j]=up[up[u][j-1]][
      j-1];
  for(int v:adj[u])if(v!=p)dep[v]=dep[u]+1,dfs(v
      ,u);
}
int lca(int u,int v){
  if(dep[u]<dep[v])swap(u,v);
  int d=dep[u]-dep[v];
  for(int j=0;j<LOG;j++)if(d>>j&1)u=up[u][j];
  if(u==v)return u;
  for(int j=LOG-1;j>=0;j--)if(up[u][j]!=up[v][j
      ])u=up[u][j],v=up[v][j];
  return up[u][0];
}
```

Tiền xử lý $O(N\log N)$, truy vấn $O(\log N)$. Có thể đặt cha của gốc là chính gốc.

Cầu / đỉnh khớp / Euler

Ngữ cảnh dùng: Cầu/đỉnh khớp để tìm điểm nghẽn của graph vô hướng; Euler khi cần đi qua mỗi cạnh đúng một lần. Kiểm tra điều kiện bậc và liên thông trước.

Cầu và đỉnh khớp - Tarjan DFS

```
int timer,tin[N],low[N];
void dfsBridge(int u,int pe=-1){
  tin[u]=low[u]=++timer;int child=0;
  for(auto [v,id]:adjE[u]){
    if(id==pe)continue;
    if(!tin[v]){
      dfsBridge(v,id);child++;
      low[u]=min(low[u],low[v]);
      if(low[v]>tin[u]){/* (u,v) la cau */}
      if(pe!=-1&&low[v]>=tin[u]){/* u la khop */
      }
    }else low[u]=min(low[u],tin[v]);
  }
  if(pe==-1&&child>1){/* u la khop */}
}
```

Đồ thị vô hướng. Cầu: $low[v] > tin[u]$. Khớp (không phải gốc): $low[v] \ge tin[u]$.

Euler Path/Circuit - Hierholzer

```
vector<int>path,it(n+1);
void euler(int s){
  vector<int>st={s};
  while(!st.empty()){
    int u=st.back();
    while(it[u]<sz(g[u])&&used[g[u][it[u]].sd])
      it[u]++;
    if(it[u]==sz(g[u]))path.pb(u),st.pop_back();
    else{
      auto [v,id]=g[u][it[u]++];
      if(!used[id])used[id]=1,st.pb(v);
    }
  }
  reverse(all(path));
}
```

Vô hướng: circuit khi mọi bậc chẵn; path khi đúng 0 hoặc 2 đỉnh bậc lẻ. Có hướng: so sánh indeg/outdeg và cần điều kiện liên thông phù hợp. Độ dài đường đi phải là $E+1$.

Max Flow - Dinic

Ngữ cảnh dùng: Bài có tài nguyên/capacity qua mạng, min-cut, edge-disjoint/vertex-disjoint paths hoặc matching có thể mô hình thành flow. Capacity phải không âm.

```
struct Edge{int v,rev;ll cap;}; // ạngoi ệl:
     struct giúp code flow ắngn/an toàn
vector<Edge>gF[N];int lv[N],itF[N];
void addEdge(int u,int v,ll c){
  Edge a{v,sz(gF[v]),c},b{u,sz(gF[u]),0};
  gF[u].pb(a);gF[v].pb(b);
}
bool bfsF(int s,int t){
  fill(lv,lv+N,-1);queue<int>q;q.push(s);lv[s
      ]=0;
  while(!q.empty()){
    int u=q.front();q.pop();
    for(auto &e:gF[u])if(e.cap&&lv[e.v]<0)lv[e.v
      ]=lv[u]+1,q.push(e.v);
  }return lv[t]>=0;
}
ll dfsF(int u,int t,ll f){
  if(u==t)return f;
  for(int &i=itF[u];i<sz(gF[u]);i++){
    Edge &e=gF[u][i];
    if(e.cap&&lv[e.v]==lv[u]+1){
      ll got=dfsF(e.v,t,min(f,e.cap));
      if(got){e.cap-=got;gF[e.v][e.rev].cap+=got
      ;return got;}
    }
  }return 0;
}
ll maxflow(int s,int t){
  ll ans=0,f;
  while(bfsF(s,t)){
    fill(itF,itF+N,0);
    while((f=dfsF(s,t,LINF)))ans+=f;
  }return ans;
}
```

Dinic chạy rất nhanh trong thực tế. Min-cut: sau max-flow, các đỉnh còn reachable từ nguồn trong

residual graph nằm phía nguồn.

Matching hai phía - Hopcroft-Karp (khung)

Ngữ cảnh dùng: Ghép cặp tối đa giữa hai tập rời nhau, cạnh không có cost. Nếu có cost assignment thì nghĩ Hungarian/MCMF; graph không bipartite thì HK không dùng được.

```
// L: 1..nL, R: 1..nR; mtR[v]=u
bool aug(int u){
 if(vis[u])return 0;vis[u]=1;
 for(int v:g[u])if(!mtR[v]||aug(mtR[v])){mtR[v]=
     u;return 1;}
 return 0;
}
// åBn DFS đơn ågin: O(VE), đủ khi ỏnh.
// N ổln: Hopcroft-Karp dùng BFS phân ớlp + DFS,
      O(E*sqrt(V)).
```

Nhận diện: ghép tối đa giữa 2 tập, mỗi đỉnh dùng tối đa một lần. Với constraints lớn hãy dùng Hopcroft-Karp chuẩn.

2-SAT

Ngữ cảnh dùng: Mỗi biến Boolean có 2 lựa chọn và mối ràng buộc quy về clause 2 literal. Dùng implication graph + SCC; nếu một biến và phủ định cùng SCC thì vô nghiệm.

Mỗi mệnh đề $(a \vee b)$ tạo hai implication: $\neg a \to b$, $\neg b \to a$. Chạy SCC trên đồ thị $2N$ đỉnh. Vô nghiệm iff biến x và $\neg x$ thuộc cùng SCC. Gán theo thứ tự SCC topo ngược.

Prim - MST theo đỉnh

```
ll prim(int s){
 vector<int>vis(n+1);ll ans=0;int cnt=0;
 priority_queue<pair<ll,int>,vector<pair<ll,int
     >>,greater<pair<ll,int>>>q;
 q.push({0,s});
 while(!q.empty()){
  auto [w,u]=q.top();q.pop();
  if(vis[u])continue;vis[u]=1;ans+=w;cnt++;
  for(auto [v,c]:adj[u])if(!vis[v])q.push({c,v})
      ;
 }
 return cnt==n?ans:-1;
}
```

Prim $O(E \log V)$, tiện khi đã có adjacency list. Kruskal thường ngắn hơn; Prim hữu ích khi muốn mở rộng cây dần từ một đỉnh.

Meet-in-the-middle / tìm kiếm hai phía

```
// subset sum n~40
vector<ll> gen(vector<ll>a){
 vector<ll>s={0};
 for(ll x:a){int z=sz(s);for(int i=0;i<z;i++)s.
     pb(s[i]+x);}
 return s;
}
auto L=gen(left),R=gen(right);sort(all(R));
// tim tong lon nhat <= S
for(ll x:L){
 auto it=upper_bound(all(R),S-x);
 if(it!=R.begin())--it,ans=max(ans,x+*it);
}
```

MITM: chia trạng thái thành hai nửa, thường từ 2^N xuống khoảng $2^{N/2}$. Với state-space có nguồn và đích rõ, cân nhắc BFS hai phía và gặp nhau ở giữa. A*: ưu tiên $f = g + h$, trong đó h nên là cận dưới (admissible).

Floyd cycle-finding - rùa/thỏ

```
template<class F> pair<ll,ll> floydCycle(ll x0,F
      f){
 ll t=f(x0),h=f(f(x0));
 while(t!=h)t=f(t),h=f(f(h));
 ll mu=0;h=x0;
 while(t!=h)t=f(t),h=f(h),mu++;
 ll lam=1;h=f(t);
 while(t!=h)h=f(h),lam++;
 return {mu,lam};
}
```

μ = độ dài tiền chu kỳ, λ = chu kỳ. $O(\mu+\lambda)$ thời gian, $O(1)$ bộ nhớ. Hữu ích cho dãy sinh bởi $x_{i+1} = f(x_i)$.

Min-Cost Max-Flow - khung SPFA

```
vector<int> gCost[N];
int to[M],cap[M],costE[M],ec;
void addCost(int u,int v,int c,int w){
 to[ec]=v;cap[ec]=c;costE[ec]=w;gCost[u].pb(ec
     ++);
 to[ec]=u;cap[ec]=0;costE[ec]=-w;gCost[v].pb(ec
     ++);
}
pair<ll,ll> mcmf(int s,int t){
 ll flow=0,cost=0;
 while(1){
  vector<ll>d(n+1,LINF);vector<int>pe(n+1,-1),
      inq(n+1);
  queue<int>q;d[s]=0;q.push(s);inq[s]=1;
  while(!q.empty()){
   int u=q.front();q.pop();inq[u]=0;
   for(int e:gCost[u])if(cap[e]&&d[to[e]]>d[u]+
       costE[e]){
    d[to[e]]=d[u]+costE[e];pe[to[e]]=e;
    if(!inq[to[e]])inq[to[e]]=1,q.push(to[e]);
   }
  }
  if(pe[t]==-1)break;
  int f=INF;for(int v=t;v!=s;v=to[pe[v]^1])f=min
      (f,cap[pe[v]]);
  for(int v=t;v!=s;v=to[pe[v]^1]){int e=pe[v];
      cap[e]-=f;cap[e^1]+=f;cost+=1LL*f*costE[e
      ];}
  flow+=f;
 }
 return {flow,cost};
}
```

Dùng khi mỗi cạnh có cả capacity và cost. Reverse edge có cost âm để hoàn tác flow. Bản SPFA ngắn, phù hợp constraints vừa/nhỏ; bản mạnh hơn dùng potential + Dijkstra.

Min Path Cover DAG

Với DAG, dựng bipartite graph: mỗi đỉnh có bản trái/phải, cạnh $u_L \to v_R$ nếu có cạnh $u \to v$. Nếu matching cực đại có kích thước m thì số path ít nhất phủ mọi đỉnh là $n - m$. Có thể dùng DFS matching hoặc Hopcroft-Karp.

Chinese Postman - đồ thị vô hướng

```
// dist ũgia các đỉnh ậbc ẻl đã có ừt APSP
ll dp[1<<20];
ll matchOdd(int mask){
 if(!mask)return 0;
 ll &res=dp[mask];if(res!=-1)return res;
 int i=__builtin_ctz(mask);res=LINF;
 for(int j=i+1;j<k;j++)if(mask>>j&1)
  res=min(res,dOdd[i][j]+matchOdd(mask^(1<<i)
      ^(1<<j)));
 return res;
}
// answer = sumWeightAllEdges + matchOdd((1<<k)
      -1)
```

Nếu mọi bậc chẵn thì chỉ cần Euler tour. Nếu có đỉnh bậc lẻ: ghép cặp chúng với tổng shortest-path nhỏ nhất rồi nhân đôi các đường tương ứng. k nhỏ -> DP bitmask.

CSES Graph - mẫu nâng cao

BFS truy vết / Multi-source BFS

```
// parent de dung lai duong di
vector<int> par(n+1,-1); queue<int>q;
par[s]=0;q.push(s);
while(!q.empty()){
 int u=q.front();q.pop();
 for(int v:g[u])if(par[v]==-1)
  par[v]=u,q.push(v);
}
// multi-source: day tat ca nguon vao queue ngay
      tu dau
for(int s:src)d[s]=0,q.push(s);
```

Labyrinth/Message Route: lưu cha hoặc hướng đi để dựng đường. Monsters: BFS toàn bộ quái vật trước, chỉ cho A sang ô (x, y) khi $t_A + 1 < t_M[x][y]$.

Dijkstra biến thể: giảm giá / k đường / thống kê

```
// Flight Discount: d1 tu 1, dn tu n tren graph
      dao
ans=min(ans,d1[u]+w/2+dn[v]);

// k shortest: moi dinh duoc pop toi da k lan
while(!pq.empty()){
 auto [d,u]=pq.top();pq.pop();
 if(cnt[u]>=k)continue;cnt[u]++;
 if(u==n)ans.pb(d);
 for(auto [v,w]:g[u]) if(cnt[v]<k) pq.push({d+w,
      v});
}

// Investigation, khi nd == dist[v]:
ways[v]=(ways[v]+ways[u])%MOD;
mn[v]=min(mn[v],mn[u]+1);
mx[v]=max(mx[v],mx[u]+1);
```

Một Dijkstra có thể mang thêm trạng thái/metadata. Nếu cạnh được dùng một lần theo cách đặc biệt, thử 2 trạng thái hoặc Dijkstra thuận + đảo.

Bellman-Ford: chu trình ảnh hưởng đích

```
// High Score: chi vo han neu canh relax o vong
     n
// nam tren duong tu 1 den n
if(it==n && reach1[u] && reachN[v]) infinite=1;

// Cycle Finding: tim chu trinh am o bat ky
     component
vector<ll>d(n+1,0); // KHONG chi d[s]=0
int x=-1;
for(int it=1;it<=n;it++){
 x=-1;
 for(auto [u,v,w]:ed) if(d[v]>d[u]+w)
  d[v]=d[u]+w,par[v]=u,x=v;
}
if(x!=-1){for(int i=0;i<n;i++)x=par[x]; /* x nam
      trong cycle */}
```

High Score = Bellman-Ford cực đại + lọc reachability. Cycle Finding: khởi tạo mọi khoảng cách bằng 0 để coi như có siêu nguồn nối tới mọi đỉnh.

Topo + DAG DP

```
// longest path
for(int u:topo) if(dp[u]>NEG)
 for(int v:g[u]) if(dp[v]<dp[u]+1)
  dp[v]=dp[u]+1,par[v]=u;

// dem so duong
for(int u:topo)
 for(int v:g[u]) dp[v]=(dp[v]+dp[u])%MOD;
```

Course Schedule lấy topo. Longest Flight Route/Game Routes chỉ là DP chạy theo topo; tránh DFS DP khi cần thứ tự rõ ràng và truy vết.

Functional graph: peel cycle + binary lifting

```
// moi dinh co dung 1 canh ra
for(int i=1;i<=n;i++) indeg[to[i]]++;
queue<int>q;
for(int i=1;i<=n;i++)if(!indeg[i])q.push(i);
while(!q.empty()){
 int u=q.front();q.pop(); removed[u]=1;
 if(--indeg[to[u]]==0)q.push(to[u]);
}
// !removed = cac dinh tren cycle
// BFS tren reverse graph: depth[x]=khoang cach
     toi cycle

int jump(int u,long long k){
 for(int j=0;j<LOG;j++)if(k>>j&1)u=up[u][j];
 return u;
}
```

Planets Queries/Cycles: lưu component cycle, vị trí trên cycle, độ dài cycle và depth. Truy vấn $a \to b$: nếu b ở cây thì b phải là tổ tiên theo successor; nếu b ở cycle thì nhảy a tới cycle rồi cộng khoảng cách vòng.

SCC condensation / 2-SAT

```
// gom gia tri trong SCC
for(int u=1;u<=n;u++) sum[comp[u]]+=val[u];
// canh DAG neu comp[u]!=comp[v], sau do DP tren
     DAG

// literal 2-SAT: id(true)=2*x, id(false)=2*x+1
// (A or B) => (!A -> B), (!B -> A)
add(neg(A),B); add(neg(B),A);
if(comp[x]==comp[x^1]) impossible=1;
```

Coin Collector = SCC -> DAG -> longest path theo tổng coin SCC. Giant Pizza = 2-SAT; cực kỳ đáng nhớ vì cùng một SCC template giải được cả hai kiểu bài.

Euler có hướng / De Bruijn

```
// Euler path 1 -> n:
out[1]==in[1]+1;
in[n]==out[n]+1;
// moi dinh 2..n-1: in==out
// Sau Hierholzer phai co path.size()==m+1

// De Bruijn binary order n:
// node = (n-1)-bit, edge = them bit b
v=((u<<1)&mask)|b;
```

Mail Delivery: vô hướng cần bậc chẵn và dùng hết cạnh. Teleporters Path: kiểm tra indeg/outdeg. De Bruijn biến bài chuỗi thành Euler trên graph bit.

Hamiltonian Flights - bitmask DP

```
// dp[mask][v] = so duong 1 -> v dung dung tap
     mask
dp[1][0]=1;
for(int mask=1;mask<(1<<n);mask++){
 if(!(mask&1))continue;
 if((mask>>(n-1)&1)&&mask!=(1<<n)-1)continue;
 for(int v=1;v<n;v++)if(mask>>v&1){
  int pm=mask^(1<<v);
  for(int u:pre[v])if(pm>>u&1)
   dp[mask][v]=(dp[mask][v]+dp[pm][u])%MOD;
 }
}
```

$n \le 20$ là tín hiệu rất mạnh cho bitmask DP. Tối ưu quan trọng: không cho đích xuất hiện trước mask cuối.

Max-flow -> Min-cut / phân rã đường đi

```
// sau max-flow: BFS residual tu s
if(e.cap>0 && !vis[e.v]) vis[e.v]=1,q.push(e.v);
// canh goc (u,v) voi vis[u]!=vis[v] nam tren
     min-cut

// edge-disjoint paths: capacity moi canh = 1
// max-flow = so duong; sau do DFS theo cac canh
      co flow=1
```

Download Speed: max-flow. Police Chase: max-flow=min-cut, lấy partition bằng residual reachability. Distinct Routes: unit-capacity max-flow rồi phân rã flow thành từng đường 1->n.

Knight's Tour - Warnsdorff

```
// trong backtracking, sort nuoc di theo so nuoc
      tiep theo tang dan
vector<tuple<int,int,int>> cand;
for(each knight move) if(ok(nx,ny))
 cand.pb({degree(nx,ny),nx,ny});
sort(all(cand));
for(auto [d,nx,ny]:cand) if(dfs(nx,ny,step+1))
     return 1;
```

Heuristic Warnsdorff chọn ô "khó đi tiếp" trước; với bàn 8×8 chạy cực nhanh khi kết hợp backtracking.

CẤU TRÚC DỮ LIỆU

Fenwick / BIT

Ngữ cảnh dùng: Cần prefix/range sum động với point update, hoặc frequency để tìm k-th. Nhẹ hơn Segment Tree khi phép toán chỉ là cộng/trừ có inverse.

```
ll bit[N];
void add(int i,ll x){
  for(;i<=n;i+=i&-i)bit[i]+=x;
}
ll sum(int i){
  ll s=0;
  for(;i;i-=i&-i)s+=bit[i];
  return s;
}
ll sum(int l,int r){
  return sum(r)-sum(l-1);
}
```

Cộng tại một điểm + tổng tiền tố/tổng đoạn. $O(\log N)$. Nhớ: cập nhật i+=i&-i; truy vấn i-=i&-i.

Fenwick - cộng đoạn, truy vấn điểm

```
ll bit[N];
void add(int i,ll x){
  for(;i<=n;i+=i&-i)bit[i]+=x;
}
void rangeAdd(int l,int r,ll x){
  add(l,x);add(r+1,-x);
}
ll point(int i){
  ll s=0;
  for(;i;i-=i&-i)s+=bit[i];
  return s;
}
```

Dùng hiệu: cộng x cho $[l,r]$ bằng +x tại l, -x tại $r+1$. Cập nhật đoạn và hỏi một điểm đều $O(\log N)$.

Fenwick - cộng đoạn + tổng đoạn (2 BIT)

```
ll b1[N],b2[N];
void add(ll b[],int i,ll x){
  for(;i<=n;i+=i&-i)b[i]+=x;
}
void rangeAdd(int l,int r,ll x){
  add(b1,l,x);add(b1,r+1,-x);
  add(b2,l,x*(l-1));add(b2,r+1,-x*r);
}
ll pref(int i){
  ll s1=0,s2=0;
  for(int x=i;x;x-=x&-x)s1+=b1[x],s2+=b2[x];
  return s1*i-s2;
}
ll rangeSum(int l,int r){return pref(r)-pref(l
     -1);}
```

Range add + range sum: $O(\log N)$ mỗi thao tác. Công thức: $prefix(i) = i \cdot sum(B_1, i) - sum(B_2, i)$.

Fenwick 2D / tổng trên hình chữ nhật

```
ll bit[N][N];
void add2(int x,int y,ll v){
 for(int i=x;i<=n;i+=i&-i)
  for(int j=y;j<=m;j+=j&-j)bit[i][j]+=v;
}
ll sum2(int x,int y){ll s=0;
 for(int i=x;i;i-=i&-i)
  for(int j=y;j;j-=j&-j)s+=bit[i][j];
 return s;
}
ll rect(int x1,int y1,int x2,int y2){
 return sum2(x2,y2)-sum2(x1-1,y2)-sum2(x2,y1-1)+
     sum2(x1-1,y1-1);
}
```

Cập nhật điểm + tổng trên hình chữ nhật: $O(\log^2 N)$. Nếu chỉ tĩnh, ưu tiên prefix 2D để query $O(1)$.

Prefix 2D / Submatrix

```
for(int i=1;i<=n;i++)for(int j=1;j<=m;j++)
  pre[i][j]=a[i][j]+pre[i-1][j]+pre[i][j-1]-pre[
      i-1][j-1];
ll rect(int x1,int y1,int x2,int y2){
 return pre[x2][y2]-pre[x1-1][y2]-pre[x2][y1-1]+
     pre[x1-1][y1-1];
}
```

Ma trận tĩnh: build $O(NM)$, query hình chữ nhật $O(1)$.

Segment Tree

Ngữ cảnh dùng: Có update/query trên đoạn và phép gộp associative như sum/min/max/gcd. Dùng Lazy khi update cả đoạn; identity phải chọn đúng cho từng phép gộp.

Cập nhật điểm + tổng đoạn

```
ll seg[4*N];
void build(int id,int l,int r){
  if(l==r){seg[id]=a[l];return;}
  int m=(l+r)/2;
  build(id*2,l,m);build(id*2+1,m+1,r);
  seg[id]=seg[id*2]+seg[id*2+1];
}
void update(int id,int l,int r,int pos,ll val){
  if(l==r){seg[id]=val;return;}
  int m=(l+r)/2;
  if(pos<=m)update(id*2,l,m,pos,val);
  else update(id*2+1,m+1,r,pos,val);
  seg[id]=seg[id*2]+seg[id*2+1];
}
ll query(int id,int l,int r,int u,int v){
  if(v<l||r<u)return 0;
  if(u<=l&&r<=v)return seg[id];
  int m=(l+r)/2;
  return query(id*2,l,m,u,v)+query(id*2+1,m+1,r,
      u,v);
}
```

TỔNG: ngoài đoạn=0, gộp=+; NHỎ NHẤT: ngoài đoạn=INF, gộp=min; LỚN NHẤT: ngoài đoạn=-INF, gộp=max; GCD: ngoài đoạn=0, gộp=gcd. Xây $O(N)$, truy vấn/cập nhật $O(\log N)$.

SegTree - tìm phần tử thứ k theo tần suất

```
int kth(int id,int l,int r,int k){
  if(l==r)return l;
  int m=(l+r)/2;
  if(seg[id*2]>=k)
    return kth(id*2,l,m,k);
  return kth(id*2+1,m+1,r,k-seg[id*2]);
}
```

seg[id] lưu tổng tần suất. Điều kiện: $1 \le k \le seg[1]$. Đi trái nếu số lượng bên trái đủ chứa phần tử thứ k, ngược lại đi phải và trừ lượng bên trái. $O(\log N)$.

SegTree - tìm vị trí đầu tiên có giá trị $\ge x$

```
int firstGE(int id,int l,int r,int ql,ll x){
  if(r<ql || seg[id]<x)return -1;
  if(l==r)return l;
  int m=(l+r)/2;
  int p=firstGE(id*2,l,m,ql,x);
  if(p!=-1)return p;
  return firstGE(id*2+1,m+1,r,ql,x);
}
```

seg[id] phải lưu MAX trên đoạn. Trả vị trí đầu tiên $\ge ql$ có $a[pos] \ge x$. Nhờ bỏ cả node có max<x, chạy $O(\log N)$ trong dạng tìm kiếm đơn điệu trên cây. Có thể đổi MAX thành điều kiện gộp phù hợp.

Lazy - cộng đoạn + tổng đoạn

```
ll seg[4*N],lazy[4*N];
void push(int id,int l,int r){
  if(!lazy[id])return;
  seg[id]+=lazy[id]*(r-l+1);
  if(l!=r){
    lazy[id*2]+=lazy[id];
    lazy[id*2+1]+=lazy[id];
  }
  lazy[id]=0;
}
void upd(int id,int l,int r,int u,int v,ll x){
  push(id,l,r);
  if(v<l||r<u)return;
  if(u<=l&&r<=v){lazy[id]+=x;push(id,l,r);return
      ;}
  int m=(l+r)/2;
  upd(id*2,l,m,u,v,x);upd(id*2+1,m+1,r,u,v,x);
  seg[id]=seg[id*2]+seg[id*2+1];
}
ll qry(int id,int l,int r,int u,int v){
  push(id,l,r);
  if(v<l||r<u)return 0;
  if(u<=l&&r<=v)return seg[id];
  int m=(l+r)/2;
  return qry(id*2,l,m,u,v)+qry(id*2+1,m+1,r,u,v)
      ;
}
```

Persistent Segment Tree - k-th / median

Ngữ cảnh dùng: Cần truy vấn trên nhiều phiên bản/prefix, điển hình k-th/median trên subarray tĩnh. Thường cần nén tọa độ và đủ memory cho $O(N \log M)$ node.

```
int L[M],R[M],cnt[M],root[N],node;
int upd(int pre,int l,int r,int p){
  int cur=++node;
  L[cur]=L[pre];R[cur]=R[pre];
  cnt[cur]=cnt[pre]+1;
  if(l==r)return cur;
  int m=(l+r)/2;
  if(p<=m)L[cur]=upd(L[pre],l,m,p);
  else R[cur]=upd(R[pre],m+1,r,p);
  return cur;
}
int kth(int u,int v,int l,int r,int k){
  if(l==r)return l;
  int m=(l+r)/2;
  int c=cnt[L[v]]-cnt[L[u]];
  if(k<=c)return kth(L[u],L[v],l,m,k);
  return kth(R[u],R[v],m+1,r,k-c);
}
```

Nén tọa độ các giá trị. Xây phiên bản prefix: root[i]=upd(root[i-1],1,m,pos). Đoạn $[l,r]$: so sánh root[r] với root[l-1]; truy vấn k-th bằng hai root. Trung vị trái: $k = (r-l+2)/2$. Xây $O(N \log N)$, query $O(\log N)$.

PBDS / Ordered Set

Ngữ cảnh dùng: Cần order statistics online: số phần tử nhỏ hơn x, phần tử k-th. Với duplicate, lưu pair(value,id). Chỉ có trên GCC libstdc++.

```
#include <bits/extc++.h>
using namespace __gnu_pbds;
template<class T>
using ordered_set=tree<T,null_type,less<T>,
rb_tree_tag,tree_order_statistics_node_update>;

ordered_set<int>s;
s.insert(x);s.erase(x);
*s.find_by_order(k); // ậphn ứt ứth k, đánh ố
     ứt 0
s.order_of_key(x);   // đếm ố ậphn ứt < x
```

Tất cả $O(\log N)$. Có phần tử trùng: ordered_set<pair<int,int>>; chèn {value,id++}. Đếm $<x$: order_of_key({x,-INF}); đếm $\le x$: order_of_key({x,INF}).

Sparse Table / RMQ tĩnh

Ngữ cảnh dùng: Mảng không đổi và có rất nhiều query min/max/gcd. Query $O(1)$ kiểu overlap chỉ đúng cho phép toán idempotent.

```
int st[20][N],lg[N];
void buildST(vector<int>&a){
  int n=sz(a);for(int i=2;i<=n;i++)lg[i]=lg[i
      /2]+1;
  for(int i=0;i<n;i++)st[0][i]=a[i];
  for(int k=1;(1<<k)<=n;k++)
   for(int i=0;i+(1<<k)<=n;i++)
    st[k][i]=min(st[k-1][i],st[k-1][i+(1<<(k-1))
      ]);
}
int rmq(int l,int r){
  int k=lg[r-l+1];
  return min(st[k][l],st[k][r-(1<<k)+1]);
}
```

Preprocess $O(N \log N)$, truy vấn min/max/gcd $O(1)$ khi phép kết hợp idempotent. Mảng phải tĩnh.

DSU rollback

```
int p[N],sz[N];
vector<pair<int,int>> hist;
void initRB(int n){for(int i=1;i<=n;i++)p[i]=i,
     sz[i]=1;}
int findRB(int u){while(u!=p[u])u=p[u];return u
     ;}
int snap(){return sz(hist);}
bool joinRB(int a,int b){
  a=findRB(a);b=findRB(b);if(a==b){hist.pb
      ({-1,-1});return 0;}
  if(sz[a]<sz[b])swap(a,b);
  hist.pb({b,sz[a]});p[b]=a;sz[a]+=sz[b];return
      1;
}
void rollback(int s){while(sz(hist)>s){
  auto [b,old]=hist.back();hist.pop_back();if(b
      ==-1)continue;
  int a=p[b];sz[a]=old;p[b]=b;
}}
```

Dùng cho truy vấn kết nối offline/chia để trị theo thời gian. Không path-compression để hoàn tác được. Mỗi merge/rollback gần $O(\log N)$ với union-by-size.

Mo's Algorithm

Ngữ cảnh dùng: Nhiều query đoạn offline trên mảng tĩnh, và thêm/xóa một đầu đoạn cập nhật đáp án rẻ. Không phù hợp nếu query phải trả online.

```
int B=sqrt(n)+1;
sort(all(q),[&](auto A,auto C){
  int x=A.l/B,y=C.l/B;
  if(x!=y)return x<y;
  return (x&1)?A.r>C.r:A.r<C.r;
});
int L=0,R=-1;
for(auto qu:q){
  while(L>qu.l)add(--L); while(R<qu.r)add(++R);
  while(L<qu.l)del(L++); while(R>qu.r)del(R--);
  ans[qu.id]=cur;
}
```

Truy vấn đoạn offline khi add/del $O(1)$. Thường $O((N+Q)\sqrt{N})$. Có thể dùng thứ tự ziczac ở cột

phải để giảm hằng số.

Convex Hull Trick / đường thẳng

Ngữ cảnh dùng: DP có transition tối ưu hóa các hàm tuyến tính $mx + b$. CHT monotone cần thứ tự slope/query phù hợp; không chắc điều kiện thì dùng Li Chao.

Tối ưu dạng $dp[i] = \min_j(m_j x_i + b_j)$. Nếu slope và query x đơn điệu: deque CHT. Trường hợp bất kỳ: Li Chao Tree / LineContainer. Điểm giao thứ tự phải đơn điệu; dùng __int128 khi nhân hệ số lớn.

CÂY - DP - TÌM KIẾM - CHUỖI

HLD - Heavy Light Decomposition

Ngữ cảnh dùng: Cần update/query trên đường đi hoặc cây con với nhiều truy vấn động. HLD biến một path thành $O(\log N)$ đoạn để dùng Segment Tree/Fenwick.

```
int par[N],dep[N],szv[N],heavy[N],head[N],pos[N],timer;
int dfsSz(int u,int p){par[u]=p;szv[u]=1;int best=0;
 for(int v:adj[u])if(v!=p){dep[v]=dep[u]+1;int s=dfsSz(v,u);szv[u]+=s;
  if(s>best)best=s,heavy[u]=v;} return szv[u];}
void decomp(int u,int h){head[u]=h;pos[u]=++timer;
 if(heavy[u])decomp(heavy[u],h);
 for(int v:adj[u])if(v!=par[u]&&v!=heavy[u])decomp(v,v);
}
// query path u-v: ảnhy theo head, ỗmi đạon query trên seg[pos]
```

Biến đường đi cây thành $O(\log N)$ đoạn liên tiếp; ghép với Segment Tree -> query/update path thường $O(\log^2 N)$.

Euler Tour / flatten cây

```
int tin[N],tout[N],ord[N],timer;
void dfsTour(int u,int p){
 tin[u]=++timer;ord[timer]=u;
 for(int v:adj[u])if(v!=p)dfsTour(v,u);
 tout[u]=timer;
}
// subtree(u) <=> đạon [tin[u],tout[u]]
```

Biến truy vấn subtree thành truy vấn đoạn; ghép Fenwick/Segment Tree. u là tổ tiên của v iff $tin[u] \le tin[v] \le tout[u]$.

Virtual Tree / cây ảo

Khi mỗi query chỉ đánh dấu K đỉnh trên cây lớn: sort theo Euler tin, thêm LCA của các cặp kề, sort lại rồi dựng bằng stack. Cây ảo có $O(K)$ đỉnh, giúp DP/query theo tập đỉnh nhanh hơn.

Mảng / DP / tìm kiếm

Tổng tiền tố

```
pref[0]=0;
for(int i=1;i<=n;i++)pref[i]=pref[i-1]+a[i];
// ổtng(l,r)=pref[r]-pref[l-1]
```

Tìm kiếm nhị phân trên đáp án

```
ll lo=L,hi=R,ans=R;
while(lo<=hi){
  ll mid=(lo+hi)/2;
  if(ok(mid))ans=mid,hi=mid-1;
  else lo=mid+1;
}
```

Cần điều kiện đơn điệu: sai...sai, đúng...đúng (hoặc ngược lại).

LIS - dãy con tăng nghiêm ngặt

```
vector<int>d;
for(int x:a){
  auto it=lower_bound(all(d),x);
  if(it==d.end())d.pb(x);
  else *it=x;
}
// đáp án = sz(d)
```

$O(N \log N)$. LIS không giảm: dùng upper_bound.

Ba lô 0/1

```
vector<ll>dp(W+1);
for(auto [w,val]:item)
  for(int j=W;j>=w;j--)
    dp[j]=max(dp[j],dp[j-w]+val);
```

Duyệt sức chứa giảm để không dùng lại một món.

Hai con trỏ / cửa sổ trượt

```
int l=0;
for(int r=0;r<n;r++){
  add(a[r]);
  while(!valid())remove(a[l++]);
  // [l..r] đang ợhp ệl
}
```

Sliding Window - các mẫu CSES

Ngữ cảnh dùng: Bài hỏi giá trị của mọi cửa sổ độ dài cố định hoặc hai con trỏ có biên di chuyển đơn điệu. Chọn deque/multiset/Fenwick theo statistic cần duy trì.

Tổng / XOR - cập nhật $O(1)$

```
ll sum=0;int xr=0;
for(int i=0;i<n;i++){
 sum+=a[i]; xr^=a[i];
 if(i>=k) sum-=a[i-k],xr^=a[i-k];
 if(i>=k-1){/* sum, xr cua cua so */}
}
```

Phép có nghịch đảo như tổng/XOR: thêm phần tử mới và bỏ phần tử cũ trực tiếp.

Minimum/Maximum - monotonic deque $O(N)$

```
deque<int>q;
for(int i=0;i<n;i++){
 while(!q.empty()&&a[q.back()]>=a[i])q.pop_back();
 q.push_back(i);
 while(q.front()<=i-k)q.pop_front();
 if(i>=k-1) ans.pb(a[q.front()]);
}
```

Đổi >= thành <= để lấy maximum. Mỗi chỉ số vào/ra deque tối đa một lần.

OR - đếm bit $O(BN)$

```
int bc[31]={};
void updOr(int x,int d){
 for(int b=0;b<31;b++)if(x>>b&1)bc[b]+=d;
}
int getOr(){
 int r=0;for(int b=0;b<31;b++)if(bc[b])r|=1<<b;
 return r;
}
```

OR không có phép "trừ", nên lưu số phần tử đang bật từng bit. Cũng dùng được cho AND bằng cách so bc[b]==k.

Distinct / Mode

```
unordered_map<int,int>f;
int distinct=0;
void addD(int x){if(f[x]++==0)distinct++;}
void delD(int x){if(--f[x]==0)distinct--;}

map<int,int>fm;
set<pair<int,int>>best; // {-freq,value}
void updMode(int x,int d){
 if(fm[x])best.erase({-fm[x],x});
 fm[x]+=d;
 if(fm[x])best.insert({-fm[x],x});
}
// mode = best.begin()->second (hòa -> ỏnh ấnht)
```

MEX cửa sổ

```
vector<int>cnt(k+1);set<int>miss;
for(int x=0;x<=k;x++)miss.insert(x);
void addM(int x){if(x<=k&&cnt[x]++==0)miss.erase(x);}
void delM(int x){if(x<=k&&--cnt[x]==0)miss.insert(x);}
// mex = *miss.begin()
```

MEX của cửa sổ kích thước k luôn không vượt quá k, nên bỏ qua mọi giá trị $> k$.

Median + Cost - hai multiset

```
multiset<ll>lo,hi;ll slo=0,shi=0;int K;
void balance(){
 while(sz(lo)>(K+1)/2){ll x=*prev(lo.end());lo.erase(prev(lo.end()));slo-=x;hi.insert(x);shi+=x;}
 while(sz(lo)<(K+1)/2){ll x=*hi.begin();hi.erase(hi.begin());shi-=x;lo.insert(x);slo+=x;}
}
void addW(ll x){
 if(lo.empty()||x<=*prev(lo.end()))lo.insert(x),slo+=x;
 else hi.insert(x),shi+=x; balance();
}
void delW(ll x){
 auto it=lo.find(x);if(it!=lo.end())slo-=x,lo.erase(it);
 else it=hi.find(x),shi-=x,hi.erase(it); balance();
}
ll median(){return *prev(lo.end());}
ll cost(){ll m=median();return m*sz(lo)-slo+shi-m*sz(hi);}
```

Giữ |lo|=(k+1)/2; median là phần tử lớn nhất của lo. Cost = tổng $|a_i - m|$ nhờ hai tổng slo, shi.

Inversions - nén tọa độ + Fenwick

```
ll inv=0;int cur=0;
void addRight(int r){
 inv+=cur-sum(r); add(r,1); cur++;
}
void removeLeft(int r){
 inv-=sum(r-1); add(r,-1); cur--;
}
```

Khi thêm bên phải, số nghịch thế mới = số phần tử hiện tại $> x$. Khi bỏ phần tử trái, trừ số phần tử còn lại $< x$.

Advertisement - histogram động theo cửa sổ

```
// bien lon nhat ma a[i] la minimum
stack<int>st;
for(int i=0;i<n;i++){
 while(!st.empty()&&a[st.top()]>=a[i])st.pop();
 L[i]=st.empty()?0:st.top()+1;st.push(i);
}
while(!st.empty())st.pop();
for(int i=n-1;i>=0;i--){
 while(!st.empty()&&a[st.top()]>=a[i])st.pop();
 R[i]=st.empty()?n-1:st.top()-1;st.push(i);
}
```

Với mỗi cột i, diện tích trong cửa sổ bắt đầu tại s là $a_i \cdot (\min(s+k-1, R_i) - \max(s, L_i) + 1)$: hàm theo s gồm 3 đoạn tuyến tính. Bài Sliding Window Advertisement dùng các đoạn này + Li Chao/CHT đoạn để lấy max $O(N \log N)$.

QHD - thiết kế trạng thái và mẫu chuyển

Ngữ cảnh dùng: Bài có các trạng thái con lặp lại và có thứ tự phụ thuộc rõ. Chọn state đủ thông tin nhưng không thừa; kiểm tra memory trước khi code.

QHD 1 chiều / nhiều trạng thái

```
// dp[i] = dap an cho prefix i
vector<ll>dp(n+1,INF);dp[0]=0;
for(int i=0;i<n;i++)
 for(auto [to,cost]:trans(i))
  dp[to]=min(dp[to],dp[i]+cost);

// dp[pos][state], co the cuon 2 hang
for(int pos=0;pos<n;pos++){
 fill(all(ndp),BAD);
 for(int st=0;st<S;st++)for(auto [nst,val]:go(st,pos))
  ndp[nst]=combine(ndp[nst],dp[st],val);
 dp.swap(ndp);
}
```

Chọn state sao cho biết state hiện tại là đủ quyết định bước kế tiếp. Nếu chỉ phụ thuộc hàng trước, cuộn mảng để giảm bộ nhớ.

Thứ tự vòng for trong QHD - rất dễ WA

```
// 0/1 knapsack: moi vat chi dung 1 lan -> j GIAM
for(int i=0;i<n;i++)
 for(int j=W;j>=w[i];j--)
  dp[j]=max(dp[j],dp[j-w[i]]+v[i]);

// complete knapsack: duoc dung lai -> j TANG
for(int i=0;i<n;i++)
 for(int j=w[i];j<=W;j++)
  dp[j]=max(dp[j],dp[j-w[i]]+v[i]);

// coin combinations: coin ngoai -> khong tinh thu tu
for(int c:coin)for(int s=c;s<=X;s++)dp[s]+=dp[s-c];
// ordered sequences: sum ngoai -> co tinh thu tu
for(int s=1;s<=X;s++)for(int c:coin)if(s>=c)dp[s]+=dp[s-c];
```

Quy tắc: nếu transition dp[A] <- dp[B], hãy hỏi B phải là giá trị CŨ hay được phép là giá trị VỪA cập nhật. Không được tái dùng trong cùng lượt -> thường duyệt ngược; được tái dùng -> thường duyệt xuôi.

Dạng	Thứ tự nên duyệt
0/1 Knapsack	item → capacity giảm
Complete Knapsack	item → capacity tăng
Coin combinations	coin → sum tăng
Coin ordered	sum → coin
Grid từ trên/trái	i tăng → j tăng
Grid từ dưới/phải	i giảm → j giảm
Interval DP	len → l → split k
DAG DP	topo → cạnh
Tree subtree DP	DFS con trước rồi gộp
Bitmask assignment	mask → choice
TSP bitmask	mask → last → next
Submask DP	mask → submask
SOS DP	bit → mask
Digit DP	pos → state → digit

Grid DP

```
dp[0][0]=1;
for(int i=0;i<n;i++)for(int j=0;j<m;j++){
 if(block[i][j])continue;
 if(i)dp[i][j]+=dp[i-1][j];
 if(j)dp[i][j]+=dp[i][j-1];
 dp[i][j]%=MOD;
}
```

Interval DP

```
for(int len=1;len<=n;len++)
 for(int l=0;l+len-1<n;l++){
  int r=l+len-1;
  for(int k=l;k<r;k++)
   dp[l][r]=min(dp[l][r],dp[l][k]+dp[k+1][r]+cost(l,k,r));
 }
```

Dạng "gộp đoạn", "xóa hai đầu", "đặt dấu ngoặc" thường là $dp[l][r]$. Nếu chuyển theo điểm chia k -> thường $O(N^3)$ trước tối ưu.

Bitmask DP - gán / TSP

```
// assignment: i = so bit da chon
vector<ll>dp(1<<n,LINF);dp[0]=0;
for(int mask=0;mask<(1<<n);mask++){
 int i=__builtin_popcount((unsigned)mask);
 for(int j=0;j<n;j++)if(!(mask>>j&1))
  dp[mask|1<<j]=min(dp[mask|1<<j],dp[mask]+cost[
      i][j]);
}

// TSP: dp[mask][v] = min cost, ket thuc tai v
```

$n \le 20$ là tín hiệu mạnh cho $O(n2^n)$. Có thể lưu predecessor để truy vết.

Duyệt submask / SOS DP

```
for(int sub=mask;;sub=(sub-1)&mask){
 // sub la tap con cua mask
 if(sub==0)break;
}

// SOS: f[mask] = tong ban dau tren submask
for(int b=0;b<n;b++)
 for(int mask=0;mask<(1<<n);mask++)
  if(mask>>b&1)f[mask]+=f[mask^(1<<b)];
```

Duyệt mọi submask của mọi mask tổng cộng $O(3^n)$. SOS DP tính tổng trên mọi submask trong $O(n2^n)$.

Digit DP - khung trạng thái

```
ll dfs(int pos,bool tight,bool started,int state
     ){
 if(pos==len)return good(state,started);
 if(!tight&&memo[pos][started][state]!=-1)return
      memo[pos][started][state];
 ll ans=0,lim=tight?dig[pos]:9;
 for(int d=0;d<=lim;d++)
  ans+=dfs(pos+1,tight&&d==lim,started||d,state2
      (state,d));
 if(!tight)memo[pos][started][state]=ans;
 return ans;
}
// count[L,R] = solve(R)-solve(L-1)
```

State thường gồm vị trí, tight, đã bắt đầu hay chưa, và thuộc tính cần theo dõi (sum mod, số chữ số, automaton state...).

Monotonic Stack / nén tọa độ

```
// ảphn ứt ỏnh ơhn ằgn ấnht bên trái
stack<int>st;
for(int i=0;i<n;i++){
  while(!st.empty()&&a[st.top()]>=a[i])st.pop();
  prv[i]=st.empty()?-1:st.top();
  st.push(i);
}

vector<int>v=a;
sort(all(v));v.erase(unique(all(v)),v.end());
int id=lower_bound(all(v),x)-v.begin();
```

Tối ưu DP - Knuth / Chia để trị

D&C optimization: $dp[i] = \min_{k \in [L_i, R_i]} f(i,k)$ và vị trí tối ưu không giảm theo i -> tính một hàng trong khoảng $O(N \log N)$ hoặc $O(N)$ tùy cost. Knuth: $dp[l][r] = \min_k(dp[l][k] + dp[k][r]) + C(l,r)$ và $opt[l][r-1] \le opt[l][r] \le opt[l+1][r]$ -> từ $O(N^3)$ xuống $O(N^2)$.

KMP / prefix function

Ngữ cảnh dùng: Một pattern cần tìm trong text, hoặc bài về border/prefix-suffix/period. $O(N+M)$ và không có collision.

```
vector<int> pi(string s){
  int n=s.size();vector<int>p(n);
  for(int i=1;i<n;i++){
    int j=p[i-1];
    while(j&&s[i]!=s[j])j=p[j-1];
    if(s[i]==s[j])j++;
    p[i]=j;
  }
  return p;
}
```

$p[i]$ = độ dài tiền tố thực sự dài nhất đồng thời là hậu tố của `s[0..i]`. Ghép mẫu: chạy hàm tiền tố trên `mẫu + ký tự phân cách + văn bản`.

Z-function / Hash / Manacher

Ngữ cảnh dùng: Z cho so khớp prefix với mọi suffix; hash cho so sánh substring nhanh; Manacher cho palindrome radius/longest palindrome trong $O(N)$.

Z-function

```
vector<int> zfunc(string s){
  int n=sz(s),l=0,r=0;vector<int>z(n);
  for(int i=1;i<n;i++){
    if(i<=r)z[i]=min(r-i+1,z[i-l]);
    while(i+z[i]<n&&s[z[i]]==s[i+z[i]])z[i]++;
    if(i+z[i]-1>r)l=i,r=i+z[i]-1;
  }return z;
}
```

$z[i]$ = độ dài tiền tố dài nhất khớp với chuỗi bắt đầu tại i. Ghép extttpattern + '#' + text để tìm pattern.

Rolling Hash 1 modulo

```
ll pw[N],hs[N];
void hashBuild(string s,ll B){
  pw[0]=1;
  for(int i=0;i<sz(s);i++)pw[i+1]=pw[i]*B%MOD,hs
      [i+1]=(hs[i]*B+s[i])%MOD;
}
ll getHash(int l,int r){ // [l,r)
  return (hs[r]-hs[l]*pw[r-l]%MOD+MOD)%MOD;
}
```

Có xác suất va chạm; nếu bài nhạy, dùng hai modulo hoặc hash 64-bit.

Manacher - palindrome dài nhất quanh tâm

```
vector<int> manacherOdd(string s){
  int n=sz(s),l=0,r=-1;vector<int>d(n);
  for(int i=0;i<n;i++){
    int k=i>r?1:min(d[l+r-i],r-i+1);
    while(i-k>=0&&i+k<n&&s[i-k]==s[i+k])k++;
    d[i]=k--;
    if(i+k>r)l=i-k,r=i+k;
  }return d;
}
```

$d[i]$ là bán kính palindrome lẻ (tính cả tâm). Bản chẵn tương tự; có thể chèn ký tự ngăn cách để dùng một bản duy nhất.

Chuỗi nâng cao - nhóm bài CSES/ICPC

Ngữ cảnh dùng: Bài có nhiều pattern, nhiều truy vấn substring, thứ tự từ điển hoặc palindrome khác nhau. Chọn Trie/Aho/SA/SAM/Eertree theo loại truy vấn.

Word Combinations - Trie + DP

```
int tr[N][26],term[N],nodes=1;
void addWord(string s){
 int u=0;
 for(char c:s){
  int x=c-'a';
  if(!tr[u][x])tr[u][x]=nodes++;
  u=tr[u][x];
 }
 term[u]++;
}
ll wordComb(string s){
 int n=sz(s);vector<ll>dp(n+1);dp[0]=1;
 for(int i=0;i<n;i++)if(dp[i]){
  int u=0;
  for(int j=i;j<n;j++){
   int x=s[j]-'a';if(!tr[u][x])break;
   u=tr[u][x];
   if(term[u])dp[j+1]=(dp[j+1]+dp[i]*term[u])%
      MOD;
  }
 }
 return dp[n];
}
```

Dạng tách chuỗi thành các từ trong từ điển. Trie tránh thử mọi từ tại mọi vị trí.

Borders - prefix function

```
vector<int> borders(string s){
 auto pi=prefixFunction(s);vector<int>ans;
 for(int x=pi.back();x;x=pi[x-1])ans.pb(x);
 reverse(all(ans));return ans;
}
```

Border = prefix đồng thời là suffix. Chuỗi các border đi theo $x = \pi[x-1]$.

Periods - Z function

```
vector<int> periods(string s){
 auto z=zFunction(s);int n=sz(s);vector<int>ans;
 for(int p=1;p<=n;p++)
  if(p==n || z[p]>=n-p)ans.pb(p);
 return ans;
}
```

Period p hợp lệ khi $s[p..]$ trùng prefix dài ít nhất $n-p$.

Required Substring - KMP automaton + DP

```
// nxt[state][c] = độ dài prefix pattern ớkhp
     sau khi thêm c
for(int st=0;st<m;st++)for(int c=0;c<26;c++){
 string t=p.substr(0,st)+char('A'+c);
 int j=min(m,st+1);
 while(j && t.substr(sz(t)-j)!=p.substr(0,j))j
     --;
 nxt[st][c]=j;
}
// dp[pos][state][found]; khi nxt==m thì found=1
```

Dạng đếm chuỗi chứa/không chứa pattern: automaton KMP + DP theo vị trí. Khi code thật nên build automaton từ π để tránh substring chậm.

Aho-Corasick - nhiều pattern

```
int go[N][26],linkA[N],out[N],nodesA=1;
void addPat(string s,int id){
 int u=0;
 for(char c:s){int x=c-'a';if(!go[u][x])go[u][x
     ]=nodesA++;u=go[u][x];}
 out[u]=id;
}
void buildAho(){
 queue<int>q;
 for(int c=0;c<26;c++)if(go[0][c])q.push(go[0][c
     ]);
 while(!q.empty()){
  int u=q.front();q.pop();
  for(int c=0;c<26;c++){
   int v=go[u][c];
   if(v){linkA[v]=go[linkA[u]][c];q.push(v);}
   else go[u][c]=go[linkA[u]][c];
  }
 }
}
```

Finding Patterns / Counting Patterns / Pattern Positions: Aho-Corasick. Để đếm số lần, tăng cnt[state] khi quét text rồi cộng ngược theo failure tree; vị trí đầu tiên lưu min.

Distinct Subsequences - DP last occurrence

```
ll distinctSubseq(string s){
 vector<ll>dp(sz(s)+1);vector<int>last(26,-1);dp
     [0]=1;
 for(int i=0;i<sz(s);i++){
  dp[i+1]=2*dp[i]%MOD;
  int c=s[i]-'a';
  if(last[c]!=-1)dp[i+1]=(dp[i+1]-dp[last[c]]+
     MOD)%MOD;
  last[c]=i;
 }
 return (dp[sz(s)]-1+MOD)%MOD; // ỏb ỗchui ỗrng
}
```

Suffix Array / LCP - các bài substring

```
vector<int> suffixArray(string s){
 s+='$';int n=sz(s);vector<int>p(n),c(n);
 vector<pair<char,int>>a(n);
 for(int i=0;i<n;i++)a[i]={s[i],i};sort(all(a));
 for(int i=0;i<n;i++)p[i]=a[i].sd;
 c[p[0]]=0;for(int i=1;i<n;i++)c[p[i]]=c[p[i
     -1]]+(a[i].ft!=a[i-1].ft);
 for(int k=0;(1<<k)<n;k++){
  for(int i=0;i<n;i++)p[i]=(p[i]-(1<<k)+n)%n;
  vector<int>cnt(n),np(n),nc(n);
  for(int x:c)cnt[x]++;for(int i=1;i<n;i++)cnt[i
     ]+=cnt[i-1];
  for(int i=n-1;i>=0;i--)np[--cnt[c[p[i]]]]=p[i
     ];p=np;
  nc[p[0]]=0;for(int i=1;i<n;i++){
   ii A={c[p[i-1]],c[(p[i-1]+(1<<k))%n]};
   ii B={c[p[i]],c[(p[i]+(1<<k))%n]};
   nc[p[i]]=nc[p[i-1]]+(A!=B);
  } c=nc;
 }
 p.erase(p.begin());return p;
}
vector<int> LCP(string s,vector<int>sa){
 int n=sz(s),k=0;vector<int>r(n),lcp(max(0,n-1))
     ;
 for(int i=0;i<n;i++)r[sa[i]]=i;
 for(int i=0;i<n;i++){
  if(r[i]==n-1){k=0;continue;}int j=sa[r[i]+1];
  while(i+k<n&&j+k<n&&s[i+k]==s[j+k])k++;
  lcp[r[i]]=k;if(k)k--;
 } return lcp;
}
```

Distinct Substrings: $n(n+1)/2 - \sum LCP$. Repeating Substring: max LCP. Substring Order I: suffix $SA[i]$ đóng góp $n - SA[i] - LCP_{\text{trước}}$ substring mới. Build $O(N \log N)$, LCP $O(N)$.

Suffix Automaton - Order II / Distribution

```
int nxtS[2*N][26],linkS[2*N],lenS[2*N],occS[2*N
     ],szS=1,lastS=0;
void samAdd(char ch){
 int c=ch-'a',cur=szS++;lenS[cur]=lenS[lastS]+1;
     occS[cur]=1;
 int p=lastS;
 while(p!=-1&&!nxtS[p][c])nxtS[p][c]=cur,p=linkS
     [p];
 if(p==-1)linkS[cur]=0;
 else{
  int q=nxtS[p][c];
  if(lenS[p]+1==lenS[q])linkS[cur]=q;
  else{
   int cl=szS++;memcpy(nxtS[cl],nxtS[q],sizeof
      nxtS[q]);
   lenS[cl]=lenS[p]+1;linkS[cl]=linkS[q];occS[cl
      ]=0;
   while(p!=-1&&nxtS[p][c]==q)nxtS[p][c]=cl,p=
      linkS[p];
   linkS[q]=linkS[cur]=cl;
  }
 } lastS=cur;
}
void initSAM(){linkS[0]=-1;}
```

Mỗi state v tạo $len[v] - len[link[v]]$ substring khác nhau. Muốn số lần xuất hiện, sort state theo len giảm rồi cộng $occ[v]$ vào $occ[link[v]]$. Distribution: cộng +1 trên đoạn độ dài $[len(link[v])+1, len(v)]$ bằng mảng hiệu.

Eertree - All Palindromes

```
int goP[N][26],linkP[N],lenP[N],nodesP=2,lastP
    =1;
string curP="#";
void initPal(){lenP[0]=-1;linkP[0]=0;lenP[1]=0;
    linkP[1]=0;}
int addPal(char ch){
 curP+=ch;int pos=sz(curP)-1,c=ch-'a',u=lastP;
 while(curP[pos-1-lenP[u]]!=ch)u=linkP[u];
 if(!goP[u][c]){
  int v=nodesP++;lenP[v]=lenP[u]+2;
  int x=linkP[u];
  while(curP[pos-1-lenP[x]]!=ch)x=linkP[x];
  linkP[v]=(lenP[v]==1?1:goP[x][c]);goP[u][c]=v;
 }
 lastP=goP[u][c];return lenP[lastP];
}
```

Sau khi thêm $s[i]$, giá trị trả về là độ dài palindrome dài nhất kết thúc tại i. Longest Palindrome đơn thuần: Manacher ngắn hơn.

Palindrome Queries động - Fenwick hash

```
ll pw[N],invPw[N],bitF[N],bitR[N];int nH;
void addBit(ll bit[],int i,ll v){for(;i<=nH;i+=i
    &-i)bit[i]=(bit[i]+v)%MOD;}
ll sumBit(ll bit[],int i){ll s=0;for(;i;i-=i&-i)
    s=(s+bit[i])%MOD;return s;}
ll rangeBit(ll bit[],int l,int r){return (sumBit
    (bit,r)-sumBit(bit,l-1)+MOD)%MOD;}
ll getHF(int l,int r){return rangeBit(bitF,l,r)*
    invPw[l]%MOD;}
ll getHR(int l,int r){return rangeBit(bitR,nH-r
    +1,nH-l+1)*invPw[nH-r+1]%MOD;}
// ưlu s[i]*BASE^i; update: ộcng (new-old)*BASE^
    i vào bitF,
// và ịv trí n-i+1 vào bitR. Palindrome [l,r]
    iff getHF(l,r)==getHR(l,r).
```

Dùng double hash hoặc unsố nguyên có dấu 64-bit hash nếu sợ va chạm. Mỗi update/query $O(\log N)$.

String Transform / Inverse BWT

```
string inverseBWT(string L){
 int n=sz(L);vector<pair<char,int>>F(n),A(n);int
     cnt[256]={};
 for(int i=0;i<n;i++)A[i]={L[i],cnt[(unsigned
     char)L[i]]++};
 F=A;sort(all(F));map<pair<char,int>,int>pos;
 for(int i=0;i<n;i++)pos[F[i]]=i;
 vector<int>lf(n);for(int i=0;i<n;i++)lf[i]=pos[
     A[i]];
 int p=0;while(L[p]!='$'&&L[p]!='#')p++;
 string s;for(int k=0;k<n-1;k++){p=lf[p];s+=L[p
     ];}
 reverse(all(s));return s;
}
```

Dùng khi để cho Burrows-Wheeler transform có sentinel duy nhất (thường $ hoặc #). Inverse Suffix Array là constructive riêng, không có template ngắn dùng chung.

Chuỗi nâng cao từ KACTL

Suffix Array: sắp xếp mọi hậu tố, dùng LCP cho truy vấn lặp/so sánh chuỗi; build thường $O(N \log N)$. Aho-Corasick: tìm nhiều pattern cùng lúc $O(|text| + \#matches)$. Min Rotation: xoay từ điển nhỏ nhất $O(N)$. Suffix Tree/Ukkonen mạnh nhưng dài, chỉ mang nếu quen tay.

Minimum Rotation - xoay từ điển nhỏ nhất

```
int minRotation(string s){
 int n=sz(s),i=0,j=1,k=0;s+=s;
 while(i<n&&j<n&&k<n){
  if(s[i+k]==s[j+k]){k++;continue;}
  if(s[i+k]>s[j+k])i=i+k+1;else j=j+k+1;
  if(i==j)j++;k=0;
 }
 return min(i,j);
}
```

Booth $O(N)$. Kết quả là vị trí bắt đầu phép xoay nhỏ nhất theo thứ tự từ điển.

Tập khoảng / phủ khoảng

KACTL có cấu trúc để thêm/xóa và gộp các khoảng rời nhau. Dạng greedy phủ đoạn: sort theo đầu trái, trong mọi interval bắt đầu trước điểm hiện tại chọn interval vươn xa nhất; lặp tới khi phủ xong hoặc thất bại.

Nhận diện DP nâng cao

- Digit DP: đếm số $\le X$ theo chữ số, trạng thái thường gồm vị trí / tight / thuộc tính.
- Interval DP: trạng thái $dp[l][r]$, chuyển bằng cách tách tại k hoặc loại đầu/cuối.
- DAG DP: topo trước, sau đó chuyển DP theo thứ tự topo.
- Rerooting DP trên cây: tính đáp án gốc 1, rồi chuyển đáp án khi đổi gốc qua cạnh trong $O(1)$ hoặc $O(\deg)$.
- SOS/Subset DP: trạng thái theo mask, thường $O(N2^N)$ hoặc $O(N \log N)$ với biến đổi phù hợp.

Thiết kế state DP - mẹo CP3

Các tham số state hay gặp: vị trí i; cặp chỉ số (i, j); đoạn (l, r); đỉnh DAG; knapsack value; tập nhỏ dạng bitmask. Bài khó thường ghép 2-3 tham số.

```
// state co the am -> offset
int id=value+SHIFT; // SHIFT >= -minValue


// sparse memo khi ảbng ấrt ưtha
unordered_map<long long,ll> memo;
long long key=((long long)i<<32)^state;

// drop 1 parameter neu suy ra duoc
// vd area(mask)=sum[mask], h=area(mask)/w
```

MLE/TLE: thử nén state, dùng memo sparse, hoặc bỏ một tham số nếu có thể khôi phục từ các tham số khác. Đây thường là khác biệt giữa state $O(N^3)$ và $O(N^2)$.

Phân rã bài khó - pattern ghép thuật toán

```
// binary search answer + predicate don dieu
while(lo<hi){
 ll mid=(lo+hi)/2;
 if(ok(mid))hi=mid;else lo=mid+1;
}
```

Các combo đáng nhớ: Binary Search Answer + Greedy/-Connectivity/SSSP/Flow/Matching/Geometry; Sieve + Prefix Sum để trả nhiều query; BFS/APSP nén grid thành graph nhỏ rồi DP TSP; SCC contraction -> DAG rồi DP; toán tạo cạnh của implicit graph rồi BFS/Dijkstra. Khi gặp bài lạ, thử tách thành 2 thành phần quen thuộc.

Cốt lõi code kinh điển - tra nhanh

KMP / Border / Z

```
// KMP match
string t=pat+"#"+text;auto p=pi(t);
for(int i=0;i<sz(t);i++)if(p[i]==sz(pat)){} //
     match
// ọmi border
for(int x=p.back();x;x=p[x-1])ans.pb(x);
reverse(all(ans));
```

KMP mismatch: j=pi[j-1]. Border tiếp theo cũng đi theo liên kết này. Z[i] = prefix dài nhất khớp tại i.

Suffix / palindrome cốt lõi

Distinct substrings bằng SA: $n(n+1)/2 - \sum LCP$. Longest repeating: $\max LCP$. SAM: $\#distinct = \sum_{v \neq root}(len[v] - len[link[v]])$. Longest palindrome: Manacher. Minimal rotation: Booth.

Trie/Aho chọn nhanh

Một pattern: KMP/Z. Nhiều pattern: Aho-Corasick. Tách chuỗi theo từ điển: Trie + DP vị trí. Palindrome có update: hash thuận + hash đảo + Fenwick/SegTree.

Graph / DS cốt lõi

```
// Dijkstra relax
if(du>d[u])continue;
for(auto [v,w]:adj[u])if(d[v]>du+w)
 d[v]=du+w,q.push({d[v],v});
// Fenwick
for(;i<=n;i+=i&-i)bit[i]+=x;
for(;i;i-=i&-i)s+=bit[i];

// DSU
int Find(int u){return u==p[u]?u:p[u]=Find(p[u])
    ;}
// Persistent k-th: u=root[l-1], v=root[r]
int c=cnt[L[v]]-cnt[L[u]];
if(k<=c) goLeft(); else k-=c,goRight();
```

Geometry cốt lõi

```
ll cross(P a,P b){return a.ft*b.sd-a.sd*b.ft;}
ll orient(P a,P b,P c){
 return cross({b.ft-a.ft,b.sd-a.sd},{c.ft-a.ft,c
     .sd-a.sd});
}
ld x=cx+r*cos(t),y=cy+r*sin(t);
ld ang=atan2(y-cy,x-cx);
```

Cross > 0: trái/CCW; < 0: phải/CW; $= 0$: thẳng hàng. Hull: pop khi `orient<=0` nếu bỏ điểm thẳng hàng. Shoelace: $2A = |\sum cross(p_i, p_{i+1})|$. C++ dùng radian.

20 từ khóa cuối

KMP / Z / Manacher / Trie / Aho / Suffix Array+LCP / SAM / binpow / gcd / C(n,k) / BFS / Dijkstra / DSU / Fenwick / Segment Tree / Persistent k-th / cross / orient / Convex Hull / sin-cos-atan2.

SỐ HỌC - TỔ HỢP

BINPOW / GCD / sàng

Ngữ cảnh dùng: Toán số cơ bản: lũy thừa với exponent lớn, gcd/lcm, hoặc cần prime/factor cho nhiều số trong một giới hạn vừa.

BINPOW - lũy thừa nhị phân

```
ll binpow(ll a,ll b,ll mod){
  ll r=1;
  while(b){
    if(b&1)r=r*a%mod;
    a=a*a%mod;b>>=1;
  }
  return r;
}
```

Dùng binpow(a,b,MOD) để tính $a^b \bmod MOD$. $O(\log b)$. Nếu MOD là số nguyên tố và $a \not\equiv 0$, $a^{-1} = a^{MOD-2} \pmod{MOD}$.

GCD / LCM

```
ll gcd2(ll a,ll b){
  while(b){a%=b;swap(a,b);}return a;
}
ll lcm2(ll a,ll b){return a/gcd2(a,b)*b;}
```

Sàng nguyên tố

```
vector<int>pr;bool comp[N];
void sieve(int n){
  for(int i=2;i<=n;i++)if(!comp[i]){
    pr.pb(i);
    if(1LL*i*i<=n)
      for(int j=i*i;j<=n;j+=i)comp[j]=1;
  }
}
```

$O(N \log \log N)$.

Euclid mở rộng / Diophantine / CRT

Ngữ cảnh dùng: Có phương trình nguyên, inverse modulo composite hoặc hệ đồng dư. CRT tổng quát cần kiểm tra tính tương thích modulo gcd.

GCD mở rộng

```
ll exgcd(ll a,ll b,ll &x,ll &y){
  if(!b){x=1;y=0;return a;}
  ll x1,y1,g=exgcd(b,a%b,x1,y1);
  x=y1;y=x1-(a/b)*y1;
  return g;
}
```

$ax + by = gcd(a, b)$. Phương trình $ax + by = c$ có nghiệm nguyên khi và chỉ khi $gcd(a, b) \mid c$. Một nghiệm: nhân các hệ số Bezout với c/g. Nghiệm tổng quát: $x = x_0 + (b/g)t$, $y = y_0 - (a/g)t$.

CRT - hai đồng dư, modulo không nhất thiết nguyên tố cùng nhau

```
bool crt(ll r1,ll m1,ll r2,ll m2,ll &r,ll &m){
  ll x,y,g=exgcd(m1,m2,x,y);
  ll d=r2-r1;if(d%g)return 0;
  ll md=m2/g;
  ll t=(ll)((int128)(d/g)*x%md);
  t=(t%md+md)%md;
  m=m1/g*m2; // dùng int128 ếnu có nguy ơc tràn
      ố
  r=(ll)((r1+(int128)m1*t)%m);
  r=(r%m+m)%m;
  return 1;
}
```

CRT hợp lệ khi và chỉ khi $r_1 \equiv r_2 \pmod{gcd(m_1, m_2)}$. Mod mới $= lcm(m_1, m_2)$.

Euler Phi / Miller-Rabin

Ngữ cảnh dùng: Phi cho chu kỳ lũy thừa khi coprime; Miller-Rabin để kiểm tra nguyên tố 64-bit nhanh. Không dùng Fermat inverse nếu modulus không prime.

Euler Phi

```
ll phi(ll n){
  ll r=n;
  for(ll p=2;p*p<=n;p++)if(n%p==0){
    while(n%p==0)n/=p;
    r-=r/p;
  }
  if(n>1)r-=r/n;
  return r;
}
```

$\phi(n) = n \prod_{p|n}(1 - 1/p)$. Nếu $gcd(a, m) = 1$: $a^{\phi(m)} \equiv 1 \pmod{m}$. Hữu ích để giảm số mũ khi modulo không phải số nguyên tố.

Miller-Rabin tất định cho số nguyên 64-bit

```
ll mulmod(ll a,ll b,ll m){return (int128)a*b%m;}
ll pow64(ll a,ll b,ll m){
  ll r=1;
  while(b){if(b&1)r=mulmod(r,a,m);a=mulmod(a,a,m);b
      >>=1;}
  return r;
}
bool isPrime(ll n){
  if(n<2)return 0;
  for(ll p:{2,3,5,7,11,13,17,19,23,29,31,37})
    if(n%p==0)return n==p;
  ll d=n-1,s=0;while(!(d&1))d>>=1,s++;
  for(ll a
      :{2,325,9375,28178,450775,9780504,1795265022}){
    if(a%n==0)continue;
    ll x=pow64(a%n,d,n);
    if(x==1||x==n-1)continue;
    bool ok=0;
    for(int r=1;r<s;r++){
      x=mulmod(x,x,n);
      if(x==n-1){ok=1;break;}
    }
    if(!ok)return 0;
  }
  return 1;
}
```

Kiểm tra nguyên tố $O(\log n)$ phép nhân modulo cho mỗi cơ sở; int128 giúp tránh tràn khi nhân.

SPF / factorization nhanh

```
int spf[N];
void buildSPF(int n){
  for(int i=2;i<=n;i++)if(!spf[i]){
    spf[i]=i;
    if(1LL*i*i<=n)for(ll j=1LL*i*i;j<=n;j+=i)if
      (!spf[j])spf[j]=i;
  }
}
vector<pair<int,int>> factorSPF(int x){
  vector<pair<int,int>>f;
  while(x>1){int p=spf[x],c=0;while(x%p==0)x/=p,
      c++;f.pb({p,c});}
  return f;
}
```

Sau tiền xử lý, phân tích nhiều số rất nhanh. Nếu cần prime lớn 64-bit, dùng Miller-Rabin/Pollard-Rho.

Exponentiation II

```
ll expo2(ll a,ll b,ll c){
  ll e=binpow(b,c,MOD-1);
  return binpow(a,e,MOD);
}
```

Khi MOD là số nguyên tố và thỏa điều kiện Fermat, có thể giảm số mũ theo modulo $MOD-1$.

Gom floor(n/i) / tổng các ước

```
for(ll l=1,r;l<=n;l=r+1){
  ll q=n/l;
  r=n/q;
  // floor(n/x)=q ới qmi x trong [l,r]
}
```

Chỉ có $O(\sqrt{N})$ giá trị khác nhau của $\lfloor n/i \rfloor$. Với $\sum_{i=1}^{n}\sigma(i)$ dùng $\sum_{d=1}^{n} d\lfloor n/d \rfloor$ và gom các thương bằng nhau.

```
ll rangeSum(ll l,ll r){
  ll x=(l+r)%MOD,y=(r-l+1)%MOD;
  return x*y%MOD*binpow(2,MOD-2,MOD)%MOD;
}
ll sumDiv(ll n){
  ll ans=0;
  for(ll l=1,r;l<=n;l=r+1){
    ll q=n/l;r=n/q;
    ans=(ans+(q%MOD)*rangeSum(l,r))%MOD;
  }
  return ans;
}
```

Divisor Analysis

Nếu $n=\prod p_i^{a_i}$: số lượng ước $D=\prod(a_i+1)$; tổng các ước $S=\prod(1+p_i+\cdots+p_i^{a_i})$. Tích các ước = $n^{D/2}$ về mặt công thức, nhưng phải xử lý phép chia 2 ở số mũ trước khi giảm modulo $MOD-1$.

```
ll numDiv(vector<pair<ll,ll>>&f){
  ll ans=1;
  for(auto [p,a]:f)ans=ans*(a+1)%MOD;
  return ans;
}
ll sumDivisors(vector<pair<ll,ll>>&f){
  ll ans=1;
  for(auto [p,a]:f){
    ll up=(binpow(p,a+1,MOD)-1+MOD)%MOD;
    ll dn=binpow(p-1,MOD-2,MOD);
    ans=ans*up%MOD*dn%MOD;
  }
  return ans;
}
```

Tích các ước: nếu có (a_i+1) là chẵn, chia đôi một thừa số như vậy trước khi nhân số mũ đếm ước theo modulo $MOD-1$; nếu tất cả (a_i+1) đều lẻ thì mọi a_i đều chẵn, khi đó dùng $\sqrt{n^D}$.

Möbius / cặp nguyên tố cùng nhau

```
int mu[N],freq[N];vector<int>prime;bool cmp[N];
void mobius(int n){
  mu[1]=1;
  for(int i=2;i<=n;i++){
    if(!cmp[i])prime.pb(i),mu[i]=-1;
    for(int p:prime){
      if(1LL*i*p>n)break;
      cmp[i*p]=1;
      if(i%p==0){mu[i*p]=0;break;}
      mu[i*p]=-mu[i];
    }
  }
}
ll coprimePairs(int mx){
  ll ans=0;
  for(int d=1;d<=mx;d++){
    ll c=0;
    for(int x=d;x<=mx;x+=d)c+=freq[x];
    ans+=1LL*mu[d]*c*(c-1)/2;
  }
  return ans;
}
```

$[\gcd(a,b)=1]=\sum_{d|\gcd(a,b)}\mu(d)$. Do đó đáp án $=\sum_d \mu(d)C(cnt_d,2)$.

Tổ hợp

Factorial + tổ hợp C(n,k)

```
ll fact[N],invFact[N];
void initComb(int n){
  fact[0]=1;
  for(int i=1;i<=n;i++)fact[i]=fact[i-1]*i%MOD;
  invFact[n]=binpow(fact[n],MOD-2,MOD);
  for(int i=n;i>=1;i--)invFact[i-1]=invFact[i]*i
      %MOD;
}
ll C(int n,int k){
  if(k<0||k>n)return 0;
  return fact[n]*invFact[k]%MOD*invFact[n-k]%MOD
      ;
}
```

Tiền xử lý $O(N)$, mỗi truy vấn $O(1)$. Hoán vị đa tập: $n!/\prod cnt_i!$.

Chia vật / Catalan / hoán vị không điểm cố định

$x_1+\cdots+x_k=n, x_i\geq 0$: $C(n+k-1,k-1)$. Nếu $x_i\geq 1$: $C(n-1,k-1)$.
$Catalan(n)=C(2n,n)/(n+1)=C(2n,n)-C(2n,n+1)$.

```
D[0]=1;D[1]=0;
for(int i=2;i<=n;i++)
  D[i]=1LL*(i-1)*(D[i-1]+D[i-2])%MOD;
```

Bao hàm - loại trừ

```
ll ans=0;
for(int mask=1;mask<(1<<k);mask++){
  __int128 prod=1;int bits=0;
  for(int i=0;i<k;i++)if(mask>>i&1){
    prod*=p[i];bits++;
    if(prod>n)break;
  }
  if(prod>n)continue;
  if(bits&1)ans+=n/(ll)prod;
  else ans-=n/(ll)prod;
}
```

Bracket / Ballot

```
ll bracketCount(int n,string s){
  int bal=0;
  for(char c:s){
    bal+=(c=='('?1:-1);
    if(bal<0)return 0;
  }
  int rem=n-(int)s.size();
  if(rem<bal||((rem-bal)&1))return 0;
  int op=(rem-bal)/2;
  return (C(rem,op)-C(rem,op-1)+MOD)%MOD;
}
```

Dãy ngoặc hợp lệ: mọi tiền tố có độ cân bằng ≥ 0, cuối cùng độ cân bằng $=0$.

Burnside

$ans=\frac{1}{|G|}\sum_{g\in G} batBien(g)$. Đếm số cấu hình bất biến dưới từng phép đối xứng, rồi lấy trung bình.

Vòng cổ có n vị trí, m màu: $\frac{1}{n}\sum_{k=0}^{n-1} m^{\gcd(n,k)}$.

```
ll necklace(ll n,ll m){
  ll ans=0;
  for(ll k=0;k<n;k++)
    ans=(ans+binpow(m,std::gcd(n,k),MOD))%MOD;
  return ans*binpow(n,MOD-2,MOD)%MOD;
}
```

Công thức tổng / cấp số nhân

$\sum_{i=1}^{n} i=\frac{n(n+1)}{2}$; $\sum_{i=1}^{n} i^2=\frac{n(n+1)(2n+1)}{6}$; $\sum_{i=1}^{n} i^3=(\frac{n(n+1)}{2})^2$.

Cấp số nhân: $1+a+\cdots+a^n=(a^{n+1}-1)/(a-1)$. Với modulo nguyên tố, chỉ chia bằng cách nhân nghịch đảo khi mẫu khả nghịch. Cấp số cộng: $S=n(\text{đầu}+\text{cuối})/2$.

Mẹo số học / bit / modulo

```
ll normMod(ll x){x%=MOD;if(x<0)x+=MOD;return x;}
ll ceilDiv(ll a,ll b){return (a+b-1)/b;} // a,b
    >0
// x&-x: bit 1 ấthp ấnht; x&(x-1): xóa bit 1
    ấthp ấnht
```

Tiền tố modulo + Dirichlet: hai prefix cùng phần dư modulo m thì đoạn giữa có tổng chia hết cho m. Tổng XOR/AND/OR thường xử lý độc lập theo từng bit.

MA TRẬN - XÁC SUẤT - GAME - KACTL NÂNG CAO

Lũy thừa ma trận

Ngữ cảnh dùng: Recurrence tuyến tính bậc nhỏ hoặc đếm walk đúng số bước rất lớn. Complexity $O(d^3\log k)$ nên chỉ hợp khi dimension nhỏ.

```
using Matrix=vector<vector<ll>>;
Matrix mul(Matrix A,Matrix B){
  int n=A.size();Matrix C(n,vector<ll>(n));
  for(int i=0;i<n;i++)
    for(int k=0;k<n;k++)
      for(int j=0;j<n;j++)
        C[i][j]=(C[i][j]+A[i][k]*B[k][j])%MOD;
  return C;
}
Matrix mpow(Matrix A,ll k){
  int n=A.size();Matrix R(n,vector<ll>(n));
  for(int i=0;i<n;i++)R[i][i]=1;
  while(k){
    if(k&1)R=mul(R,A);
    A=mul(A,A);k>>=1;
  }
  return R;
}
```

Nhân $O(N^3)$; lũy thừa $O(N^3\log K)$. Ma trận chuyển Fibonacci $\begin{bmatrix}1&1\\1&0\end{bmatrix}$. Với ma trận kề A, $(A^k)_{ij}$ đếm số đường đi bộ đúng k cạnh.

Min-plus Matrix

```
Matrix minMul(Matrix A,Matrix B){
  int n=A.size();Matrix C(n,vector<ll>(n,LINF));
  for(int i=0;i<n;i++)for(int k=0;k<n;k++){
    if(A[i][k]==LINF)continue;
    for(int j=0;j<n;j++)if(B[k][j]!=LINF)
      C[i][j]=min(C[i][j],A[i][k]+B[k][j]);
  }
  return C;
}
Matrix minPow(Matrix A,ll k){
  int n=A.size();Matrix R(n,vector<ll>(n,LINF));
  for(int i=0;i<n;i++)R[i][i]=0;
  while(k){
    if(k&1)R=minMul(R,A);
    A=minMul(A,A);k>>=1;
  }
  return R;
}
```

Phép toán thường: gộp=+, nối=*. Min-plus: gộp=min, nối=+. Nếu $A[u][v]=$ trọng số cạnh, $(A^k)[u][v]=$ chi phí nhỏ nhất khi dùng đúng k cạnh.

Khử Gauss

Ngữ cảnh dùng: Giải hệ tuyến tính, xác định vô nghiệm/vô số nghiệm, hoặc bài kỳ vọng có cycle. Với modulo prime, thay division bằng modular inverse.

```
int gauss(vector<vector<ld>>a,vector<ld>&ans){
  int n=a.size(),m=a[0].size()-1;
  vector<int>trong đó(m,-1);
  for(int col=0,row=0;col<m&&row<n;col++){
    int sel=row;
    for(int i=row;i<n;i++)
      if(abs(a[i][col])>abs(a[sel][col]))sel=i;
    if(abs(a[sel][col])<EPS)continue;
    swap(a[sel],a[row]);trong đó[col]=row;
    for(int i=0;i<n;i++)if(i!=row){
      ld c=a[i][col]/a[row][col];
      for(int j=col;j<=m;j++)a[i][j]-=a[row][j]*
        c;
    }
    row++;
  }
  ans.assign(m,0);
  for(int i=0;i<m;i++)if(trong đó[i]!=-1)
    ans[i]=a[trong đó[i]][m]/a[trong đó[i]][i];
  for(int i=0;i<n;i++){
    ld s=0;for(int j=0;j<m;j++)s+=ans[j]*a[i][j
      ];
    if(abs(s-a[i][m])>EPS)return 0;
  }
  for(int i=0;i<m;i++)if(trong đó[i]==-1)return
      2;
  return 1;
}
```

Trả về 0=vô nghiệm, 1=nghiệm duy nhất, 2=vô số nghiệm. Chọn phần tử trụ có trị tuyệt đối lớn nhất. $O(N^3)$.

Xác suất / kỳ vọng

$E[X]=\sum_x xP(X=x)$. Tính tuyến tính: $E[X+Y]=E[X]+E[Y]$ kể cả khi các biến không độc lập. Kỳ vọng số nghịch thế: $E[inv]=\sum_{i<j}P(X_i>X_j)$. Với $X_i\sim U[1,a]$, $X_j\sim U[1,b]$, đếm số cặp thỏa thay vì liệt kê mọi kết quả ngẫu nhiên.

```
ll invWays(ll a,ll b){
  ll k=min(a-1,b);
  return k*(k+1)/2+max(0LL,a-b-1)*b;
}
ld probInv(ll a,ll b){
  return (ld)invWays(a,b)/(a*b);
}
```

Sprague-Grundy / game theory

```
int mex(vector<int>a){
  sort(all(a));a.erase(unique(all(a)),a.end());
  int x=0;
  for(int v:a){if(v==x)x++;else if(v>x)break;}
  return x;
}
```

$SG(s)=\text{mex}\{SG(t):t\in next(s)\}$. Các trò chơi độc lập gộp bằng XOR: $SG=SG_1\oplus SG_2\oplus\cdots$. Khác 0 -> thắng; bằng 0 -> thua. Nim chuẩn chính là XOR kích thước các đống.

Cơ sở tuyến tính XOR

```
ll basis[63];
void addBasis(ll x){
  for(int i=62;i>=0;i--)if(x>>i&1){
    if(!basis[i]){basis[i]=x;return;}
    x^=basis[i];
  }
}
ll maxXor(ll x=0){
  for(int i=62;i>=0;i--)x=max(x,x^basis[i]);
  return x;
}
```

Duy trì các vectơ độc lập trên $GF(2)$. Hạng = số phần tử cơ sở khác 0. Nếu hạng là r thì số giá trị XOR tập con khác nhau là 2^r (trước khi xét modulo hoặc các ràng buộc đếm).

Truy hồi tuyến tính nâng cao

Matrix exponentiation: recurrence bậc K -> $O(K^3 \log n)$. KACTL còn có Berlekamp-Massey để suy ra recurrence ngắn nhất từ khoảng $2K$ số hạng và LinearRecurrence để tính số hạng thứ n nhanh; dùng khi K lớn và dãy cho bằng prefix.

FFT / NTT - tích chập

Dùng khi cần nhân đa thức / convolution. FFT dùng số phức; NTT dùng modulo dạng $c \cdot 2^k+1$ và primitive root. Độ phức tạp $O(N \log N)$. Nếu đề chỉ cần convolution modulo quen thuộc (998244353), ưu tiên NTT.

Tối ưu số / tích phân

Golden Section: tìm cực trị hàm đơn đỉnh trên đoạn thực, $O(\log(1/\varepsilon))$ lần gọi hàm. Simpson thích nghi: tích phân số khi không có nguyên hàm đóng; chia đoạn cho tới khi sai số ước lượng đủ nhỏ. Simplex: quy hoạch tuyến tính $\max c^T x$ với ràng buộc tuyến tính - chỉ dùng khi đề thật sự yêu cầu.

Pollard-Rho / căn modulo

Miller-Rabin chỉ kiểm tra prime. Khi cần factor số 64-bit lớn, ghép Miller-Rabin + Pollard-Rho để tìm ước không tầm thường rồi đệ quy. Modular sqrt $x^2 \equiv a \pmod p$: với prime lẻ dùng Tonelli-Shanks; nếu $p \equiv 3 \pmod 4$ thì $x = a^{(p+1)/4} \bmod p$.

Nội suy đa thức / Lagrange

Biết $n+1$ điểm của đa thức bậc $\le n$ thì xác định duy nhất. Lagrange: $P(x) = \sum_i y_i \prod_{j \ne i} (x - x_j)/(x_i - x_j)$. Với modulo prime dùng nghịch đảo modulo; hữu ích khi cần giá trị ở x rất lớn nhưng chỉ biết vài giá trị đầu.

Josephus Queries

```
ll josephus(ll n,ll k){
  if(n==1)return 1;
  ll first=n/2;
  if(k<=first)return 2*k;
  ll x=josephus((n+1)/2,k-first);
  if(n%2==0)return 2*x-1;
  if(x==1)return n;
  return 2*x-3;
}
```

Trả về người bị loại thứ k khi bước nhảy 2. $O(\log N)$. Công thức người sống sót tổng quát khác: $J(n,k) = (J(n-1,k)+k) \bmod n$.

Chọn cấu trúc nhanh

Cần	Cấu trúc
Liên thông / gộp tập	DSU
MST	Kruskal + DSU
Cập nhật điểm + tổng	Fenwick
Min/max/gcd trên đoạn	Segment Tree
Cập nhật đoạn	Lazy SegTree
Phần tử thứ k/trung vị trên đoạn	Persistent SegTree
Phần tử thứ k động / đếm $< x$	PBDS
Tổng đoạn tĩnh	Tổng tiền tố
RMQ tĩnh	Sparse Table
Đường đi ngắn nhất cạnh 0/1	0-1 BFS
Đường đi ngắn nhất trọng số không âm	Dijkstra
Đường đi ngắn nhất đúng k cạnh	Min-plus matrix

STL cứu bài nhanh

```
auto it=lower_bound(all(a),x); // đầu tiên >=x
auto jt=upper_bound(all(a),x); // đầu tiên >x
int cnt=upper_bound(all(a),r)-lower_bound(all(a),l);
nth_element(a.begin(),a.begin()+k,a.end()); // a[k] là kth, trung bình O(n)
partial_sum(all(a),pref.begin());
iota(all(p),0);
__builtin_popcountll(x);
__builtin_ctzll(x); // x>0
```

Multiset: extttms.erase(ms.find(x)) xóa đúng 1 bản sao. Min-heap: extttpriority_queue<T,vector<T>,greater<T>>.

Checklist WA / TLE / số thực

In số thực / bỏ số 0 thừa

```
if(fabsl(x)<1e-9L)x=0;
cout<<setprecision(10)<<x; // khong fixed -> tu bo 0 thua
// string: while(s.back()=='0')s.pop_back(); if(s.back()=='.')s.pop_back();
```

fixed+setprecision(k) = k chữ số sau dấu chấm; không fixed = k chữ số có nghĩa. Với chuỗi, chỉ xóa 0 nếu có dấu chấm.

- Phân biệt int và long long; khi cần dùng 1LL<<n thay vì 1<<n.
- Đặt INF/LINF đủ lớn; tránh tràn số khi nhân a*b.
- Đặt lại mảng/vector toàn cục giữa các bộ test.
- Kiểm tra đánh số từ 0 hay 1 và biên đóng/mở.
- Phân biệt cách thêm cạnh có hướng và vô hướng.
- Dijkstra yêu cầu trọng số không âm.
- Phần tử đơn vị của Segment Tree phải đúng với phép gộp.
- PBDS find_by_order(k) đánh số từ 0.
- Persistent SegTree cần nén tọa độ và đủ bộ nhớ cho các nút ($O(N \log N)$).
- MST phải chọn đúng $n-1$ cạnh.
- Modulo âm: dùng (x%MOD+MOD)%MOD.
- Đệ quy quá sâu có thể tràn stack; dùng DFS lặp khi cần.
- Ước lượng độ phức tạp từ giới hạn trước khi code.

Phản xạ trước giờ thi

$n \le 20$: tập con/bitmask. $n \approx 40$: chia đôi tập. Tổng tĩnh: tổng tiền tố. Tổng tiền tố/đoạn có cập nhật: Fenwick. Min/max hoặc phép gộp riêng trên đoạn: SegTree. Chuyển trạng thái lặp với số mũ lớn: lũy thừa nhị phân/ma trận. "x nhỏ nhất sao cho..." + điều kiện đơn điệu: tìm kiếm nhị phân trên đáp án. Số bước ít nhất với mỗi bước chi phí 1: BFS. Trọng số không âm: Dijkstra. Gộp/liên thông lặp lại: DSU.

HÌNH HỌC 2D

Point / vector / cross / dot

Ngữ cảnh dùng: Hầu hết geometry 2D. Cross quyết định hướng/diện tích, dot quyết định góc/hình chiếu. Với tọa độ nguyên lớn nên dùng __int128 cho cross.

```
using P=pair<ll,ll>;
P subP(P a,P b){return {a.ft-b.ft,a.sd-b.sd};}
int128 cross(P a,P b){return (int128)a.ft*b.sd-(int128)a.sd*b.ft;}
int128 dot(P a,P b){return (int128)a.ft*b.ft+(int128)a.sd*b.sd;}
int128 orient(P a,P b,P c){return cross(subP(b,a),subP(c,a));}
int sgn(int128 x){return (x>0)-(x<0);}
int128 abs128(int128 x){return x<0?-x:x;}
int128 dist2(P a,P b){
  int128 x=(int128)a.ft-b.ft,y=(int128)a.sd-b.sd;
  return x*x+y*y;
}
```

Tích có hướng/hướng quay: > 0 ngược chiều kim đồng hồ/trái; < 0 thuận chiều kim đồng hồ/phải; $= 0$ thẳng hàng. **Tích vô hướng:** > 0 góc nhọn, $= 0$ vuông góc, < 0 góc tù. Với tọa độ nguyên nên dùng phép tính chính xác; tọa độ lớn có thể cần int128.

Điểm trên đoạn + giao hai đoạn

```
bool onSeg(P a,P b,P q){
  if(orient(a,b,q)!=0)return 0;
  return min(a.ft,b.ft)<=q.ft&&q.ft<=max(a.ft,b.ft)&&
      min(a.sd,b.sd)<=q.sd&&q.sd<=max(a.sd,b.sd);
}
bool segInter(P a,P b,P c,P d){
  auto x=orient(a,b,c),y=orient(a,b,d);
  auto z=orient(c,d,a),t=orient(c,d,b);
  if(sgn(x)*sgn(y)<0&&sgn(z)*sgn(t)<0)return 1;
  if(x==0&&onSeg(a,b,c))return 1;
  if(y==0&&onSeg(a,b,d))return 1;
  if(z==0&&onSeg(c,d,a))return 1;
  if(t==0&&onSeg(c,d,b))return 1;
  return 0;
}
```

Đường thẳng / hình chiếu / khoảng cách

Đường thẳng qua A có vectơ chỉ phương $D = B - A$: $X = A + tD$. Với hai đường thẳng $A + tr$ và $C + us$, nếu $cross(r,s) \ne 0$ thì $t = \frac{cross(C-A,s)}{cross(r,s)}$. Song song khi và chỉ khi $cross(r,s) = 0$.

```
using PD=pair<ld,ld>;
PD subD(PD a,PD b){return {a.ft-b.ft,a.sd-b.sd};}
PD addD(PD a,PD b){return {a.ft+b.ft,a.sd+b.sd};}
PD mulD(PD a,ld k){return {a.ft*k,a.sd*k};}
ld crossD(PD a,PD b){return a.ft*b.sd-a.sd*b.ft;}
ld dotD(PD a,PD b){return a.ft*b.ft+a.sd*b.sd;}
ld norm(PD a){return sqrt(dotD(a,a));}
ld distD(PD a,PD b){return norm(subD(a,b));}
ld distLine(PD a,PD b,PD p){
  return abs(crossD(subD(b,a),subD(p,a)))/distD(a,b);
}
PD projection(PD a,PD b,PD p){
  PD d=subD(b,a);
  ld t=dotD(subD(p,a),d)/dotD(d,d);
  return addD(a,mulD(d,t));
}
```

Khoảng cách điểm đến đoạn: tính $t = dot(P-A, B-A)/|B-A|^2$, chặn t trong $[0,1]$, rồi lấy khoảng cách đến $A + t(B-A)$.

EPS / giao đường / phản xạ / khoảng cách đoạn

```
int cmp(ld a,ld b=0){
  if(a<b-EPS)return -1;
  if(a>b+EPS)return 1;
  return 0;
}
bool lineInter(PD a,PD b,PD c,PD d,PD &out){
  PD r=subD(b,a),s=subD(d,c);
  ld den=crossD(r,s);if(abs(den)<EPS)return 0;
  ld t=crossD(subD(c,a),s)/den;
  out=addD(a,mulD(r,t));return 1;
}
PD reflectLine(PD a,PD b,PD p){
  PD h=projection(a,b,p);
  return {2*h.ft-p.ft,2*h.sd-p.sd};
}
```

Dùng extttcmp thay vì so trực tiếp số thực. Hai đường thẳng song song hoặc trùng nhau khi mẫu gần 0; hàm trên chỉ trả giao điểm duy nhất.

Khoảng cách hai đoạn

```
ld distSeg(PD a,PD b,PD p){
  PD d=subD(b,a);ld t=dotD(subD(p,a),d)/dotD(d,d);
  t=max((ld)0,min((ld)1,t));
  return distD(p,addD(a,mulD(d,t)));
}
ld distSegSeg(PD a,PD b,PD c,PD d){
  // ếnu dùng ảbn giao đạon integer/epsilon và giao nhau => 0
  return min({distSeg(a,b,c),distSeg(a,b,d),
    distSeg(c,d,a),distSeg(c,d,b)});
}
```

Giao đường tròn - đường tròn (tọa độ)

```
vector<PD> circleInter(PD A,ld r,PD B,ld R){
  ld dx=B.ft-A.ft,dy=B.sd-A.sd,d=hypot(dx,dy);
  vector<PD>res;
  if(d>r+R+EPS||d<abs(r-R)-EPS||d<EPS)return res;
  ld x=(r*r-R*R+d*d)/(2*d),h2=r*r-x*x;
  if(h2<0&&h2>-EPS)h2=0;if(h2<0)return res;
  ld h=sqrt(h2),ux=dx/d,uy=dy/d;
  PD p={A.ft+x*ux,A.sd+x*uy};
  res.pb({p.ft-h*uy,p.sd+h*ux});
  if(h>EPS)res.pb({p.ft+h*uy,p.sd-h*ux});
  return res;
}
```

Trường hợp hai đường tròn trùng nhau có vô số giao điểm nên phải xử lý riêng nếu đề có.

Đa giác / Shoelace / Pick / point-in-polygon

```
int128 area2(vector<P>&p){
  int128 s=0;int n=sz(p);
  for(int i=0;i<n;i++)s+=cross(p[i],p[(i+1)%n]);
  return abs128(s); // = 2*ệdin tích
}
ll boundaryLattice(vector<P>&p){
  ll B=0;int n=sz(p);
  for(int i=0;i<n;i++){
    ll dx=llabs(p[i].ft-p[(i+1)%n].ft);
    ll dy=llabs(p[i].sd-p[(i+1)%n].sd);
    B+=std::gcd(dx,dy);
  }
  return B;
}
```

Công thức Shoelace: $2A = |\sum x_i y_{i+1} - y_i x_{i+1}|$. Định lý Pick cho đa giác lưới: $A = I + B/2 - 1$, suy ra $I = (2A - B + 2)/2$. Số điểm biên là $B = \sum gcd(|dx|, |dy|)$ (không cộng +1 cho từng cạnh).

Điểm trong đa giác - số vòng quấn

```
bool inPoly(vector<P>&p,P q){
  int w=0,n=sz(p);
  for(int i=0;i<n;i++){
    P a=p[i],b=p[(i+1)%n];
    if(onSeg(a,b,q))return 1;
    if(a.sd<=q.sd){
      if(b.sd>q.sd&&orient(a,b,q)>0)w++;
    }else if(b.sd<=q.sd&&orient(a,b,q)<0)w--;
  }
  return w!=0;
}
```

Convex Hull / rotating calipers

Ngữ cảnh dùng: Chỉ biên lồi của tập điểm quan trọng. Hull $O(N \log N)$; rotating calipers cho diameter/width và các bài tối ưu trên polygon lồi.

Chuỗi đơn điệu

```
vector<P> hull(vector<P>p){
  sort(all(p));p.erase(unique(all(p)),p.end());
  if(sz(p)<=1)return p;
  vector<P>h;
  for(P x:p){
    while(sz(h)>=2&&orient(h[sz(h)-2],h.back(),x
      )<=0)h.pop_back();
    h.pb(x);
  }
  int low=sz(h);
  for(int i=sz(p)-2;i>=0;i--){
    P x=p[i];
    while(sz(h)>low&&orient(h[sz(h)-2],h.back(),
      x)<=0)h.pop_back();
    h.pb(x);
  }
  h.pop_back();return h;
}
```

$O(N \log N)$. Với điều kiện `<=0`, các điểm thẳng hàng nằm giữa trên biên bị loại; dùng `<0` nếu đề yêu cầu giữ mọi điểm thẳng hàng trên biên.

Đường kính đa giác lồi

```
int128 diameter2(vector<P>&h){
  int n=sz(h);if(n<=1)return 0;
  if(n==2)return dist2(h[0],h[1]);
  int j=1;int128 ans=0;
  for(int i=0;i<n;i++){
    int ni=(i+1)%n;
    while(abs128(orient(h[i],h[ni],h[(j+1)%n]))>
          abs128(orient(h[i],h[ni],h[j])))j=(j
      +1)%n;
    ans=max(ans,dist2(h[i],h[j]));
    ans=max(ans,dist2(h[ni],h[j]));
  }
  return ans;
}
```

Thước quay sau khi dựng bao lồi: $O(H)$. Trả về bình phương đường kính để tránh căn bậc hai.

Point in Convex $O(\log N)$

Ngữ cảnh dùng: Có nhiều query điểm-nằm-trong trên một polygon lồi cố định. Polygon phải có thứ tự CCW/CW nhất quán và xử lý biên riêng.

```
bool inConvex(vector<P>&p,P q){ // p theo CCW,
     không có điểm ẳthng hàng trên bao ồli
  int n=sz(p);if(n<3)return 0;
  if(orient(p[0],p[1],q)<0)return 0;
  if(orient(p[0],p[n-1],q)>0)return 0;
  int l=1,r=n-1;
  while(r-l>1){
    int m=(l+r)/2;
    if(orient(p[0],p[m],q)>=0)l=m;
    else r=m;
  }
  return orient(p[l],p[(l+1)%n],q)>=0;
}
```

Khi có nhiều truy vấn điểm-trong-đa-giác-lồi: cố định quạt tại p_0, tìm kiếm nhị phân miền, rồi kiểm tra một lần hướng quay. Có tính cả điểm biên.

Lượng giác / tam giác / phép quay

$rad = deg \cdot \pi/180$, $deg = rad \cdot 180/\pi$.

$\sin^2 x + \cos^2 x = 1$, $\tan x = \sin x / \cos x$.

$c^2 = a^2 + b^2 - 2ab\cos C$, $S = \frac{1}{2}ab\sin C$.

$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R$. Heron: $s = (a+b+c)/2$, $S = \sqrt{s(s-a)(s-b)(s-c)}$. Bán kính ngoại tiếp $R = abc/(4S)$; bán kính nội tiếp $r = S/s$. Trong C++, hàm lượng giác dùng radian. Ưu tiên `atan2(y,x)` để lấy hướng; chặn đầu vào trong $[-1,1]$ trước khi dùng `acos/asin`.

Code sin / cos / tan / góc

```
ld toRad(ld deg){return deg*PI/180;}
ld toDeg(ld rad){return rad*180/PI;}
ld clamp1(ld x){return max((ld)-1,min((ld)1,x))
     ;}

// C++ dùng radian
ld s=sin(t), c=cos(t), tg=tan(t);
ld as=asin(clamp1(x));
ld ac=acos(clamp1(x));
ld at=atan(x);
ld ang=atan2(y,x); // [-PI,PI]
```

Không gọi `sin(90)` nếu 90 là độ: phải đổi bằng `toRad(90)`. `atan2(y,x)` tự xác định góc phần tư. Với `asin/acos`, clamp về $[-1,1]$ để tránh NaN do sai số.

Góc / phép quay / định luật cos

```
ld angle(PD a){return atan2(a.sd,a.ft);}
ld angleBetween(PD a,PD b){
 return atan2(crossD(a,b),dotD(a,b));
}
PD rotate(PD p,ld t){
 ld c=cos(t),s=sin(t);
 return {p.ft*c-p.sd*s,p.ft*s+p.sd*c};
}
ld angleBySides(ld a,ld b,ld c){
 return acos(clamp1((a*a+b*b-c*c)/(2*a*b)));
}
```

Góc có hướng từ vectơ a đến b: $atan2(cross(a,b), dot(a,b))$. Với hai cạnh a, b và cạnh đối diện c, `angleBySides` trả góc xen giữa a, b.

Đường tròn trên tọa độ

Đường tròn tâm $O(c_x, c_y)$, bán kính r: $(x-c_x)^2 + (y-c_y)^2 = r^2$. So sánh bình phương khoảng cách để tránh căn bậc hai khi có thể.

```
PD pointCircle(PD o,ld r,ld t){
  return {o.ft+r*cos(t),o.sd+r*sin(t)};
}
ld pointAngle(PD o,PD p){
  ld t=atan2(p.sd-o.sd,p.ft-o.ft);
  if(t<0)t+=2*PI;
  return t;
}
```

$$\text{độ dài cung} = r\theta$$

$$\text{diện tích hình quạt} = \tfrac{1}{2}r^2\theta$$

$$\text{dây cung} = 2r\sin(\theta/2)$$

θ tính bằng radian. Với điểm ngoài P, $d = |PO| > r$: độ dài tiếp tuyến $= \sqrt{d^2 - r^2}$; các hướng tiếp tuyến là $\arg(P-O) \pm \arccos(r/d)$.

Quan hệ hai đường tròn

Đặt $d = |O_1O_2|$. $d > r_1 + r_2$: rời nhau; $d = r_1 + r_2$: tiếp xúc ngoài; $|r_1 - r_2| < d < r_1 + r_2$: 2 giao điểm; $d = |r_1 - r_2|$: tiếp xúc trong; $d < |r_1 - r_2|$: một đường tròn nằm trong đường tròn kia; $d = 0, r_1 = r_2$: trùng nhau. Dùng EPS khi so sánh số thực.

Hai giao điểm đường tròn - đường tròn

Với $d > 0$: $a = (r_1^2 - r_2^2 + d^2)/(2d)$, $h = \sqrt{r_1^2 - a^2}$. Điểm cơ sở $P_0 = O_1 + a(O_2 - O_1)/d$. Nếu vectơ đơn vị là (u_x, u_y), vectơ vuông góc là $(-u_y, u_x)$, hai giao điểm $P_0 \pm h(-u_y, u_x)$.

Line-Circle / phương trình bậc hai

Đường thẳng tham số $P(t) = A + tD$, đường tròn tâm O, bán kính r. Đặt $F = A - O$: giải $at^2 + bt + c = 0$, trong đó $a = D \cdot D$, $b = 2(F \cdot D)$, $c = F \cdot F - r^2$. Biệt thức $\Delta = b^2 - 4ac$: < 0 không giao, $= 0$ tiếp xúc, > 0 có hai giao điểm. $t = (-b \pm \sqrt{\Delta})/(2a)$.

Hai điểm + bán kính -> tâm đường tròn

```
bool circle2PtsRad(PD a,PD b,ld r,PD &c){
 ld dx=a.ft-b.ft,dy=a.sd-b.sd,d2=dx*dx+dy*dy;
 if(d2==0)return false;
 ld det=r*r/d2-0.25;
 if(det<-EPS)return false;
 ld h=sqrt(max((ld)0,det));
 c.ft=(a.ft+b.ft)/2+(a.sd-b.sd)*h;
 c.sd=(a.sd+b.sd)/2+(b.ft-a.ft)*h;
 return true; // dao a,b de lay tam con lai
}
```

Tồn tại iff khoảng cách hai điểm $\le 2r$. Thường gặp bài circle packing / dựng đường tròn qua hai điểm.

Great-circle distance - lat/lon

```
ld gcDist(ld lat1,ld lon1,ld lat2,ld lon2,ld R){
 lat1=toRad(lat1);lon1=toRad(lon1);
 lat2=toRad(lat2);lon2=toRad(lon2);
 ld cs=sin(lat1)*sin(lat2)+cos(lat1)*cos(lat2)*
      cos(lon1-lon2);
 cs=max((ld)-1,min((ld)1,cs));
 return R*acos(cs);
}
```

Khoảng cách ngắn nhất trên mặt cầu = $R\theta$. Hữu ích cho bài Earth/Airline; nhớ đổi độ sang radian và clamp trước `acos`.

Trọng tâm / tâm tam giác

```
PD centroidPoly(vector<PD>&p){
  ld A=0,cx=0,cy=0;int n=sz(p);
  for(int i=0;i<n;i++){
    auto a=p[i],b=p[(i+1)%n];
    ld z=crossD(a,b);A+=z;
    cx+=(a.ft+b.ft)*z;cy+=(a.sd+b.sd)*z;
  }
  return {cx/(3*A),cy/(3*A)}; // A có ấdu = ổtng
       cross
}
```

Trọng tâm tam giác: $G = (A+B+C)/3$. Tâm nội tiếp: $(aA + bB + cC)/(a+b+c)$ trong đó $a = |BC|, b = |CA|, c = |AB|$. Tâm ngoại tiếp = giao hai đường trung trực; bán kính ngoại tiếp $R = abc/(4S)$.

Manhattan transform

$L_1(P,Q) = |x_1 - x_2| + |y_1 - y_2|$. Biến đổi $u = x + y$, $v = x - y$. Cặp điểm có khoảng cách Manhattan lớn nhất trong 2D:

$$\max\left(\max u - \min u,\ \max v - \min v\right).$$

Khoảng cách Chebyshev $L_\infty = \max(|dx|, |dy|)$. Quay 45° thường biến ràng buộc Manhattan thành ràng buộc song song trục trong (u, v).

Hình học KACTL bổ sung

Closest Pair: chia để trị/sweep line $O(N \log N)$. Minimum Enclosing Circle: thuật toán randomized kỳ vọng $O(N)$. Circle tangents: tiếp tuyến chung ngoài/nội. Circle-polygon intersection: diện tích giao. Line-hull intersection và point-in-convex: khai thác tính lồi để xuống $O(\log N)$. 3D: cross/dot, thể tích khối đa diện, convex hull 3D chỉ nên dùng khi đề bắt buộc.

Tree Toolkit - Euler Tour / cây con

Ngữ cảnh dùng: Cần biến cây con thành đoạn liên tiếp để dùng Fenwick/Segment Tree. Euler flatten là nền cho subtree query/update.

```
int tin[N],tout[N],timer;
void dfsEuler(int u,int p){
  tin[u]=++timer;
  for(int v:adj[u]) if(v!=p) dfsEuler(v,u);
  tout[u]=timer;
}
// subtree(u) <=> [tin[u], tout[u]]
```

Dùng để biến truy vấn/cập nhật cây con thành đoạn liên tiếp trên Fenwick/Segment Tree. Nếu cần giá trị trên cạnh, thường gán cạnh $(parent, u)$ vào vị trí `tin[u]`.

Binary Lifting + LCA

Ngữ cảnh dùng: Cây tĩnh, nhiều query LCA/jump k tổ tiên/khoảng cách. Precompute $O(N \log N)$, query $O(\log N)$.

```
int up[LOG][N],dep[N];
void dfsLCA(int u,int p){
  up[0][u]=p;
  for(int j=1;j<LOG;j++)
    up[j][u]=up[j-1][up[j-1][u]];
  for(int v:adj[u]) if(v!=p){
    dep[v]=dep[u]+1;
    dfsLCA(v,u);
  }
}
int jump(int u,int k){
  for(int j=0;j<LOG;j++)
    if(k>>j&1) u=up[j][u];
  return u;
}
int lca(int a,int b){
  if(dep[a]<dep[b])swap(a,b);
  a=jump(a,dep[a]-dep[b]);
  if(a==b)return a;
  for(int j=LOG-1;j>=0;j--)
    if(up[j][a]!=up[j][b])
      a=up[j][a],b=up[j][b];
  return up[0][a];
}
int distTree(int a,int b){
  int c=lca(a,b);
  return dep[a]+dep[b]-2*dep[c];
}
```

$O(N \log N)$ tiền xử lý, $O(\log N)$ mỗi LCA/jump. K-th node trên đường $u \to v$: tách theo LCA và độ dài hai nửa.

Binary Lifting - max cạnh trên đường

```
int mx[LOG][N];
void dfsLift(int u,int p,int w){
  up[0][u]=p; mx[0][u]=w;
  for(int j=1;j<LOG;j++){
    mx[j][u]=max(mx[j-1][u],mx[j-1][up[j-1][u]])
      ;
    up[j][u]=up[j-1][up[j-1][u]];
  }
  for(auto [v,c]:g[u]) if(v!=p){
    dep[v]=dep[u]+1; dfsLift(v,u,c);
  }
}
int climbMax(int &u,int k){
  int ans=0;
  for(int j=LOG-1;j>=0;j--) if(k>>j&1){
    ans=max(ans,mx[j][u]); u=up[j][u];
  }
  return ans;
}
int pathMax(int a,int b){
  int ans=0;
  if(dep[a]<dep[b])swap(a,b);
  ans=max(ans,climbMax(a,dep[a]-dep[b]));
  if(a==b)return ans;
  for(int j=LOG-1;j>=0;j--) if(up[j][a]!=up[j][b
     ]){
    ans=max({ans,mx[j][a],mx[j][b]});
    a=up[j][a]; b=up[j][b];
  }
  return max({ans,mx[0][a],mx[0][b]});
}
```

Cực hữu ích cho minimax path trên MST, second-best MST, truy vấn max/min cạnh trên path. Có thể đổi `max` thành min/sum/gcd nếu phép gộp phù hợp.

Đường kính cây / tâm cây

```
pair<int,int> farthest(int s){
  vector<int>d(n+1,-1); queue<int>q;
  d[s]=0;q.push(s);int best=s;
  while(!q.empty()){
    int u=q.front();q.pop();
    if(d[u]>d[best])best=u;
    for(int v:adj[u])if(d[v]<0)
      d[v]=d[u]+1,q.push(v);
  }
  return {best,d[best]};
}
// a=farthest(1).first;
// b=farthest(a); b.second = diameter
```

Cây không trọng số: 2 BFS/DFS. Center nằm giữa đường kính. Với trọng số dương dùng DFS cộng trọng số.

Rerooting DP - tổng khoảng cách

Ngữ cảnh dùng: Cần đáp án cho mọi lựa chọn root nhưng công thức giữa root cha-con có thể cập nhật $O(1)$. Thường làm hai DFS.

```
ll sub[N],dp[N],ans[N];
void dfs1(int u,int p){
  sub[u]=1; dp[u]=0;
  for(int v:adj[u])if(v!=p){
    dfs1(v,u);
    sub[u]+=sub[v];
    dp[u]+=dp[v]+sub[v];
  }
}
void dfs2(int u,int p){
  for(int v:adj[u])if(v!=p){
    ans[v]=ans[u]-sub[v]+(n-sub[v]);
    dfs2(v,u);
  }
}
// dfs1(root,0); ans[root]=dp[root]; dfs2(root
     ,0);
```

Pattern reroot: tính đáp án ở một root, sau đó chuyển root qua cạnh bằng cách trừ phần con và cộng phần còn lại. Dùng cho tổng khoảng cách, số cặp, score theo root.

HLD - khung đầy đủ

```
int par[N],dep[N],heavy[N],head[N],pos[N],subsz[
    N],cur;
int dfsH(int u,int p){
  par[u]=p;subsz[u]=1;int best=0;
  for(int v:adj[u])if(v!=p){
    dep[v]=dep[u]+1;int s=dfsH(v,u);
    subsz[u]+=s;
    if(s>best)best=s,heavy[u]=v;
  }
  return subsz[u];
}
void decomp(int u,int h){
  head[u]=h;pos[u]=++cur;
  if(heavy[u])decomp(heavy[u],h);
  for(int v:adj[u])if(v!=par[u]&&v!=heavy[u])
    decomp(v,v);
}
ll queryPath(int a,int b){
  ll res=0;
  while(head[a]!=head[b]){
    if(dep[head[a]]<dep[head[b]])swap(a,b);
    res=max(res,segQuery(pos[head[a]],pos[a]));
    a=par[head[a]];
  }
  if(dep[a]>dep[b])swap(a,b);
  res=max(res,segQuery(pos[a],pos[b]));
  return res;
}
```

HLD + Segment Tree: point/range update, query path/subtree. Path có $O(\log N)$ đoạn, mỗi đoạn query $O(\log N)$ -> $O(\log^2 N)$.

Centroid Decomposition - khung

Ngữ cảnh dùng: Truy vấn khoảng cách động trên cây, như node màu đỏ gần nhất hoặc đếm node theo distance. Mỗi node xuất hiện ở $O(\log N)$ mức centroid.

```
int subC[N],cpar[N]; bool dead[N];
int calcSz(int u,int p){
  subC[u]=1;
  for(int v:adj[u])if(v!=p&&!dead[v])
    subC[u]+=calcSz(v,u);
  return subC[u];
}
int getCent(int u,int p,int tot){
  for(int v:adj[u])if(v!=p&&!dead[v])
    if(subC[v]>tot/2)return getCent(v,u,tot);
  return u;
}
void buildCent(int u,int p){
  int c=getCent(u,0,calcSz(u,0));
  cpar[c]=p;dead[c]=1;
  for(int v:adj[c])if(!dead[v])buildCent(v,c);
}
```

Dùng cho truy vấn khoảng cách động trên cây, "node đỏ gần nhất", đếm cặp theo distance. Mỗi node thuộc $O(\log N)$ centroid ancestors.

Dinic Max Flow - template

```
struct Edge{int v,rev;ll cap;};
vector<Edge> fg[N];
int level[N],itF[N];
void addEdge(int u,int v,ll c){
  Edge a{v,(int)fg[v].size(),c};
  Edge b{u,(int)fg[u].size(),0};
  fg[u].push_back(a);fg[v].push_back(b);
}
bool bfsF(int s,int t){
  fill(level,level+n+1,-1);
  queue<int>q;q.push(s);level[s]=0;
  while(!q.empty()){
    int u=q.front();q.pop();
    for(auto &e:fg[u])if(e.cap&&level[e.v]<0)
      level[e.v]=level[u]+1,q.push(e.v);
  }
  return level[t]>=0;
}
ll dfsF(int u,int t,ll f){
  if(u==t)return f;
  for(int &i=itF[u];i<(int)fg[u].size();i++){
    Edge &e=fg[u][i];
    if(e.cap&&level[e.v]==level[u]+1){
      ll got=dfsF(e.v,t,min(f,e.cap));
      if(got){e.cap-=got;fg[e.v][e.rev].cap+=got
      ;return got;}
    }
  }
  return 0;
}
ll dinic(int s,int t){
  ll flow=0,f;
  while(bfsF(s,t)){
    fill(itF,itF+n+1,0);
    while((f=dfsF(s,t,LINF)))flow+=f;
  }
  return flow;
}
```

Sau max-flow, BFS/DFS từ source trên cạnh residual còn cap > 0: tập reachable là phía source của min-cut.

Mô hình Flow - capacity đỉnh / nhiều nguồn

```
// vertex capacity c[u]:
// u_in -> u_out capacity c[u]
// edge u->v becomes u_out -> v_in

// multi-source/sink:
// superS -> each source (INF)
// each sink -> superT (INF)
```

Edge-disjoint paths: capacity cạnh = 1. Vertex-disjoint paths: split vertex với capacity 1. Min number cạnh cần cắt = max-flow khi capacity = 1.

Lower-bound Flow / Circulation

```
// edge u->v requires L <= f <= R
// add residual capacity R-L
// demand[u]-=L; demand[v]+=L
// super source SS -> v if demand[v]>0
// v -> TT if demand[v]<0 with -demand[v]
// feasible iff max-flow(SS,TT) == sum positive
     demand
```

Nếu cần flow từ s đến t có lower bound: thêm cạnh $t \to s$ capacity INF trước khi kiểm tra circulation; sau đó bỏ super nodes để tối ưu tiếp.

Hopcroft-Karp - bipartite matching

```
vector<int> bg[N];
int matchL[N],matchR[N],distHK[N];
bool bfsHK(){
  queue<int>q;
  for(int u=1;u<=L;u++)
    if(!matchL[u])distHK[u]=0,q.push(u);
    else distHK[u]=-1;
  bool found=0;
  while(!q.empty()){
    int u=q.front();q.pop();
    for(int v:bg[u]){
      int u2=matchR[v];
      if(!u2)found=1;
      else if(distHK[u2]<0)
        distHK[u2]=distHK[u]+1,q.push(u2);
    }
  }
  return found;
}
bool dfsHK(int u){
  for(int v:bg[u]){
    int u2=matchR[v];
    if(!u2||(distHK[u2]==distHK[u]+1&&dfsHK(u2))
      ){
      matchL[u]=v;matchR[v]=u;return 1;
    }
  }
  distHK[u]=-1;return 0;
}
int hopcroftKarp(){
  int ans=0;
  while(bfsHK())
    for(int u=1;u<=L;u++)if(!matchL[u])ans+=
      dfsHK(u);
  return ans;
}
```

$O(E\sqrt{V})$. Min vertex cover bipartite từ matching theo König; Min path cover DAG = $n -$ maxMatching.

Hungarian - assignment min cost

Ngữ cảnh dùng: Bài assignment đầy đủ giữa hai phía có cost, thường dense. $O(n^3)$ và ngắn hơn MCMF khi mô hình đúng dạng một-một.

```
// 1-index, n rows, m>=n columns
vector<ll> u(n+1),v(m+1);
vector<int> p(m+1),way(m+1);
for(int i=1;i<=n;i++){
  p[0]=i;int j0=0;
  vector<ll> minv(m+1,LINF);
  vector<char> used(m+1,0);
  do{
    used[j0]=1;int i0=p[j0],j1=0;ll delta=LINF;
    for(int j=1;j<=m;j++)if(!used[j]){
      ll cur=cost[i0][j]-u[i0]-v[j];
      if(cur<minv[j])minv[j]=cur,way[j]=j0;
      if(minv[j]<delta)delta=minv[j],j1=j;
    }
    for(int j=0;j<=m;j++)
      if(used[j])u[p[j]]+=delta,v[j]-=delta;
      else minv[j]-=delta;
    j0=j1;
  }while(p[j0]);
  do{
    int j1=way[j0];p[j0]=p[j1];j0=j1;
  }while(j0);
}
// assignment[p[j]]=j, min cost = -v[0]
```

$O(n^2 m)$; vuông $O(n^3)$. Dùng khi assignment đầy đủ có trọng số, nhanh/gọn hơn MCMF.

Implicit Treap - split/merge

```
struct TN{
  int val,pri,sz;TN *l,*r;
  TN(int v):val(v),pri((rand()<<16)^rand()),sz
     (1),l(0),r(0){}
};
int tsz(TN*t){return t?t->sz:0;}
void pull(TN*t){if(t)t->sz=1+tsz(t->l)+tsz(t->r)
    ;}
void split(TN*t,TN*&a,TN*&b,int k){
  if(!t){a=b=0;return;}
  if(tsz(t->l)>=k)split(t->l,a,t->l,k),b=t;
  else split(t->r,t->r,b,k-tsz(t->l)-1),a=t;
  pull(t);
}
void merge(TN*&t,TN*a,TN*b){
  if(!a||!b)t=a?a:b;
  else if(a->pri>b->pri)merge(a->r,a->r,b),t=a;
  else merge(b->l,a,b->l),t=b;
  pull(t);
}
```

Implicit Treap coi vị trí là key. Range reverse/add/sum: thêm lazy tags rồi push(). Split theo số phần tử, merge giữ thứ tự.

Li Chao Tree - min line query

Ngữ cảnh dùng: Thêm nhiều đường thẳng và query min/max tại x, nhưng slope/query không có thứ tự thuận lợi cho CHT. Cần xác định miền x hoặc nén tọa độ.

```
struct Line{
  ll m,b;
  ll get(ll x)const{return m*x+b;}
};
Line lc[4*N];
bool has[4*N];
void addLine(Line nw,int id,int l,int r){
  int m=(l+r)/2;
  if(!has[id]){lc[id]=nw;has[id]=1;return;}
  bool lef=nw.get(xs[l])<lc[id].get(xs[l]);
  bool mid=nw.get(xs[m])<lc[id].get(xs[m]);
  if(mid)swap(nw,lc[id]);
  if(l==r)return;
  if(lef!=mid)addLine(nw,id*2,l,m);
  else addLine(nw,id*2+1,m+1,r);
}
ll queryLC(int pos,int id,int l,int r){
  ll ans=has[id]?lc[id].get(xs[pos]):LINF;
  if(l==r)return ans;
  int m=(l+r)/2;
  if(pos<=m)return min(ans,queryLC(pos,id*2,l,m)
    );
  return min(ans,queryLC(pos,id*2+1,m+1,r));
}
```

Coordinate-compress các x query thành xs. Add line/query $O(\log N)$. Đổi dấu hoặc so sánh để lấy max. CHT monotone ngắn hơn nếu slope/x có thứ tự.

Rollback DSU

```
int rp[N],rsz[N];
vector<pair<int,int>> hist;
void initRB(int n){for(int i=1;i<=n;i++)rp[i]=i,
     rsz[i]=1;}
int findRB(int u){while(u!=rp[u])u=rp[u];return
     u;}
int snap(){return hist.size();}
bool uniteRB(int a,int b){
  a=findRB(a);b=findRB(b);
  if(a==b){hist.push_back({-1,-1});return 0;}
  if(rsz[a]<rsz[b])swap(a,b);
  hist.push_back({b,rsz[a]});
  rp[b]=a;rsz[a]+=rsz[b];return 1;
}
void rollback(int t){
  while((int)hist.size()>t){
    auto [b,old]=hist.back();hist.pop_back();
    if(b==-1)continue;
    int a=rp[b];rsz[a]=old;rp[b]=b;
  }
}
```

Không path compression. Dùng với divide-and-conquer theo thời gian / segment tree of time cho dynamic connectivity offline.

Dynamic Connectivity offline - mẫu

```
// ỗmi edge ốsng trong interval [l,r)
// add edge vào segment tree ởthi gian
void solveTime(int id,int l,int r){
  int t=snap();
  for(auto [u,v]:bucket[id])uniteRB(u,v);
  if(l+1==r){
    // answer query at time l
  }else{
    int m=(l+r)/2;
    solveTime(id*2,l,m);
    solveTime(id*2+1,m,r);
  }
  rollback(t);
}
```

Mỗi cạnh được gắn vào $O(\log Q)$ node thời gian; tổng thường $O((Q+K)\log Q\,\alpha(N))$ bỏ hằng số rollback.

Mo's Algorithm

```
int B=sqrt(n)+1;
sort(qs.begin(),qs.end(),[&](Q a,Q b){
  int A=a.l/B,C=b.l/B;
  if(A!=C)return A<C;
  return (A&1)?a.r>b.r:a.r<b.r;
});
int L=0,R=-1;
for(auto q:qs){
  while(L>q.l)add(--L);
  while(R<q.r)add(++R);
  while(L<q.l)del(L++);
  while(R>q.r)del(R--);
  ans[q.id]=getAns();
}
```

Dùng query offline đoạn tĩnh khi add/del $O(1)$ hoặc rẻ. Complexity xấp xỉ $O((N+Q)\sqrt{N})$. Có update theo thời gian -> Mo 3D.

Fenwick 2D

```
ll bit2[N][N];
void add2(int x,int y,ll v){
  for(int i=x;i<=n;i+=i&-i)
    for(int j=y;j<=m;j+=j&-j)bit2[i][j]+=v;
}
ll sum2(int x,int y){
  ll s=0;
  for(int i=x;i;i-=i&-i)
    for(int j=y;j;j-=j&-j)s+=bit2[i][j];
  return s;
}
ll rect(int x1,int y1,int x2,int y2){
  return sum2(x2,y2)-sum2(x1-1,y2)
       -sum2(x2,y1-1)+sum2(x1-1,y1-1);
}
```

$O(\log^2 N)$. Nếu không update, Prefix 2D đơn giản hơn và query $O(1)$.

Sparse Table RMQ

```
int st[LOG][N],lg2[N];
void buildST(vector<int>&a){
  int n=a.size();
  lg2[1]=0;for(int i=2;i<=n;i++)lg2[i]=lg2[i
      /2]+1;
  for(int i=0;i<n;i++)st[0][i]=a[i];
  for(int k=1;(1<<k)<=n;k++)
    for(int i=0;i+(1<<k)<=n;i++)
      st[k][i]=min(st[k-1][i],
        st[k-1][i+(1<<(k-1))]);
}
int rmq(int l,int r){
  int k=lg2[r-l+1];
  return min(st[k][l],st[k][r-(1<<k)+1]);
}
```

Idempotent như min/max/gcd: query $O(1)$. Sum không dùng overlap kiểu này; dùng prefix hoặc disjoint sparse table.

Digit DP - khung đếm

Ngữ cảnh dùng: Đếm số trong $[0,X]$ theo tính chất của chữ số: tổng, modulo, pattern, số lần xuất hiện. State thường gồm pos/tight/started/thông tin phụ.

```
string S;
ll memo[20][2][2][STATE];
ll dfsDigit(int pos,bool tight,bool started,int
     state){
  if(pos==(int)S.size())return accept(state,
     started);
  ll &res=memo[pos][tight][started][state];
  if(!tight&&res!=-1)return res;
  ll ans=0;int lim=tight?S[pos]-'0':9;
  for(int d=0;d<=lim;d++){
    bool ns=started||d;
    int nstate=trans(state,d,ns);
    if(nstate<0)continue;
    ans+=dfsDigit(pos+1,tight&&d==lim,ns,nstate)
      ;
  }
  if(!tight)res=ans;
  return ans;
}
ll solve(ll x){
  if(x<0)return 0;
  S=to_string(x);memset(memo,-1,sizeof memo);
  return dfsDigit(0,1,0,INIT);
}
// answer [L,R]=solve(R)-solve(L-1)
```

State thường chứa: modulo, tổng chữ số, chữ số trước, automaton state, mask chữ số đã dùng. Memo chỉ khi `tight=0`.

Profile DP / Broken Profile DP

```
// domino tiling n x m, m ỏnh
void gen(int row,int mask,int nextMask,ll ways){
  if(row==m){ndp[nextMask]+=ways;return;}
  if(mask>>row&1)gen(row+1,mask,nextMask,ways);
  else{
    // domino ọdc sang ộct ếk
    gen(row+1,mask,nextMask|(1<<row),ways);
    // domino ngang trong ộct
    if(row+1<m && !(mask>>(row+1)&1))
      gen(row+2,mask,nextMask,ways);
  }
}
dp[0]=1;
for(int col=0;col<n;col++){
  fill(ndp.begin(),ndp.end(),0);
  for(int mask=0;mask<(1<<m);mask++)if(dp[mask])
    gen(0,mask,0,dp[mask]);
  swap(dp,ndp);
}
```

Dùng khi một chiều nhỏ ($m \lesssim 10$–15), quét theo cột/hàng, mask mô tả biên chưa xử lý.

Divide & Conquer DP Optimization

Ngữ cảnh dùng: DP dạng $dp[g][i]=\min_k(prev[k]+cost(k+1,i))$ và vị trí tối ưu không giảm theo i. Phải chứng minh monotone opt trước.

```
// dp[g][mid] = min_{k<mid}(prev[k]+cost(k+1,mid
     ))
// ầcn opt[g][i] không ảgim theo i
void compute(int l,int r,int optL,int optR){
  if(l>r)return;
  int mid=(l+r)/2,best=-1;
  pair<ll,int> cur={LINF,-1};
  for(int k=optL;k<=min(mid-1,optR);k++)
    cur=min(cur,{prev[k]+cost(k+1,mid),k});
  dp[mid]=cur.first;best=cur.second;
  compute(l,mid-1,optL,best);
  compute(mid+1,r,best,optR);
}
```

Giảm từ $O(KN^2)$ xuống $O(KN\log N)$ hoặc gần $O(KN)$ tùy cost. Chỉ dùng khi chứng minh monotonicity của vị trí tối ưu.

Knuth Optimization

Ngữ cảnh dùng: Interval DP dạng split và thỏa quadrangle inequality/Monge cùng điều kiện opt lồng nhau. Nếu không chứng minh được, không dùng.

```
// dp[l][r]=min_k dp[l][k]+dp[k][r]+C(l,r)
// opt[l][r-1] <= opt[l][r] <= opt[l+1][r]
for(int i=0;i<n;i++)opt[i][i+1]=i;
for(int len=2;len<=n;len++)
 for(int l=0;l+len<=n;l++){
  int r=l+len;dp[l][r]=LINF;
  for(int k=opt[l][r-1];k<=opt[l+1][r];k++){
    ll v=dp[l][k]+dp[k][r]+C(l,r);
    if(v<dp[l][r])dp[l][r]=v,opt[l][r]=k;
  }
 }
```

Từ $O(N^3)$ xuống $O(N^2)$. Cần quadrangle inequality/Monge và điều kiện opt. Hay gặp merge intervals, optimal BST biến thể.

Bitset DP - Subset Sum

```
bitset<MAXS+1> dp;
dp[0]=1;
for(int x:a)dp|=(dp<<x);
// dp[s]=1 iff có subset sum = s
```

Rất nhanh do xử lý theo word CPU. Có thể dùng bitset cho reachability, LCS bitset nâng cao, transitive operations.

SCC condensation -> DAG DP

```
// sau SCC: comp[u] in [0..C-1]
vector<ll>wc(C);
for(int u=1;u<=n;u++)wc[comp[u]]+=w[u];
vector<vector<int>> dag(C);
for(auto [u,v]:edges)if(comp[u]!=comp[v])
  dag[comp[u]].push_back(comp[v]);
// topo + DP longest/max-sum
for(int u:topo){
  dp[u]=max(dp[u],wc[u]);
  for(int v:dag[u])dp[v]=max(dp[v],dp[u]+wc[v]);
}
```

Condensation graph luôn là DAG. Pattern rất hay: graph có cycle -> SCC -> gom weight -> DAG DP.

Floyd Cycle Finding - tìm chu kỳ

```
pair<ll,ll> floydCycle(ll x0){
  ll tort=f(x0),hare=f(f(x0));
  while(tort!=hare)tort=f(tort),hare=f(f(hare));
  ll mu=0;hare=x0;
  while(tort!=hare)tort=f(tort),hare=f(hare),mu
     ++;
  ll lam=1;hare=f(tort);
  while(tort!=hare)hare=f(hare),lam++;
  return {mu,lam};
}
```

$O(\mu+\lambda)$ thời gian, $O(1)$ bộ nhớ. Dùng recurrence/-function graph khi chỉ cần vị trí bắt đầu cycle và độ dài cycle.

Miller-Rabin 64-bit

Ngữ cảnh dùng: Cần primality cho số 64-bit lớn, không thể sieve đến n. Bộ base deterministic phải đúng cho miền 64-bit.

```
using u128=__uint128_t;
ull mulmod(ull a,ull b,ull m){return (u128)a*b%m
     ;}
ull powmod(ull a,ull d,ull m){
  ull r=1;
  while(d){if(d&1)r=mulmod(r,a,m);a=mulmod(a,a,m
     );d>>=1;}
  return r;
}
bool isPrime64(ull n){
  if(n<2)return 0;
  for(ull p:{2,3,5,7,11,13,17,19,23,29,31,37})
    if(n%p==0)return n==p;
  ull d=n-1,s=0;while(!(d&1))d>>=1,s++;
  for(ull a:{2ULL,325ULL,9375ULL,28178ULL,450775
     ULL,
           9780504ULL,1795265022ULL}){
    if(a%n==0)continue;
    ull x=powmod(a%n,d,n);
    if(x==1||x==n-1)continue;
    bool comp=1;
    for(ull r=1;r<s;r++){
      x=mulmod(x,x,n);
      if(x==n-1){comp=0;break;}
    }
    if(comp)return 0;
  }
  return 1;
}
```

Bộ base trên deterministic cho unsố nguyên có dấu 64-bit. Dùng `__int128` để tránh overflow multiplication.

Pollard-Rho 64-bit

Ngữ cảnh dùng: Cần factor số 64-bit lớn sau khi Miller-Rabin xác định composite. Randomized; nếu thất bại/repeat factor n thì restart.

```
ull rho(ull n){
  if(n%2==0)return 2;
  static mt19937_64 rng(chrono::steady_clock::
     now().time_since_epoch().count());
  while(1){
    ull c=rng()%(n-1)+1,x=rng()%n,y=x,d=1;
    auto F=[&](ull v){return (mulmod(v,v,n)+c)%n
     ;};
    while(d==1){
      x=F(x);y=F(F(y));
      d=gcd<ull>(x>y?x-y:y-x,n);
    }
    if(d!=n)return d;
  }
}
void factor(ull n,vector<ull>&f){
  if(n==1)return;
  if(isPrime64(n)){f.push_back(n);return;}
  ull d=rho(n);factor(d,f);factor(n/d,f);
}
```

Factor 64-bit lớn khi trial division/SPF không đủ. Sau factor sort để gom số mũ.

Discrete Log - Baby Step Giant Step

Ngữ cảnh dùng: Giải $a^x \equiv b \pmod m$ với modulus vừa, thường khi gcd(a,m)=1. Memory/time $O(\sqrt{m})$.

```
// gcd(a,m)=1: find x: a^x = b mod m
ll bsgs(ll a,ll b,ll m){
  a%=m;b%=m;
  ll n=sqrt((ld)m)+1,e=1;
  unordered_map<ll,ll> baby;
  for(ll j=0;j<n;j++){
    if(!baby.count(e))baby[e]=j;
    e=(__int128)e*a%m;
  }
  ll an=binpow(a,n,m);
  ll inv=binpow(an,phi(m)-1,m); // replace by
      exgcd inverse if needed
  ll cur=b;
  for(ll i=0;i<=n;i++){
    if(baby.count(cur))return i*n+baby[cur];
    cur=(__int128)cur*inv%m;
  }
  return -1;
}
```

Template trên giả sử có modular inverse; với mod prime dùng a^{m-2}. General BSGS cần xử lý gcd trước. Complexity $O(\sqrt{m})$.

Lucas Theorem

```
// p prime ỏnh, precompute C[0..p-1][0..p-1]
ll lucas(ll n,ll k,int p){
  if(k==0)return 1;
  return lucas(n/p,k/p,p,p)*C[n%p][k%p]%p;
}
```

$\binom{n}{k} \bmod p = \prod \binom{n_i}{k_i} \bmod p$ theo chữ số base p. Hữu ích khi n, k rất lớn nhưng p nhỏ prime.

NTT - convolution mod 998244353

Ngữ cảnh dùng: Cần convolution đa thức lớn chính xác modulo phù hợp. Với vài nghìn hệ số, $O(N^2)$ đôi khi đơn giản và đủ nhanh.

```
const int MODN=998244353,G=3;
void ntt(vector<int>&a,bool inv){
  int n=a.size();
  for(int i=1,j=0;i<n;i++){
    int b=n>>1;for(;j&b;b>>=1)j^=b;j^=b;
    if(i<j)swap(a[i],a[j]);
  }
  for(int len=2;len<=n;len<<=1){
    int wlen=binpow(G,(MODN-1)/len,MODN);
    if(inv)wlen=binpow(wlen,MODN-2,MODN);
    for(int i=0;i<n;i+=len){
      ll w=1;
      for(int j=0;j<len/2;j++){
        int u=a[i+j],v=w*a[i+j+len/2]%MODN;
        a[i+j]=(u+v)%MODN;
        a[i+j+len/2]=(u-v+MODN)%MODN;
        w=w*wlen%MODN;
      }
    }
  }
  if(inv){
    int ni=binpow(n,MODN-2,MODN);
    for(int &x:a)x=(ll)x*ni%MODN;
  }
}
vector<int> conv(vector<int>a,vector<int>b){
  int n=1;while(n<(int)a.size()+(int)b.size()-1)
      n<<=1;
  a.resize(n);b.resize(n);ntt(a,0);ntt(b,0);
  for(int i=0;i<n;i++)a[i]=(ll)a[i]*b[i]%MODN;
  ntt(a,1);return a;
}
```

Convolution $O(N \log N)$. Mod 998244353 hỗ trợ power-of-two đến 2^{23}.

Lagrange Interpolation - $O(k^2)$

```
// points (x[i],y[i]), evaluate at X mod MOD
ll lagrange(vector<ll>x,vector<ll>y,ll X){
  int n=x.size();ll ans=0;
  for(int i=0;i<n;i++){
    ll num=1,den=1;
    for(int j=0;j<n;j++)if(i!=j){
      num=num*(X-x[j]+MOD)%MOD;
      den=den*(x[i]-x[j]+MOD)%MOD;
    }
    ans=(ans+y[i]*num%MOD*binpow(den,MOD-2,MOD))
      %MOD;
  }
  return ans;
}
```

Đa thức bậc $< k$ xác định bởi k điểm. Nếu các $x_i = 0, 1, \ldots, k-1$ có bản $O(k)$ với prefix/suffix product.

Berlekamp-Massey - nhớ pattern

Nếu có đủ prefix của dãy tuyến tính modulo prime nhưng không biết recurrence: BM tìm recurrence ngắn nhất trong $O(N^2)$. Sau đó dùng linear recurrence / Kitamasa để tính S_k trong $O(L^2 \log k)$. Chỉ nên thêm full code nếu contest thường ra recurrence; KACTL có template tốt cho phần này.

Gaussian Elimination - hệ thực

```
int gauss(vector<vector<ld>> a,vector<ld>&ans){
  int n=a.size(),m=a[0].size()-1,row=0;
  vector<int> where(m,-1);
  for(int col=0;col<m&&row<n;col++){
    int sel=row;
    for(int i=row;i<n;i++)
      if(fabsl(a[i][col])>fabsl(a[sel][col]))sel
        =i;
    if(fabsl(a[sel][col])<EPS)continue;
    swap(a[sel],a[row]);where[col]=row;
    for(int i=0;i<n;i++)if(i!=row){
      ld c=a[i][col]/a[row][col];
      for(int j=col;j<=m;j++)a[i][j]-=a[row][j]*
       c;
    }
    row++;
  }
  ans.assign(m,0);
  for(int i=0;i<m;i++)if(where[i]!=-1)
    ans[i]=a[where[i]][m]/a[where[i]][i];
  for(int i=0;i<n;i++){
    ld s=0;for(int j=0;j<m;j++)s+=ans[j]*a[i][j
      ];
    if(fabsl(s-a[i][m])>1e-7)return 0; // no
      solution
  }
  for(int i=0;i<m;i++)if(where[i]==-1)return 2;
      // infinite
  return 1;
}
```

Modulo prime: thay division bằng modular inverse, tránh EPS. XOR linear equations: Gaussian trên bitset/GF(2).

Hình học - giao đường / hình chiếu

```
PD add(PD a,PD b){return {a.ft+b.ft,a.sd+b.sd};}
PD mul(PD a,ld k){return {a.ft*k,a.sd*k};}
PD subD(PD a,PD b){return {a.ft-b.ft,a.sd-b.sd
    };}
ld dotD(PD a,PD b){return a.ft*b.ft+a.sd*b.sd;}
ld crD(PD a,PD b){return a.ft*b.sd-a.sd*b.ft;}
bool lineInter(PD a,PD b,PD c,PD d,PD &p){
  PD r=subD(b,a),s=subD(d,c);
  ld z=crD(r,s);
  if(fabsl(z)<EPS)return false;
  ld t=crD(subD(c,a),s)/z;
  p=add(a,mul(r,t));return true;
}
PD projection(PD a,PD b,PD p){
  PD ab=subD(b,a);
  return add(a,mul(ab,dotD(subD(p,a),ab)/dotD(ab
      ,ab)));
}
PD reflection(PD a,PD b,PD p){
  PD q=projection(a,b,p);
  return add(q,subD(q,p));
}
```

Không dùng slope để tránh vertical line. Segment intersection vẫn cần orientation + onSegment riêng.

Giao hai đường tròn - code

```
vector<PD> circleInter(PD a,ld r,PD b,ld R){
  PD d=subD(b,a);ld D=sqrt(dotD(d,d));
  if(D>r+R+EPS||D+min(r,R)<max(r,R)-EPS||D<EPS)
      return {};
  ld x=(D*D+r*r-R*R)/(2*D);
  ld h2=r*r-x*x;if(h2<0&&h2>-EPS)h2=0;
  if(h2<0)return {};
  PD u=mul(d,1/D),p=add(a,mul(u,x));
  PD per={-u.sd,u.ft};ld h=sqrt(h2);
  if(h<EPS)return {p};
  return {add(p,mul(per,h)),add(p,mul(per,-h))};
}
```

Trường hợp hai circle trùng hoàn toàn có vô số giao điểm, cần xử lý riêng nếu đề hỏi.

Minkowski Sum - polygon lồi

```
// A,B CCW, không ặlp đểim ốcui; rotate để đểim
      ấthp-trái đầu
// ạto list vector ạcnh, merge theo góc ằbng
      cross
vector<P> minkowski(vector<P>A,vector<P>B){
  normalize(A);normalize(B);
  vector<P> ea,eb;
  for(int i=0;i<sz(A);i++)ea.pb(A[(i+1)%sz(A)]-A
      [i]);
  for(int i=0;i<sz(B);i++)eb.pb(B[(i+1)%sz(B)]-B
      [i]);
  vector<P> C={A[0]+B[0]};int i=0,j=0;
  while(i<sz(ea)||j<sz(eb)){
    if(j==sz(eb)||(i<sz(ea)&&cross(ea[i],eb[j])
      >0))
      C.pb(C.back()+ea[i++]);
    else if(i==sz(ea)||(j<sz(eb)&&cross(ea[i],eb
      [j])<0))
      C.pb(C.back()+eb[j++]);
    else C.pb(C.back()+ea[i++]+eb[j++]);
  }
  C.pop_back();return C;
}
```

Ý tưởng quan trọng hơn code: tổng Minkowski của hai polygon lồi = merge vector cạnh theo góc. Dùng collision/configuration space, distance giữa convex polygons.

Half-plane Intersection - nhận diện

Dùng khi nghiệm là giao của nhiều bất phương trình tuyến tính/half-planes. Sort line theo góc, dùng deque, loại giao điểm nằm ngoài half-plane mới. Cẩn code dài và cực nhạy EPS; chỉ nên giữ full template nếu contest thường có geometry khó. Với N nhỏ, polygon clipping lần lượt từng half-plane dễ code hơn.

Closest Pair - $O(N \log N)$ skeleton

```
ld closest(vector<PD>&p,int l,int r){
  if(r-l<=3){
    ld d=1e100;
    for(int i=l;i<r;i++)for(int j=i+1;j<r;j++)
      d=min(d,dist(p[i],p[j]));
    sort(p.begin()+l,p.begin()+r,[](PD a,PD b){
      return a.sd<b.sd;});
    return d;
  }
  int m=(l+r)/2;ld x=p[m].ft;
  ld d=min(closest(p,l,m),closest(p,m,r));
  inplace_merge(p.begin()+l,p.begin()+m,p.begin
      ()+r,
    [](PD a,PD b){return a.sd<b.sd;});
  vector<PD>s;
  for(int i=l;i<r;i++)if(fabsl(p[i].ft-x)<d){
    for(int j=sz(s)-1;j>=0&&p[i].sd-s[j].sd<d;j
      --)
      d=min(d,dist(p[i],s[j]));
    s.pb(p[i]);
  }
  return d;
}
```

Ban đầu sort theo x. Trong recursion giữ đoạn sort theo y. Strip chỉ cần kiểm tra vài điểm trước gần nhất.

Second-best MST

Kruskal lấy MST và đánh dấu cạnh. Build LCA với max-edge trên path MST. Với mỗi cạnh ngoài MST (u, v, w): ứng viên $= MST + w -$ maxEdge(u, v). Lấy min ứng viên hợp lệ. $O(E \log V)$.

Minimax Path

Trong MST, đường đi giữa u, v tối thiểu hóa cạnh lớn nhất (bottleneck) giữa mọi path trong graph. Vì vậy query minimax = max-edge trên path MST bằng LCA/Binary Lifting.

2-SAT - SCC assignment

```
// literal x: node 2*x, not x: 2*x^1 (ọchn
     convention ấnht quán)
void add0r(int a,int b){
  // a OR b => !a -> b, !b -> a
  g[a^1].pb(b);g[b^1].pb(a);
}
// sau SCC:
for(int x=0;x<n;x++){
  if(comp[2*x]==comp[2*x^1]) impossible;
  val[x]=comp[2*x]>comp[2*x^1]; // tùy ứth ựt
     SCC
}
```

Các ràng buộc: $a \Rightarrow b$ -> edge $a \to b$ và $\neg b \to \neg a$; XOR/equals có thể tách thành 2 clauses. Kiểm tra orientation assignment theo SCC implementation.

Stress test / brute checker

```
for(int tc=1;;tc++){
  auto in=genSmall();
  auto a=solveFast(in);
  auto b=solveBrute(in);
  if(a!=b){
    cerr<<"FAIL "<<tc<<"\n";
    print(in);break;
  }
}
```

Trước contest hoặc khi có local judge: random nhỏ + brute force là cách tốt nhất bắt bug ở DS/DP/geometry. Cố tình sinh corner cases: n=1, duplicate, all equal, disconnected, overflow, collinear.

Overflow / số học an toàn

```
// ceil a/b ớvi a,b > 0
ll ceilDiv(ll a,ll b){return (a+b-1)/b;}
// normalize mod
ll norm(ll x,ll m){x%=m;if(x<0)x+=m;return x;}
// multiplication tránh overflow
ll mul(ll a,ll b,ll m){return (int128)a*b%m;}
// in __int128
void print128(int128 x){
  if(x<0)cout<<'-',x=-x;
  string s;do{s+=char('0'+x%10);x/=10;}while(x);
  reverse(all(s));cout<<s;
}
```

$10^9 \cdot 10^9 \sim 10^{18}$ sát giới hạn số nguyên có dấu 64-bit; cộng nhiều lần dễ overflow. Dùng __int128 cho cross với tọa độ lớn, product modulo, tổng tích.

Checklist nhận diện trước khi code

Đồ thị: unweighted -> BFS; 0/1 -> deque; nonnegative -> Dijkstra; negative -> Bellman-Ford; mọi cặp nhỏ -> Floyd; DAG -> topo DP; cycles có cấu trúc -> SCC/function graph; connectivity offline -> DSU/rollback DSU.

Range query: static sum -> prefix; static idempotent -> Sparse Table; point update + prefix/range -> Fenwick; range/complex aggregate -> Segment Tree; kth/order -> frequency Segment/PBDS; lines -> CHT/Li Chao.

DP: $n \leq 20$ -> bitmask; $n \approx 40$ -> MITM; interval -> len ngoài; small width grid -> profile DP; huge numeric bound -> Digit DP; monotone opt -> D&C/Knuth/CHT.

String: 1 pattern -> KMP/Z; palindrome -> Manacher/hash; many patterns -> Trie/Aho; suffix queries -> SA/LCP/SAM.

Math: mod prime -> Fermat/invfact; 64-bit primality -> Miller; factor 64-bit -> Pollard; recurrence huge k -> matrix/BM; convolution -> NTT.

Geometry: integer first -> cross/orient with __int128; floating -> EPS+clamp; polygon -> shoelace/hull; circles -> atan2/sin/cos; convex query -> binary search/-calipers.

Checklist WA/TLE - cuối cùng

1) Đọc đúng directed/undirected, multi-edge, self-loop. 2) INF đủ lớn nhưng INF+w không overflow. 3) Dijkstra không có cạnh âm. 4) Reset global arrays giữa test. 5) đánh số 0/1 và $[l,r]$ vs $[l,r)$. 6) Comparator phải strict weak ordering. 7) Recursion sâu $2e5$ có thể stack overflow. 8) Mod âm phải normalize. 9) Số thực: fabsl(x)<EPS, clamp trước acos, tránh -0. 10) Geometry integer: cross có thể cần __int128. 11) Binary search: invariant + tránh infinite loop. 12) DP loop direction: 0/1 giảm, unbounded tăng. 13) Segment identity đúng (sum=0,min=INF,max=-INF,gcd=0). 14) Flow residual/reverse edge đúng index. 15) SCC assignment 2-SAT phụ thuộc order component. 16) Hash collision: double/64-bit nếu quan trọng. 17) Coordinate compression nhớ unique. 18) In output đúng precision; setprecision có/không fixed khác nhau.

MATH TOOLKIT - Linear Sieve + SPF + phi + mu

```
vector<int> primes;
int spf[N],phiA[N],muA[N];
void linearSieve(int n){
  phiA[1]=1;muA[1]=1;
  for(int i=2;i<=n;i++){
    if(!spf[i]){
      spf[i]=i;primes.pb(i);
      phiA[i]=i-1;muA[i]=-1;
    }
    for(int p:primes){
      if(p>spf[i]||1LL*i*p>n)break;
      spf[i*p]=p;
      if(i%p==0){
        phiA[i*p]=phiA[i]*p;
        muA[i*p]=0;
        break;
      }else{
        phiA[i*p]=phiA[i]*(p-1);
        muA[i*p]=-muA[i];
      }
    }
  }
}
```

Một lần $O(N)$ có primes + smallest prime factor + Euler phi + Möbius. Factor nhiều số $x \le N$ bằng lặp spf[x].

Factorization bằng SPF + sinh ước

```
vector<pair<int,int>> factorSPF(int x){
  vector<pair<int,int>> f;
  while(x>1){
    int p=spf[x],e=0;
    while(x%p==0)x/=p,e++;
    f.pb({p,e});
  }
  return f;
}
void genDiv(int i,ll cur,vector<pair<int,int>>&f
    ,vector<ll>&d){
  if(i==sz(f)){d.pb(cur);return;}
  auto [p,e]=f[i];ll q=1;
  for(int k=0;k<=e;k++){
    genDiv(i+1,cur*q,f,d);
    q*=p;
  }
}
```

Nếu $n = \prod p_i^{a_i}$: số ước $\tau(n) = \prod(a_i+1)$; tổng ước $\sigma(n) = \prod(1 + p_i + \cdots + p_i^{a_i})$.

Divisor Analysis - number / sum / product

```
// input prime powers p^a, MOD prime
ll cnt=1,sum=1,prod=1;
ll cntExp=1; // divisor count modulo MOD-1 for
     exponent
for(auto [p,a]:fac){
  cnt=cnt*(a+1)%MOD;
  ll geo=(binpow(p,a+1,MOD)-1+MOD)%MOD;
  geo=geo*binpow(p-1,MOD-2,MOD)%MOD;
  sum=sum*geo%MOD;
}
// product divisors:
// n^(tau/2), but handle odd tau without
     dividing exponent badly
bool oddTau=true;
for(auto [p,a]:fac) if(a&1) oddTau=false;
if(oddTau){
  // all exponents even -> sqrt(n)^tau
  ll root=1,tau=1;
  for(auto [p,a]:fac){
    root=root*binpow(p,a/2,MOD)%MOD;
    tau=tau*(a+1)%(MOD-1);
  }
  prod=binpow(root,tau,MOD);
}else{
  ll nmod=1,tauHalf=1;
  bool halved=false;
  for(auto [p,a]:fac){
    nmod=nmod*binpow(p,a,MOD)%MOD;
    ll x=a+1;
    if(!halved && x%2==0)x/=2,halved=true;
    tauHalf=tauHalf*(x%(MOD-1))%(MOD-1);
  }
  prod=binpow(nmod,tauHalf,MOD);
}
```

Pattern CSES Divisor Analysis: exponent của lũy thừa giảm modulo $MOD-1$ theo Fermat, nhưng chia 2 phải thực hiện trên một thừa số chẵn trước khi modulo.

Maximum Common Divisor - đếm bội

```
vector<int> freq(M+1);
for(int x:a)freq[x]++;
for(int d=M;d>=1;d--){
  int cnt=0;
  for(int x=d;x<=M;x+=d)cnt+=freq[x];
  if(cnt>=2){cout<<d;break;}
}
```

Tìm gcd lớn nhất của một cặp: đếm bao nhiêu phần tử chia hết cho d. Complexity $O(M \log M)$ kiểu harmonic series.

Counting Coprime Pairs - Möbius

```
vector<ll> cntDiv(M+1);
for(int d=1;d<=M;d++)
  for(int x=d;x<=M;x+=d)cntDiv[d]+=freq[x];
ll ans=0;
for(int d=1;d<=M;d++)
  ans += 1LL*muA[d]*cntDiv[d]*(cntDiv[d]-1)/2;
```

$[\gcd(a,b)=1] = \sum_{d \mid a,b} \mu(d)$. Đây là pattern kinh điển biến điều kiện gcd thành tổng theo ước.

Prime Multiples - Inclusion Exclusion

```
ll ans=0;
for(int mask=1;mask<(1<<k);mask++){
  __int128 mult=1;int bits=0;bool ok=1;
  for(int i=0;i<k;i++)if(mask>>i&1){
    bits++;
    mult*=p[i];
    if(mult>n){ok=0;break;}
  }
  if(!ok)continue;
  ll add=n/(ll)mult;
  if(bits&1)ans+=add;
  else ans-=add;
}
```

Đếm số $\le n$ chia hết ít nhất một prime. $k \le 20$ thường dùng 2^k. Luôn chặn overflow tích bằng __int128.

Gom thương floor - tổng nhanh

```
for(ll l=1,r;l<=n;l=r+1){
  ll q=n/l;
  r=n/q;
  // floor(n/x)=q for all x in [l,r]
  // contribution = q * sum(l..r)
}
```

Chỉ $O(\sqrt{n})$ block. Dùng cho $\sum \lfloor n/i \rfloor$, tổng ước, đếm lattice points và nhiều bài number theory có quotient.

AtCoder floor_sum - tổng floor tuyến tính

```
// sum_{0<=i<n} floor((a*i+b)/m), all >=0
long long floorSum(ll n,ll m,ll a,ll b){
  ll ans=0;
  while(1){
    if(a>=m){
      ans+=(n-1)*n*(a/m)/2;
      a%=m;
    }
    if(b>=m){
      ans+=n*(b/m);
      b%=m;
    }
    ll y=a*n+b;
    if(y<m)break;
    n=y/m;
    b=y%m;
    swap(m,a);
  }
  return ans;
}
```

Rất hay khi đếm điểm nguyên dưới đường thẳng / tổng floor dạng affine. Với số âm cần normalize biến đổi trước.

Linear Congruence $ax \equiv b \pmod m$

```
vector<ll> solveCong(ll a,ll b,ll m){
  ll g=gcd(a,m);
  if(b%g)return {};
  a/=g;b/=g;m/=g;
  ll inv=modInverse(a,m); // exgcd
  ll x0=(__int128)b*inv%m;
  vector<ll> sol;
  for(ll k=0;k<g;k++)sol.pb(x0+k*m);
  return sol; // modulo original modulus
}
```

Có nghiệm iff $g = \gcd(a,m)$ chia b. Khi có, có đúng g nghiệm khác nhau modulo modulus ban đầu.

CRT nhiều phương trình

```
// merge x=a1 mod m1, x=a2 mod m2
bool crtMerge(ll a1,ll m1,ll a2,ll m2,ll &a,ll &
     m){
  ll x,y;ll g=extgcd(m1,m2,x,y);
  ll d=a2-a1;
  if(d%g)return false;
  ll mod=m2/g;
  ll t=(__int128)(d/g)*x%mod;
  if(t<0)t+=mod;
  __int128 l=(__int128)m1/g*m2;
  __int128 res=a1+(__int128)m1*t;
  res%=l;if(res<0)res+=l;
  a=(ll)res;m=(ll)l;return true;
}
```

Merge tuần tự để xử lý nhiều đồng dư, kể cả mod không nguyên tố cùng nhau. Cẩn thận LCM vượt 64-bit.

Exponentiation II - tháp số mũ

```
// a^(b^c) mod MOD, MOD prime
ll e=binpow(b,c,MOD-1);
ll ans=binpow(a,e,MOD);
```

Fermat giảm số mũ modulo $MOD-1$ khi $a \not\equiv 0 \pmod{MOD}$. Nếu base có thể chia hết MOD và exponent có thể bằng 0, phải xử lý special case rõ ràng.

Fast Doubling Fibonacci

```
// returns {F(n),F(n+1)}
pair<ll,ll> fib(ll n){
  if(!n)return {0,1};
  auto [a,b]=fib(n>>1);
  ll c=a*((2*b%MOD-a+MOD)%MOD)%MOD;
  ll d=(a*a%MOD+b*b%MOD)%MOD;
  if(n&1)return {d,(c+d)%MOD};
  return {c,d};
}
```

$O(\log n)$, ngắn hơn matrix exponentiation nếu chỉ cần Fibonacci. Công thức: $F_{2k} = F_k(2F_{k+1} - F_k)$, $F_{2k+1} = F_k^2 + F_{k+1}^2$.

Kitamasa / Linear recurrence - nhận diện

Nếu $S_n = c_1 S_{n-1} + \cdots + c_k S_{n-k}$ và k nhỏ nhưng n cực lớn: matrix $O(k^3 \log n)$ hoặc Kitamasa $O(k^2 \log n)$. Nếu chỉ biết prefix của dãy mà không biết recurrence: Berlekamp-Massey -> Kitamasa.

Matrix exponent - đúng k bước

```
// adjacency matrix A
// A^k[i][j] = number of walks exactly k edges
// min-plus:
C[i][j]=min_k(A[i][k]+B[k][j]);
// power min-plus => shortest path with exactly
     k edges
```

Graph Paths I: thường matrix nhân modulo. Graph Paths II: đổi semiring sang min-plus, identity có 0 trên diagonal và INF ngoài diagonal.

Factorial / invFactorial / nCk

```
ll fact[N],ifact[N];
void initComb(int n){
  fact[0]=1;
  for(int i=1;i<=n;i++)fact[i]=fact[i-1]*i%MOD;
  ifact[n]=binpow(fact[n],MOD-2,MOD);
  for(int i=n;i;i--)ifact[i-1]=ifact[i]*i%MOD;
}
ll C(int n,int k){
  if(k<0||k>n)return 0;
  return fact[n]*ifact[k]%MOD*ifact[n-k]%MOD;
}
ll P(int n,int k){
  if(k<0||k>n)return 0;
  return fact[n]*ifact[n-k]%MOD;
}
```

MOD prime và $n < MOD$. Nếu $n \geq MOD$, factorial có thể thành 0: nghĩ Lucas / prime-power / CRT tùy đề.

Multinomial / Creating Strings

```
// number of distinct permutations of multiset
ll ways=fact[n];
for(int c:freq)ways=ways*ifact[c]%MOD;
```

$n!/\prod f_i!$. Pattern classic cho hoán vị chuỗi có ký tự lặp.

Stars and Bars / Distributing Apples

$x_1 + \cdots + x_k = n,\ x_i \geq 0$: $\binom{n+k-1}{k-1}$.

$x_i \geq 1$: đặt $y_i = x_i - 1$ -> $\binom{n-1}{k-1}$.

Có lower bound $x_i \geq L_i$: trừ $\sum L_i$ trước.

Có upper bound: dùng inclusion-exclusion.

Derangements / Christmas Party

```
D[0]=1;D[1]=0;
for(int n=2;n<=N;n++)
  D[n]=(n-1)*(D[n-1]+D[n-2])%MOD;
```

Số hoán vị không có fixed point. Công thức inclusion-exclusion: $D_n = n! \sum_{k=0}^{n} (-1)^k / k!$.

Catalan / Bracket Sequences

```
// valid bracket sequences with n pairs
ll catalan(int n){
  return C(2*n,n)*binpow(n+1,MOD-2,MOD)%MOD;
}
```

Catalan còn đếm BST shapes, triangulations, Dyck paths. Với prefix bracket đã cho: kiểm tra balance không âm; phần còn lại là ballot/reflection formula hoặc DP nếu constraints nhỏ.

Ballot / Reflection Principle

Số đường đi từ $(0,0)$ đến (a,b) bằng bước R/U và không vượt qua diagonal (với $a \geq b$): $\binom{a+b}{b} - \binom{a+b}{b-1}$.

Đây là "tất cả - đường xấu phản xạ", gốc của nhiều công thức bracket/Catalan.

Stirling số loại 2 / Bell

```
S[0][0]=1;
for(int n=1;n<=N;n++)
  for(int k=1;k<=n;k++)
    S[n][k]=(S[n-1][k-1]+1LL*k*S[n-1][k])%MOD;
// Bell[n] = sum_k S[n][k]
```

$S(n,k)$ = chia n phần tử phân biệt thành k nhóm không rỗng, không thứ tự. Surjection từ n phần tử sang k nhãn = $k!\,S(n,k)$.

Inclusion-Exclusion - onto / cấm vị trí

```
// number of onto functions [n] -> [k]
ll onto=0;
for(int i=0;i<=k;i++){
  ll term=C(k,i)*binpow(k-i,n,MOD)%MOD;
  if(i&1)onto=(onto-term+MOD)%MOD;
  else onto=(onto+term)%MOD;
}
```

PIE: đếm tất cả rồi cộng/trừ giao của tập "vi phạm". Kinh điển: derangement, prime multiples, onto functions, upper-bound stars and bars.

Burnside - vòng cổ / phép quay

```
// necklaces length n, k colors, rotations
     considered same
ll sum=0;
for(int r=0;r<n;r++)
  sum=(sum+binpow(k,gcd(n,r),MOD))%MOD;
ll ans=sum*binpow(n,MOD-2,MOD)%MOD;
```

Burnside: số quỹ đạo = trung bình số cấu hình cố định bởi mỗi phép đối xứng. Với cả reflection (bracelets) phải cộng thêm fixed configurations của các phép lật.

Eulerian numbers - ascents/descents

```
E[1][0]=1;
for(int n=2;n<=N;n++)
 for(int k=0;k<n;k++)
  E[n][k]=((k+1LL)*E[n-1][k]
    +(n-k)* (k?E[n-1][k-1]:0))%MOD;
```

$E(n,k)$ đếm permutation của n phần tử có đúng k ascents (tùy convention). Hiếm nhưng hữu ích khi đề nói số lần tăng/giảm trong permutation.

Josephus

```
// 0-index survivor
ll josephus(ll n,ll k){
  ll r=0;
  for(ll i=2;i<=n;i++)r=(r+k)%i;
  return r;
}
```

$O(n)$ survivor. Với $k = 2$: survivor 1-index $= 2(n - 2^{\lfloor \log_2 n \rfloor}) + 1$. Nếu cần thứ tự loại bỏ toàn bộ, dùng order-statistics/PBDS/Fenwick.

Expected Value - tuyến tính kỳ vọng

$E[X+Y] = E[X] + E[Y]$ dù X, Y không độc lập.

Đếm số sự kiện: đặt indicator $I_i \in \{0,1\}$, $E[\sum I_i] = \sum P(I_i = 1)$.

Expected inversions: $E = \sum_{i<j} P(X_i > X_j)$. Đây thường biến bài xác suất thành tổng cặp đơn giản.

Expected steps DP

```
// E[s] = expected steps to terminal
// E[s] = 1 + sum p(s->t)*E[t]
// if self-loop probability q:
E[s]=(1 + sum_{t!=s} p[s][t]*E[t])/(1-q);
```

Nếu dependencies tạo DAG, tính ngược topo. Nếu có cycles tổng quát, lập hệ tuyến tính rồi Gaussian elimination.

Coupon Collector

Kỳ vọng lấy đủ n loại coupon đồng xác suất: $E = n(1 + \frac{1}{2} + \cdots + \frac{1}{n}) = nH_n$.

Pattern: khi đã có i loại, xác suất lấy loại mới $(n-i)/n$, thời gian chờ kỳ vọng $n/(n-i)$.

Nim / Misere Nim

```
ll xr=0;for(ll x:pile)xr^=x;
// normal Nim: xr!=0 -> winning
// misere Nim:
// if all piles are 1: winning iff number of
     piles is even
// otherwise same as normal Nim
```

Stair Game: chỉ XOR các pile ở vị trí lẻ (theo convention từ 1), vì token ở bậc chẵn không ảnh hưởng độc lập như Nim heap.

Grundy - mex nhanh

```
int grundy(int s){
  if(memo[s]!=-1)return memo[s];
  static int seen[MAXG],turn=0;turn++;
  for(int t:nextStates(s)){
    int g=grundy(t);
    if(g<MAXG)seen[g]=turn;
  }
  int g=0;while(g<MAXG&&seen[g]==turn)g++;
  return memo[s]=g;
}
```

Không cần set mỗi state nếu bound Grundy nhỏ: timestamp array nhanh hơn. Tổng game độc lập bằng XOR các Grundy.

XOR Linear Basis

```
ll basis[63];
void insertBasis(ll x){
  for(int b=62;b>=0;b--)if(x>>b&1){
    if(!basis[b]){basis[b]=x;return;}
    x^=basis[b];
  }
}
ll maxXor(ll x=0){
  for(int b=62;b>=0;b--)
    x=max(x,x^basis[b]);
  return x;
}
```

Đếm số XOR khác nhau = 2^{rank}. Membership: reduce x bằng basis, về 0 thì biểu diễn được. Dùng cho subset XOR, cycle xor graph.

FWT XOR convolution

```
void fwtXor(vector<ll>&a,bool inv){
  int n=a.size();
  for(int len=1;2*len<=n;len<<=1)
    for(int i=0;i<n;i+=2*len)
      for(int j=0;j<len;j++){
        ll u=a[i+j],v=a[i+j+len];
        a[i+j]=(u+v)%MOD;
        a[i+j+len]=(u-v+MOD)%MOD;
      }
  if(inv){
    ll ni=binpow(n,MOD-2,MOD);
    for(ll &x:a)x=x*ni%MOD;
  }
}
```

XOR convolution: FWT(A), FWT(B), nhân pointwise, inverse. OR/AND FWT có update khác và inverse bằng trừ.

Möbius inversion - mẫu tư duy

Nếu $F(n) = \sum_{d|n} f(d)$ thì $f(n) = \sum_{d|n} \mu(d) F(n/d)$.

Nếu $F(n) = \sum_{n|d} f(d)$ trên range hữu hạn, có thể đảo bằng sieve từ lớn xuống.

Nhận diện: đề cho "tổng trên mọi ước/bội", cần khôi phục giá trị "gcd đúng bằng d" hoặc "primitive".

Primitive Pythagorean Triples

Mọi bộ nguyên thủy: $a = m^2 - n^2,\ b = 2mn,\ c = m^2 + n^2$, với $m > n$, $\gcd(m,n) = 1$, m, n khác parity.

Bộ không nguyên thủy = nhân cả ba với k.

Pick + lattice polygon

Polygon có đỉnh nguyên: $A = I + B/2 - 1$.

Số lattice points trên segment $(x_1, y_1) \to (x_2, y_2)$ tính cả hai đầu = $\gcd(|dx|, |dy|) + 1$.

Boundary toàn polygon: $B = \sum \gcd(|dx|, |dy|)$ (không +1 từng cạnh vì vertex bị đếm lặp).

Heron / inradius / circumradius

$s = (a+b+c)/2$, $Area = \sqrt{s(s-a)(s-b)(s-c)}$.

$r = Area/s$, $R = abc/(4Area)$.

Luật cos: $c^2 = a^2 + b^2 - 2ab\cos C$.

Luật sin: $a/\sin A = b/\sin B = c/\sin C = 2R$.

Regular polygon / circle

Đa giác đều n cạnh, circumradius R: $Area = \frac{n}{2} R^2 \sin(2\pi/n)$.

Cạnh $s = 2R\sin(\pi/n)$; apothem $r = R\cos(\pi/n)$.

Chord góc tâm θ: $2R\sin(\theta/2)$.

Sector area: $\frac{1}{2} R^2 \theta$ với θ radian.

Arc length: $R\theta$.

Complex numbers cho geometry

```
using C=complex<ld>;
const C I(0,1);
C rotate(C p,ld ang){return p*polar((ld)1,ang);}
ld crossC(C a,C b){return imag(conj(a)*b);}
ld dotC(C a,C b){return real(conj(a)*b);}
```

Ngắn cho rotate/angle/vector. Nhưng với geometry integer chính xác, struct point + __int128 cross vẫn an toàn hơn.

Classic Math Problems - nhận diện nhanh

Exponentiation: binpow. **Exponentiation II**: giảm exponent mod $MOD - 1$.

Counting Divisors: SPF hoặc factor $O(\sqrt{n})$ rồi $\prod(e+1)$.

Common Divisors: frequency + duyệt bội từ divisor lớn xuống.

Sum of Divisors: $\sum d\lfloor n/d \rfloor$ + gom floor.

Divisor Analysis: multiplicative formulas trên prime powers.

Prime Multiples: inclusion-exclusion subset primes.

Counting Coprime Pairs: Möbius inversion.

Binomial Coefficients: factorial/invfact; n rất lớn và mod nhỏ -> Lucas.

Creating Strings: multinomial. **Distributing Apples**: stars and bars.

Christmas Party: derangements. **Bracket Sequences**: Catalan/ballot.

Counting Necklaces/Grids: Burnside.

Fibonacci: fast doubling hoặc matrix. **Throwing Dice**: recurrence + matrix/linear recurrence.

Graph Paths I: adjacency matrix power. **Graph Paths II**: min-plus matrix power.

Probability: indicator variables / expected value DP. **Nim**: XOR / Sprague-Grundy.

Math decision table

Dấu hiệu	Nghĩ đến
$n \le 10^7$, nhiều factor	SPF / linear sieve
n 64-bit, prime?	Miller-Rabin
n 64-bit, cần factor	Pollard-Rho
Điều kiện gcd đúng bằng 1/d	Möbius
Đếm chia hết nhiều prime	Inclusion-Exclusion
$\lfloor n/i \rfloor$	quotient grouping
$\sum \lfloor (ai+b)/m \rfloor$	floor_sum
nCk, mod prime	fact + invfact
n cực lớn, mod prime nhỏ	Lucas
Đối xứng quay/lật	Burnside
Dãy recurrence, index lớn	Matrix/Kitamasa/BM
Convolution	NTT/FFT
Hệ tuyến tính	Gaussian
XOR subset	Linear basis/FWT
Game impartial	Grundy/Nim
Lattice polygon	Pick + gcd edges

Segmented Sieve - primes trên $[L, R]$

```
vector<char> is(R-L+1,1);
for(ll p:primes){
  if(p*p>R)break;
  ll st=max(p*p,((L+p-1)/p)*p);
  for(ll x=st;x<=R;x+=p)is[x-L]=0;
}
if(L==1)is[0]=0;
```

Khi R rất lớn nhưng đoạn $R-L$ nhỏ. Cần primes đến $\sqrt{R}$ bằng sieve thường.

Legendre - số mũ prime trong $n!$

```
ll vpFact(ll n,ll p){
  ll e=0;
  while(n)n/=p,e+=n;
  return e;
}
```

$v_p(n!) = \lfloor n/p \rfloor + \lfloor n/p^2 \rfloor + \cdots$. Trailing zeros cơ số $b = \prod p_i^{e_i}$ là $\min_i \lfloor v_{p_i}(n!)/e_i \rfloor$.

Binomial modulo prime - Lucas full

```
ll smallC(ll n,ll k,ll p){
  if(k<0||k>n)return 0;
  ll num=1,den=1;
  for(ll i=1;i<=k;i++){
    num=num*(n-k+i)%p;
    den=den*i%p;
  }
  return num*binpow(den,p-2,p)%p;
}
ll lucas2(ll n,ll k,ll p){
  if(k==0)return 1;
  return lucas2(n/p,k/p,p)*smallC(n%p,k%p,p)%p;
}
```

Nếu p nhỏ, precompute factorial $0..p-1$ để mỗi digit $O(1)$.

Tonelli-Shanks - modular sqrt

```
// p odd prime, returns x with x^2=a mod p, or
    -1
ll tonelli(ll a,ll p){
  a%=p;if(a==0)return 0;
  if(binpow(a,(p-1)/2,p)!=1)return -1;
  if(p%4==3)return binpow(a,(p+1)/4,p);
  ll q=p-1,s=0;while(!(q&1))q>>=1,s++;
  ll z=2;while(binpow(z,(p-1)/2,p)!=p-1)z++;
  ll c=binpow(z,q,p),x=binpow(a,(q+1)/2,p),t=
     binpow(a,q,p),m=s;
  while(t!=1){
    ll i=1,tt=t*t%p;
    while(tt!=1)tt=tt*tt%p,i++;
    ll b=binpow(c,1LL<<(m-i-1),p);
    x=x*b%p;t=t*b%p*b%p;c=b*b%p;m=i;
  }
  return x;
}
```

Có nghiệm iff Legendre symbol $a^{(p-1)/2} \equiv 1$. Nghiệm còn lại là $p-x$.

Primitive Root - prime

```
ll primitiveRoot(ll p){
  ll phi=p-1,x=phi;
  vector<ll> fac;
  for(ll q=2;q*q<=x;q++)if(x%q==0){
    fac.pb(q);while(x%q==0)x/=q;
  }
  if(x>1)fac.pb(x);
  for(ll g=2;g<p;g++){
    bool ok=1;
    for(ll q:fac)if(binpow(g,phi/q,p)==1){ok=0;
     break;}
    if(ok)return g;
  }
  return -1;
}
```

Dùng cho NTT/custom roots, discrete log/root. Với prime p, kiểm tra các prime divisors của $p-1$.

Diophantine - một nghiệm và bước dịch

```
// ax+by=c
bool dio(ll a,ll b,ll c,ll &x,ll &y,ll &g){
  g=extgcd(abs(a),abs(b),x,y);
  if(c%g)return false;
  x*=c/g;y*=c/g;
  if(a<0)x=-x;if(b<0)y=-y;
  return true;
}
// all solutions:
// x = x0 + k*(b/g)
// y = y0 - k*(a/g)
```

Đếm nghiệm trong rectangle: chuyển bound của x, y thành interval hợp lệ của k bằng floor/ceil division có dấu.

Permutation cycles / order

```
vector<int> vis(n);
ll ord=1;
for(int i=0;i<n;i++)if(!vis[i]){
  int u=i,len=0;
  while(!vis[u])vis[u]=1,u=p[u],len++;
  ord=lcm(ord,(ll)len); // beware overflow / use
      prime exponents
}
```

Order của permutation = LCM độ dài các cycle. Nếu answer modulo, factor từng cycle length và lấy max exponent prime thay vì LCM trực tiếp.

Integer Partitions

```
// number of partitions of n, unordered positive
     parts
vector<ll> dp(n+1);dp[0]=1;
for(int x=1;x<=n;x++)
  for(int s=x;s<=n;s++)
    dp[s]=(dp[s]+dp[s-x])%MOD;
```

Đây chính là coin change combinations với coin $1..n$. Nếu giới hạn số phần/giá trị, thêm chiều DP hoặc đổi range coin.

Stirling số loại 1

```
// unsigned c(n,k): permutations of n with k
     cycles
C1[0][0]=1;
for(int n=1;n<=N;n++)
  for(int k=1;k<=n;k++)
    C1[n][k]=(C1[n-1][k-1]+1LL*(n-1)*C1[n-1][k])
     %MOD;
```

Loại 1 liên quan số cycle trong permutation; loại 2 liên quan partition set. Dễ nhầm hai recurrence.

Necklace Burnside - cả reflection

Với vòng cổ chỉ xét quay: $\frac{1}{n}\sum_{r=0}^{n-1} k^{\gcd(n,r)}$.

Bracelet (quay+lật): chia cho $2n$; reflection fixed count phụ thuộc parity n: n lẻ: mỗi reflection cố định $k^{(n+1)/2}$; n chẵn: $n/2$ reflection cố định $k^{n/2+1}$ và $n/2$ cố định $k^{n/2}$.

Binomial distribution / choose probability

Nếu $X \sim Bin(n,p)$: $P(X=k) = \binom{n}{k} p^k (1-p)^{n-k}$, $E[X] = np$, $Var(X) = np(1-p)$.

Contest thường không cần phân phối đầy đủ; dùng log hoặc long double khi n lớn.

Expected maximum của dice

```
// n independent uniform values in [1..k]
ld E=0;
for(int x=1;x<=k;x++)
  E += 1.0L - pow((ld)(x-1)/k,n);
```

Dùng $E[X] = \sum_{x \ge 1} P(X \ge x)$. Với continuous/unbounded có integral/tail sum tương tự.

Dice sum probability DP

```
vector<ld> dp(6*n+1),ndp(6*n+1);
dp[0]=1;
for(int i=0;i<n;i++){
  fill(all(ndp),0);
  for(int s=0;s<=6*i;s++)if(dp[s])
    for(int d=1;d<=6;d++)ndp[s+d]+=dp[s]/6;
  swap(dp,ndp);
}
```

Nếu chỉ cần count modulo, bỏ chia 6 và đếm số cách. Nếu n lớn, convolution/polynomial exponentiation có thể nhanh hơn.

Modular Gaussian Elimination

```
// MOD prime
int row=0;
for(int col=0;col<m&&row<n;col++){
  int sel=row;
  while(sel<n&&a[sel][col]==0)sel++;
  if(sel==n)continue;
  swap(a[sel],a[row]);
  ll inv=binpow(a[row][col],MOD-2,MOD);
  for(int j=col;j<=m;j++)a[row][j]=a[row][j]*inv
     %MOD;
  for(int i=0;i<n;i++)if(i!=row&&a[i][col]){
    ll c=a[i][col];
    for(int j=col;j<=m;j++)
      a[i][j]=(a[i][j]-c*a[row][j]%MOD+MOD)%MOD;
  }
  where[col]=row++;
}
```

Hệ tuyến tính modulo prime: pivot inverse bằng Fermat. Với MOD composite, pivot không luôn khả nghịch.

Gaussian trên GF(2)

```
int row=0;
for(int col=0;col<m&&row<n;col++){
  int sel=row;
  while(sel<n&&!a[sel][col])sel++;
  if(sel==n)continue;
  swap(a[sel],a[row]);
  for(int i=0;i<n;i++)if(i!=row&&a[i][col])
    a[i]^=a[row];
  row++;
}
```

Mỗi row dùng `bitset`. Dùng cho XOR equations, lights out, linear basis dạng matrix.

SOS / subset zeta transform

```
// f[mask] -> sum over submasks
for(int b=0;b<n;b++)
  for(int mask=0;mask<(1<<n);mask++)
    if(mask>>b&1)
      f[mask]+=f[mask^(1<<b)];
// inverse Möbius:
for(int b=0;b<n;b++)
  for(int mask=0;mask<(1<<n);mask++)
    if(mask>>b&1)
      f[mask]-=f[mask^(1<<b)];
```

$O(n2^n)$. Đếm/query trên mọi submask/supermask nhanh hơn $O(3^n)$ khi phép toán tuyến tính.

FWT OR / AND

```
// OR transform
for(int b=0;b<n;b++)
 for(int mask=0;mask<(1<<n);mask++)if(!(mask>>b
     &1))
   f[mask|(1<<b)] += inv ? -f[mask] : f[mask];

// AND transform
for(int b=0;b<n;b++)
 for(int mask=0;mask<(1<<n);mask++)if(mask>>b&1)
   f[mask^(1<<b)] += inv ? -f[mask] : f[mask];
```

Sau transform: convolution OR/AND trở thành nhân pointwise. Nhớ normalize modulo khi inverse dùng phép trừ.

Berlekamp-Massey - compact

Ngữ cảnh dùng: Có đủ prefix của một dãy tuyến tính modulo prime nhưng không biết recurrence. Sau BM dùng Kitamasa để nhảy tới chỉ số cực lớn.

```
vector<ll> BM(vector<ll>s){
  vector<ll>C(1,1),B(1,1);
  ll L=0,m=1,b=1;
  for(int n=0;n<sz(s);n++){
    ll d=s[n]%MOD;
    for(int i=1;i<=L;i++)d=(d+C[i]*s[n-i])%MOD;
    if(d==0){m++;continue;}
    auto T=C;
    ll coef=d*binpow(b,MOD-2,MOD)%MOD;
    if(sz(C)<sz(B)+m)C.resize(sz(B)+m);
    for(int j=0;j<sz(B);j++)
      C[j+m]=(C[j+m]-coef*B[j]%MOD+MOD)%MOD;
    if(2*L<=n){
      L=n+1-L;B=T;b=d;m=1;
    }else m++;
  }
  C.erase(C.begin());
  for(ll &x:C)x=(MOD-x)%MOD;
  return C;
}
```

Input đủ prefix của recurrence tuyến tính modulo prime. Output coefficients $S_n = \sum C_i S_{n-i-1}$. Sau đó dùng Kitamasa/linear recurrence để nhảy đến index lớn.

Polynomial derivative / integral / evaluation

```
ll eval(vector<ll>&a,ll x){
  ll r=0;
  for(int i=sz(a)-1;i>=0;i--)
    r=(r*x+a[i])%MOD; // Horner
  return r;
}
vector<ll> deriv(vector<ll>a){
  vector<ll>d(max(0,sz(a)-1));
  for(int i=1;i<sz(a);i++)d[i-1]=a[i]*i%MOD;
  return d;
}
```

Horner $O(n)$ evaluation. Formal integral modulo prime dùng $a_i/(i+1)$ với modular inverse.

Các đồng nhất thức kinh điển

$\sum_{i=1}^{n} i = n(n+1)/2$. $\sum i^2 = n(n+1)(2n+1)/6$. $\sum i^3 = (n(n+1)/2)^2$.

$\sum_{i=0}^{n} r^i = (r^{n+1}-1)/(r-1)$ nếu mẫu khả nghịch.

$\sum_{i=0}^{n} ir^i = \frac{r-(n+1)r^{n+1}+nr^{n+2}}{(1-r)^2}$ khi hợp lệ.

$\gcd(a,b)\cdot lcm(a,b) = |ab|$ (cẩn thận overflow).

Hình học kinh điển - định thức 2D

Hai vector a, b: $dot = a_x b_x + a_y b_y$, $cross = a_x b_y - a_y b_x$.

$|cross(b-a, c-a)|/2$ = diện tích tam giác.

Khoảng cách point-line qua a, b: $|cross(b-a, p-a)|/|b-a|$.

Projection scalar $t = dot(p-a, b-a)/|b-a|^2$; clamp t vào $[0,1]$ để lấy closest point trên segment.

Hình học kinh điển - các tâm tam giác

Centroid $G = (A+B+C)/3$.

Incenter $I = (aA+bB+cC)/(a+b+c)$ với $a = |BC|$ đối diện A.

Circumcenter = giao hai trung trực.

Orthocenter $H = A+B+C-2O$ nếu gốc tùy ý và O là circumcenter trong vector coordinates; hoặc giao hai đường cao.

Math contest checklist

- Prime? thử Miller trước khi Pollard. - Nhiều query factor nhỏ? SPF. - GCD condition? thử Möbius / divisor frequency. - "ít nhất một"? Inclusion-Exclusion. - Symmetry? Burnside. - Huge exponent? Euler/Fermat/Carmichael, kiểm tra gcd/base=0. - Huge index recurrence? Matrix/BM/Kitamasa. - Count paths exact k? Matrix. - Min cost exact k? Min-plus. - Set partition? Stirling/Bell. - Valid bracket/BST shape? Catalan. - Assignment of indistinguishable objects? Stars and Bars. - XOR subspace? Linear Basis/GF(2). - Sum over submasks? SOS/FWT. - Lattice polygon? Pick. - Circle/triangle? Heron + sine/cosine laws.

Modular inverse hàng loạt $1..N$

```
// MOD prime, i < MOD
inv[1]=1;
for(int i=2;i<=N;i++)
  inv[i]=MOD-(MOD/i)*inv[MOD%i]%MOD;
```

$O(N)$, tiện cho harmonic/modular combinatorics. Với arbitrary numbers có thể dùng prefix/suffix products để lấy inverse của cả mảng chỉ với một lần binpow.

Inverse cả mảng - 1 lần binpow

```
vector<ll> pref(n+1,1),invA(n),suf(n+1,1);
for(int i=0;i<n;i++)pref[i+1]=pref[i]*a[i]%MOD;
for(int i=n-1;i>=0;i--)suf[i]=suf[i+1]*a[i]%MOD;
ll allInv=binpow(pref[n],MOD-2,MOD);
for(int i=0;i<n;i++)
  invA[i]=pref[i]*suf[i+1]%MOD*allInv%MOD;
```

Yêu cầu mọi a_i khả nghịch modulo. Pattern hay khi cần inverse của rất nhiều denominator.

Number of divisors of $n!$

```
ll ans=1;
for(int p:primes){
  if(p>n)break;
  ll e=vpFact(n,p);
  ans=ans*(e+1)%MOD;
}
```

$n! = \prod p^{v_p(n!)}$, nên $\tau(n!) = \prod (v_p(n!)+1)$.

Sum of gcd pairs bằng divisor frequency

```
// exact[d] = number of pairs with gcd exactly d
for(int d=M;d>=1;d--){
  ll c=0;
  for(int x=d;x<=M;x+=d)c+=freq[x];
  exact[d]=c*(c-1)/2;
  for(int x=2*d;x<=M;x+=d)exact[d]-=exact[x];
}
ll sumG=0;
for(int d=1;d<=M;d++)sumG+=exact[d]*d;
```

Inversion trên lattice "chia hết": đếm gcd là bội của d, rồi trừ gcd lớn hơn từ trên xuống.

Euler Phi sieve + sum phi

```
for(int i=0;i<=N;i++)phi[i]=i;
for(int p=2;p<=N;p++)if(phi[p]==p)
  for(int x=p;x<=N;x+=p)
    phi[x]-=phi[x]/p;
for(int i=1;i<=N;i++)prefPhi[i]=prefPhi[i-1]+phi
    [i];
```

$\sum_{i=1}^{n} \phi(i)$ xuất hiện trong đếm phân số tối giản/-Farey. Số cặp $1 \le a \le b \le n, \gcd(a,b) = 1$ liên quan prefix phi.

Carmichael lambda - giảm exponent chặt hơn

Với $n = \prod p_i^{e_i}$, $\lambda(n) = lcm(\lambda(p_i^{e_i}))$.

Prime lẻ hoặc $p = 2, e \le 2$: $\lambda(p^e) = \phi(p^e)$.

$2^e, e \ge 3$: $\lambda(2^e) = 2^{e-2}$.

Nếu $\gcd(a,n) = 1$, $a^{\lambda(n)} \equiv 1 \pmod n$. Thường nhỏ hơn Euler phi.

Multiplicative order

```
ll orderMod(ll a,ll m){
  if(gcd(a,m)!=1)return -1;
  ll ord=phi(m);
  auto f=factor(ord);
  for(auto [p,e]:f)
    while(ord%p==0 && binpow(a,ord/p,m)==1)
      ord/=p;
  return ord;
}
```

Order = chu kỳ nhỏ nhất của $a^k \bmod m$. Dùng cho chu kỳ số dư, primitive root, repetend thập phân.

Sum of two squares - criterion

n biểu diễn được dưới dạng a^2+b^2 iff trong factorization của n, mọi prime $p \equiv 3 \pmod 4$ có số mũ chẵn.

Đếm representations có công thức từ số divisor $\equiv 1,3 \pmod 4$, nhưng contest thường chỉ cần kiểm tra tồn tại.

Three-square / Four-square theorem

Legendre: n là tổng 3 bình phương iff sau khi bỏ hết factor 4, số còn lại không có dạng $8k+7$.

Lagrange: mọi số nguyên không âm là tổng của 4 bình phương.

Dùng như điều kiện tồn tại; nếu cần dựng nghiệm, có thể enumerate a, b và kiểm tra phần còn lại bằng hash/two-square tùy bound.

Pell equation - classic

$x^2 - Dy^2 = 1$, D không phải square. Nghiệm dương nhỏ nhất lấy từ continued fraction của $\sqrt{D}$; mọi nghiệm sinh bởi $(x_1 + y_1\sqrt{D})^k$.

Rare topic nhưng nên nhận diện khi đề hỏi nghiệm nguyên của hyperbola hoặc recurrence từ $\sqrt{D}$.

Continued Fraction

```
// rational p/q -> continued fraction
vector<ll> cf;
while(q){
  cf.pb(p/q);
  tie(p,q)=make_pair(q,p%q);
}
// convergents:
// h[-2]=0,h[-1]=1; k[-2]=1,k[-1]=0
```

Convergents cho xấp xỉ hữu tỉ tốt nhất. Dùng trong Pell, rational approximation, floor-sum biến đổi.

Prüfer sequence / Cayley

Số cây có nhãn trên n đỉnh: n^{n-2}.

Prüfer sequence dài $n-2$, mỗi sequence trên alphabet $1..n$ tương ứng đúng một labeled tree.

Degree của vertex v = số lần v xuất hiện trong Prüfer + 1.

Matrix Tree Theorem - số spanning trees

```
// Laplacian L: L[i][i]=degree(i), L[i][j]-=#
     edges(i,j)
// delete one row & column -> cofactor matrix M
// answer = det(M)
ll detMod(vector<vector<ll>> a){
  int n=a.size();ll ans=1;
  for(int c=0;c<n;c++){
    int p=c;while(p<n&&a[p][c]==0)p++;
    if(p==n)return 0;
    if(p!=c)swap(a[p],a[c]),ans=(MOD-ans)%MOD;
    ans=ans*a[c][c]%MOD;
    ll inv=binpow(a[c][c],MOD-2,MOD);
    for(int i=c+1;i<n;i++){
      ll f=a[i][c]*inv%MOD;
      for(int j=c;j<n;j++)
        a[i][j]=(a[i][j]-f*a[c][j]%MOD+MOD)%MOD;
    }
  }
  return ans;
}
```

Kirchhoff/Matrix-Tree: đếm spanning trees của graph. Với multigraph, off-diagonal trừ số cạnh. MOD prime để Gaussian inverse đơn giản.

Determinant nguyên - Bareiss idea

Nếu cần determinant chính xác integer và matrix vừa: Bareiss fraction-free elimination tránh floating error và giảm growth tốt hơn Gaussian rational. Với modulo prime thì dùng determinant modular ở trên.

Markov chain - stationary distribution

Phân phối dừng π thỏa $\pi P = \pi$, $\sum \pi_i = 1$.

Finite irreducible + aperiodic -> hội tụ về stationary distribution.

Để tìm π, lập hệ tuyến tính $(P^T - I)\pi = 0$ cộng phương trình tổng = 1 rồi Gaussian.

Absorbing Markov / expected hitting time

Với terminal $E[t] = 0$ và state thường: $E[u] = 1 + \sum_v P(u,v)E[v]$.

Chuyển về thành hệ tuyến tính nếu có cycle. Nếu transition DAG, DP ngược topo đủ.

Probability - birthday bound

Hash space size M, khoảng $\Theta(\sqrt{M})$ random values đã có xác suất collision đáng kể (birthday paradox).

Do đó hash mod 10^9+7 đơn lẻ không đủ chắc cho dữ liệu adversarial; dùng double hash hoặc 64-bit randomized.

Meet-in-the-middle subset count

```
vector<ll>A,B;
int n1=n/2,n2=n-n1;
for(int m=0;m<(1<<n1);m++){
  ll s=0;for(int i=0;i<n1;i++)if(m>>i&1)s+=a[i];
  A.pb(s);
}
for(int m=0;m<(1<<n2);m++){
  ll s=0;for(int i=0;i<n2;i++)if(m>>i&1)s+=a[n1+
      i];
  B.pb(s);
}
sort(all(B));
// count pairs sum == X
ll ans=0;
for(ll s:A)
  ans+=upper_bound(all(B),X-s)-lower_bound(all(B
      ),X-s);
```

$n \approx 40$ classic. Với max sum $\le X$: dùng upper_bound. Với optimization, sort hai nửa rồi two pointers.

Meet-in-the-middle 4SUM

```
vector<ll> ab,cd;
for(ll x:A)for(ll y:B)ab.pb(x+y);
for(ll x:C)for(ll y:D)cd.pb(x+y);
sort(all(cd));
ll cnt=0;
for(ll s:ab)
  cnt+=upper_bound(all(cd),T-s)-lower_bound(all(
      cd),T-s);
```

Biến $O(n^4)$ thành $O(n^2 \log n)$. Pattern classic cho 4SUM/count quadruples.

Fast k-th XOR from linear basis

Sau khi Gaussian-reduce basis theo bit tăng/giảm thành canonical form, mỗi subset coefficient tạo một XOR riêng nếu basis độc lập. Có thể map binary index k sang XOR bằng XOR các basis vectors tương ứng. Cần cẩn thận nếu 0 có nhiều representation do rank < number inputs.

Polynomial convolution use-cases

- Đếm pair sums: coefficient của $A(x)^2$.

- Dice sums: $(x + x^2 + \cdots + x^6)^n$.

- String matching binary alphabet: convolution để đếm mismatch theo shift.

- Large combinatorial DP có transition convolution -> NTT.

Formal Power Series - nhận diện

Nếu bài hỏi coefficients của $1/A(x)$, $\log A$, $\exp A$, sqrt polynomial hoặc recurrence convolution lớn, đây là FPS territory. KACTL/advanced libraries có Newton iteration; chỉ nên dùng khi team đã test template trước.

Bẫy modulo thường gặp

- Không được dùng $(a/b)\%MOD$; phải nhân modular inverse khi b khả nghịch.

- Giảm exponent mod $\phi(m)$ chỉ an toàn trực tiếp khi gcd(base,m)=1; nếu không, cần xử lý exponent đủ lớn/prime powers.

- 0^0 trong contest thường cần đọc định nghĩa đề; binpow code thường trả 1.

- MOD composite: Fermat inverse không dùng được; exgcd nếu gcd(a,MOD)=1.

- Nhân 64-bit modulo gần 10^{18}: dùng __int128.

Bẫy đếm thường gặp

- Vật phân biệt / hộp phân biệt khác hoàn toàn stars-and-bars.
- Circular arrangements: thường chia rotation nhưng phải dùng Burnside nếu có symmetry/fixed points.
- "At least one" -> PIE hoặc complement.
- "Exactly gcd=d" -> scale by d rồi Möbius/inversion.
- Prefix constraint (bracket/path không âm) -> reflection principle/Catalan, không chỉ C(n,k).

Geometry math - circle from 3 points

```
bool circumcenter(PD A,PD B,PD C,PD &O){
  PD m1={(A.ft+B.ft)/2,(A.sd+B.sd)/2};
  PD m2={(A.ft+C.ft)/2,(A.sd+C.sd)/2};
  PD d1={-(B.sd-A.sd),B.ft-A.ft};
  PD d2={-(C.sd-A.sd),C.ft-A.ft};
  return lineInter(m1,add(m1,d1),m2,add(m2,d2),O
      );
}
```

Ba điểm thẳng hàng -> không có circumcircle hữu hạn. Radius = dist(O,A).

Tangents từ point đến circle

```
vector<PD> tangency(PD p,PD c,ld r){
  PD v=subD(p,c);ld d2=dotD(v,v);
  if(d2<r*r-EPS)return {};
  ld d=sqrt(d2),base=atan2(v.sd,v.ft);
  if(fabsl(d-r)<EPS)return {p};
  ld alpha=acos(r/d);
  return {
    {c.ft+r*cos(base+alpha),c.sd+r*sin(base+
     alpha)},
    {c.ft+r*cos(base-alpha),c.sd+r*sin(base-
     alpha)}
  };
}
```

Point trong circle: 0 tangent; trên circle: 1; ngoài: 2.

Polygon centroid

```
PD centroid(vector<PD>&p){
  ld A=0,cx=0,cy=0;
  for(int i=0;i<sz(p);i++){
    PD a=p[i],b=p[(i+1)%sz(p)];
    ld cr=a.ft*b.sd-a.sd*b.ft;
    A+=cr;cx+=(a.ft+b.ft)*cr;cy+=(a.sd+b.sd)*cr;
  }
  A*=0.5;
  return {cx/(6*A),cy/(6*A)};
}
```

Polygon simple có mật độ đều. Orientation CW cho signed area âm nhưng công thức vẫn nhất quán nếu không lấy abs trước.

Minimum Enclosing Circle - randomized

Ngữ cảnh dùng: Cần circle nhỏ nhất phủ tất cả điểm. Randomized incremental có expected $O(N)$; khác bài phủ ít nhất K điểm ở cuối tài liệu.

```
struct Cir{PD c;ld r;};
bool inside(Cir C,PD p){return dist(C.c,p)<=C.r+
     EPS;}
Cir mec(vector<PD> p){
  shuffle(all(p),rng);
  Cir C{{0,0},-1};
  for(int i=0;i<sz(p);i++)if(C.r<0||!inside(C,p[
     i])){
    C={p[i],0};
    for(int j=0;j<i;j++)if(!inside(C,p[j])){
      C={{(p[i].ft+p[j].ft)/2,(p[i].sd+p[j].sd)
     /2},dist(p[i],p[j])/2};
      for(int k=0;k<j;k++)if(!inside(C,p[k])){
        PD o;circumcenter(p[i],p[j],p[k],o);
        C={o,dist(o,p[i])};
      }
    }
  }
  return C;
}
```

Expected $O(N)$ randomized Welzl-style incremental. Collinear triples cần circumcenter failure; trong triple loop chỉ gọi khi ba điểm không collinear.

Geometry math decision table

2 points + radius -> circle centers. 3 points -> circumcircle. 2 circles -> intersection/tangents. Point to line/segment -> projection/clamp.

Many points, boundary -> convex hull. Farthest convex pair -> rotating calipers. Closest pair -> divide-conquer. Enclosing all points -> MEC. Linear inequalities -> polygon clipping/HPI. Lattice polygon -> Pick.

Big Integer C++ - boost

```
#include <boost/multiprecision/cpp_int.hpp>
using boost::multiprecision::cpp_int;
cpp_int a=1;
for(int i=1;i<=n;i++)a*=i;
cout<<a;
```

Dùng khi đề cần giá trị chính xác vượt 128-bit: factorial lớn, Fibonacci lớn, combinatorics không modulo. Chậm hơn integer thường nên chỉ dùng khi cần.

Integer sqrt an toàn

```
ll isqrt(ll n){
  ll x=sqrtl((ld)n);
  while((int128)(x+1)*(x+1)<=n)x++;
  while((int128)x*x>n)x--;
  return x;
}
bool isSquare(ll n){
  if(n<0)return false;
  ll x=isqrt(n);
  return (int128)x*x==n;
}
```

Không tin hoàn toàn sqrt(double) gần 10^{18}. Lấy ước lượng rồi chỉnh bằng __int128.

Integer k-th root

```
bool lePow(ll a,int k,ll n){
  int128 p=1;
  for(int i=0;i<k;i++){
    p*=a;
    if(p>n)return false;
  }
  return true;
}
ll kthRoot(ll n,int k){
  ll l=0,r=min(n,1000000000LL)+1;
  while(l+1<r){
    ll m=(l+r)/2;
    if(lePow(m,k,n))l=m;else r=m;
  }
  return l;
}
```

Classic cho perfect power / root floor. Chọn upper bound theo k nếu muốn chặt hơn.

Garner - CRT pairwise coprime

```
// x = r[i] mod m[i], pairwise coprime
ll garner(vector<ll> r,vector<ll> m,ll MODOUT){
  int n=r.size();
  vector<ll> coef(n+1,1),con(n+1,0);
  m.pb(MODOUT);
  for(int i=0;i<n;i++){
    ll t=(r[i]-con[i]+m[i])%m[i];
    t=t*modInverse(coef[i]%m[i],m[i])%m[i];
    for(int j=i+1;j<=n;j++){
      con[j]=(con[j]+coef[j]*t)%m[j];
      coef[j]=coef[j]*m[i]%m[j];
    }
  }
  return con[n];
}
```

Hữu ích khi ghép kết quả từ nhiều NTT primes về modulo khác hoặc integer range. Pairwise coprime moduli.

Multiplicative functions - tau/sigma sieve

```
vector<int> tau(N+1);
vector<ll> sigma(N+1);
for(int d=1;d<=N;d++)
  for(int x=d;x<=N;x+=d){
    tau[x]++;
    sigma[x]+=d;
  }
```

$O(N \log N)$, cực đơn giản khi $N \le 10^6$–10^7 tùy time. Nếu cần nhanh hơn/multiple functions cùng lúc, linear sieve.

Dirichlet convolution - nhận diện

$(f * g)(n) = \sum_{d|n} f(d)g(n/d)$.

$\mathbf{1} * \mu = \varepsilon$; $\mathbf{1} * id = \sigma$; $\phi * \mathbf{1} = id$.

Möbius inversion chính là nhân convolution với μ. Đây là ngôn ngữ gọn để nhận ra nhiều identity divisor-sum.

Tổng số ước từ 1 đến n

```
// sum_{i=1}^n tau(i) = sum_{d=1}^n floor(n/d)
ll S=0;
for(ll l=1,r;l<=n;l=r+1){
  ll q=n/l;r=n/q;
  S+=q*(r-l+1);
}
```

Đếm cặp (d, k) với $dk \le n$. Cùng kỹ thuật quotient grouping, $O(\sqrt{n})$.

Tổng sigma từ 1 đến n

```
// sum_{i=1}^n sigma(i)=sum_{d=1}^n d*floor(n/d)
ll S=0;
for(ll l=1,r;l<=n;l=r+1){
  ll q=n/l;r=n/q;
  ll sumLR=(l+r)*(r-l+1)/2;
  S += q*sumLR;
}
```

Nếu modulo, tính sum arithmetic progression modulo và dùng __int128 nếu không modulo.

Kitamasa - linear recurrence full

```
// rec: S[n]=sum rec[i]*S[n-1-i]
vector<ll> combine(vector<ll>a,vector<ll>b,
     vector<ll>&rec){
  int k=rec.size();
  vector<ll> t(2*k);
  for(int i=0;i<k;i++)for(int j=0;j<k;j++)
    t[i+j]=(t[i+j]+a[i]*b[j])%MOD;
  for(int i=2*k-2;i>=k;i--)
    for(int j=1;j<=k;j++)
      t[i-j]=(t[i-j]+t[i]*rec[j-1])%MOD;
  t.resize(k);return t;
}
ll linearRec(vector<ll> init,vector<ll> rec,long
     long n){
  int k=rec.size();
  if(n<k)return init[n];
  vector<ll> pol(k),e(k);pol[0]=1;
  if(k==1)e[0]=rec[0];else e[1]=1;
  while(n){
    if(n&1)pol=combine(pol,e,rec);
    e=combine(e,e,rec);n>>=1;
  }
  ll ans=0;
  for(int i=0;i<k;i++)ans=(ans+pol[i]*init[i])%
     MOD;
  return ans;
}
```

$O(k^2 \log n)$, tốt hơn matrix $O(k^3 \log n)$. Ghép trực tiếp với Berlekamp-Massey.

Cayley-Hamilton - ý tưởng

Mọi matrix vuông thỏa characteristic polynomial của chính nó. Vì vậy A^n có thể giảm về tổ hợp của $I, A, \ldots, A^{k-1}$, cùng tinh thần Kitamasa. Đây là nền tảng của nhiều linear recurrence optimizations.

Wilson / Fermat / Euler - 1 dòng

Fermat prime p: $a^{p-1} \equiv 1 \pmod p$ nếu $p \nmid a$.

Euler: $a^{\phi(m)} \equiv 1 \pmod m$ nếu $\gcd(a, m) = 1$.

Wilson: $(p-1)! \equiv -1 \pmod p$ iff p prime. Chủ yếu dùng nhận diện/chứng minh, hiếm khi tính trực tiếp.

Wythoff Nim

Hai pile (a, b), mỗi lượt lấy tùy ý từ một pile hoặc cùng số lượng từ cả hai. Losing positions: $(\lfloor k\varphi \rfloor, \lfloor k\varphi^2 \rfloor)$, $k \ge 0$, $\varphi = (1+\sqrt{5})/2$.

Rare nhưng là pattern game toán kinh điển.

Subtraction Game - Grundy periodicity

```
vector<int> g(N+1);
for(int x=1;x<=N;x++){
  bool seen[64]={};
  for(int d:moves)if(x>=d)seen[g[x-d]]=1;
  while(seen[g[x]])g[x]++;
}
```

Với finite move set, win/lose hoặc Grundy thường eventually periodic; nếu N cực lớn, thử detect period trên prefix đã tính.

Probability - prefix optimize dice DP

```
// dice faces 1..K, count ways for n dice
auto getPref=[&](int x)->ll{
  return x<0?0:pref[x];
};
for(int i=1;i<=n;i++){
  pref[0]=dp[0];
  for(int s=1;s<=S;s++)pref[s]=(pref[s-1]+dp[s])
     %MOD;
  for(int s=0;s<=S;s++){
    ndp[s]=getPref(s-1)-getPref(s-K-1);
    ndp[s]=(ndp[s]%MOD+MOD)%MOD;
  }
  swap(dp,ndp);
}
```

Transition range sum -> prefix sum giảm $O(NKS)$ xuống $O(NS)$. Đây là pattern DP optimization rất hay.

Combinatorics mini-table

Đối tượng	Công thức / pattern
Chọn k từ n	$C(n,k)$
Sắp k từ n	$P(n,k)$
Multiset permutation	$n!/\prod f_i!$
Boxes, xi>=0	$C(n+k-1,k-1)$
Derangement	$(n-1)(D_{n-1}+D_{n-2})$
Valid parentheses	Catalan
Set into k groups	Stirling 2
Permutation k cycles	Stirling 1
Labeled trees	n^{n-2}
Onto functions	$k!S(n,k)$ / PIE
Symmetry classes	Burnside

3D vector basics

```
struct P3{ld x,y,z;};
P3 sub(P3 a,P3 b){return {a.x-b.x,a.y-b.y,a.z-b.
    z};}
ld dot(P3 a,P3 b){return a.x*b.x+a.y*b.y+a.z*b.z
    ;}
P3 cross(P3 a,P3 b){
  return {a.y*b.z-a.z*b.y,
          a.z*b.x-a.x*b.z,
          a.x*b.y-a.y*b.x};
}
ld norm(P3 a){return sqrt(dot(a,a));}
```

Distance point-plane $|n\cdot(p-p_0)|/|n|$. Volume tetrahedron $=|a\cdot(b\times c)|/6$. 3D geometry hiếm nhưng dễ ăn điểm nếu công thức sẵn.

Spherical / great-circle

Lat/lon đổi sang radian. Central angle: $\theta = \arccos(\sin\phi_1\sin\phi_2+\cos\phi_1\cos\phi_2\cos(\Delta\lambda))$.

Distance trên sphere $= R\theta$. Clamp cosine vào $[-1,1]$ trước acos.

Mẫu chứng minh kinh điển

- Pigeonhole: $n+1$ vật vào n hộp -> có hộp ít nhất 2.
- Extremal: chọn phần tử nhỏ/lớn nhất rồi suy ra constraint.
- Invariant/monovariant: parity, gcd, sum modulo, inversion count.
- Double counting: đếm cùng tập cặp theo hai cách.
- Complement counting: "ít nhất một" = tất cả - không có.
- Symmetry: quotient bằng Burnside, không chia mù cho số phép đối xứng nếu có fixed points.

Geometric series modulo - không cần inverse

```
// returns {a^n, 1+a+...+a^(n-1)}
pair<ll,ll> geom(ll a,ll n,ll mod){
  if(n==0)return {1,0};
  auto [p,s]=geom(a,n/2,mod);
  ll p2=(__int128)p*p%mod;
  ll s2=(s+(__int128)p*s)%mod;
  if(n%2==0)return {p2,s2};
  return {(__int128)p2*a%mod,(s2+p2)%mod};
}
```

Tính tổng cấp số nhân $O(\log n)$ ngay cả khi $a-1$ không có inverse modulo composite.

Sum of powers $1^k+\cdots+n^k$

Với k nhỏ, đây là polynomial theo n bậc $k+1$. Tính $k+2$ giá trị đầu rồi Lagrange để query n cực lớn modulo prime.

Pattern rất hay: "giá trị cần tính theo n là polynomial bậc nhỏ" -> nội suy thay vì derive Faulhaber.

Kummer theorem - valuation của C(n,k)

$v_p\binom{n}{k}$ bằng số carry khi cộng k và $n-k$ trong base p.

Tương đương $v_p(n!)-v_p(k!)-v_p((n-k)!)$. Hữu ích để kiểm tra divisibility / số trailing zeros của binomial.

Upper-bound Stars and Bars + PIE

```
// xi>=0, sum=n, xi<=U (same U)
ll ans=0;
for(int j=0;j<=k;j++){
  ll rem=n-1LL*j*(U+1);
  if(rem<0)break;
  ll term=C(k,j)*C(rem+k-1,k-1)%MOD;
  if(j&1)ans=(ans-term+MOD)%MOD;
  else ans=(ans+term)%MOD;
}
```

Nếu upper bound khác nhau theo biến, subset PIE tổng quát; chỉ chạy được khi số biến nhỏ.

Catalan prefix completion

Prefix bracket hiện có balance b, còn r vị trí. Cần final balance 0 và không bao giờ âm.

Nếu số '(' cần thêm là $x=(r-b)/2$ hợp lệ, số completion có thể tính bằng reflection/ballot: $C(r,x)-C(r,x-1)$ sau khi quy đổi đúng orientation. Luôn kiểm tra prefix ban đầu chưa từng âm.

Vandermonde / Hockey-stick

Vandermonde: $\sum_k C(r,k)C(s,n-k)=C(r+s,n)$.

Hockey-stick: $\sum_{i=r}^{n}C(i,r)=C(n+1,r+1)$.

Nhận diện giúp biến DP/combinatorial sum $O(n)$ thành $O(1)$.

Binomial inversion

Nếu $g(n)=\sum_{k=0}^{n}C(n,k)f(k)$ thì $f(n)=\sum_{k=0}^{n}(-1)^{n-k}C(n,k)g(k)$.

Rare nhưng xuất hiện khi subset counting có binomial transform.

Matrix inverse modulo prime

```
// augment A with identity, Gaussian elimination
for(int i=0;i<n;i++)
  for(int j=0;j<n;j++)aug[i][j+n]=(i==j);
for(int col=0,row=0;col<n;col++,row++){
  int p=row;while(p<n&&aug[p][col]==0)p++;
  if(p==n){/* singular */break;}
  swap(aug[p],aug[row]);
  ll iv=binpow(aug[row][col],MOD-2,MOD);
  for(int j=0;j<2*n;j++)aug[row][j]=aug[row][j]*
      iv%MOD;
  for(int i=0;i<n;i++)if(i!=row&&aug[i][col]){
    ll c=aug[i][col];
    for(int j=0;j<2*n;j++)
      aug[i][j]=(aug[i][j]-c*aug[row][j]%MOD+MOD
      )%MOD;
  }
}
```

Right half sau elimination là A^{-1}. Singular nếu thiếu pivot.

Binary exponentiation cho matrix/vector

Nếu chỉ cần $A^k v$, vẫn có thể binary exponentiate matrix và mỗi bit nhân vector/matrix; nhưng với recurrence bậc d, Kitamasa thường tiết kiệm hơn d^3.

Root finding - bisection

```
ld lo=L,hi=R;
for(int it=0;it<100;it++){
  ld mid=(lo+hi)/2;
  if(f(mid)<=0)lo=mid;
  else hi=mid;
}
ld root=(lo+hi)/2;
```

Cần monotone hoặc bracket sign/invariant đúng. 80-100 iterations đủ cho long double trong hầu hết contest.

Ternary Search - integer

```
ll ternaryInt(ll l,ll r){
  while(r-l>5){
    ll m1=l+(r-l)/3;
    ll m2=r-(r-l)/3;
    if(f(m1)<=f(m2))r=m2-1;
    else l=m1+1;
  }
  ll best=l;
  for(ll x=l+1;x<=r;x++)if(f(x)<f(best))best=x;
  return best;
}
```

Chỉ cho unimodal/convex discrete function. Nếu answer có predicate monotone, binary search tốt hơn.

Golden Section - real unimodal

```
ld golden(ld l,ld r){
  const ld gr=(sqrtl(5)-1)/2;
  ld x1=r-gr*(r-l),x2=l+gr*(r-l);
  ld f1=f(x1),f2=f(x2);
  for(int it=0;it<120;it++){
    if(f1<f2){
      r=x2;x2=x1;f2=f1;
      x1=r-gr*(r-l);f1=f(x1);
    }else{
      l=x1;x1=x2;f1=f2;
      x2=l+gr*(r-l);f2=f(x2);
    }
  }
  return (l+r)/2;
}
```

Ít gọi hàm hơn ternary real vì tái sử dụng một điểm. Minimize; đảo so sánh để maximize.

Simpson integration

```
ld simp(ld l,ld r){
  ld m=(l+r)/2;
  return (f(l)+4*f(m)+f(r))*(r-l)/6;
}
ld adapt(ld l,ld r,ld whole,ld eps){
  ld m=(l+r)/2,L=simp(l,m),R=simp(m,r);
  if(fabsl(L+R-whole)<=15*eps)
    return L+R+(L+R-whole)/15;
  return adapt(l,m,L,eps/2)+adapt(m,r,R,eps/2);
}
```

Adaptive Simpson cho tích phân smooth khi không có công thức đóng. Cẩn thận singularity/discontinuity.

Newton-Raphson

```
ld x=x0;
for(int it=0;it<50;it++){
  ld dx=f(x)/df(x);
  x-=dx;
  if(fabsl(dx)<1e-15)break;
}
```

Hội tụ rất nhanh gần nghiệm nhưng không robust như bisection. Nếu derivative nhỏ hoặc initial guess xấu có thể diverge; hybrid Newton+bisection an toàn hơn.

Complex roots of unity

$\omega_n=e^{2\pi i/n}$, $\omega_n^n=1$, tổng $1+\omega+\cdots+\omega^{n-1}=0$ nếu $\omega\neq 1$.

Nền tảng FFT/DFT và các bài regular polygon trên complex plane.

Roots of polynomial degree 2

```
ld D=b*b-4*a*c;
if(D<-EPS){/* no real roots */}
else{
  D=max((ld)0,D);
  ld q=-0.5L*(b+(b>=0?1:-1)*sqrtl(D));
  ld x1=q/a;
  ld x2=(fabsl(q)<EPS?x1:c/q);
}
```

Dạng ổn định số hơn công thức trực tiếp khi b lớn gây cancellation.

Modular arithmetic - checklist

Normalize sau phép trừ. Inverse tồn tại iff gcd(a,m)=1. Prime mod -> Fermat. Composite -> exgcd/CRT. Exponent reduction -> kiểm tra gcd. Multiplication gần 10^{18} -> __int128. Division by 2 modulo odd prime -> nhân $(MOD+1)/2$.

Combinatorial symmetry - khi nào Burnside

Nếu mỗi orbit có cùng size thì có thể chia trực tiếp. Nhưng khi có cấu hình đối xứng đặc biệt, orbit size nhỏ hơn -> phải Burnside/Polya. Dấu hiệu: vòng cổ, tô màu polygon/grid, rotations/reflections.

Parity / modulo invariants

Operation thay đổi đại lượng theo một residue cố định -> xét modulo gcd của các thay đổi.

Nếu mỗi move đổi parity của inversion/sum/popcount có quy luật -> trạng thái đích phải cùng invariant class.

Trước BFS/DP lớn, thử tìm invariant để loại impossible cases.

GCD invariant

Các phép dạng $a_i\leftarrow a_i\pm a_j$ giữ gcd toàn bộ dãy.

Linear combinations của dãy tạo đúng các bội của gcd. Bezout biến "có biểu diễn được không" thành gcd divisibility.

Classic geometry trig traps

C++ sin/cos/atan2 dùng radian. Angle giữa vectors ổn định: `atan2(cross,dot)`. Trước acos: clamp $[-1,1]$. So sánh squared distances nếu chỉ cần thứ tự. Không dùng slope khi có vertical line.

Math formula bank - số học

$\gcd(a,b)=\gcd(b,a \bmod b)$. $lcm(a,b)=a/\gcd(a,b)\cdot b$.

Bezout: $\exists x,y: ax+by=\gcd(a,b)$.

Số nghiệm $ax\equiv b \pmod m$: 0 nếu $g\nmid b$, ngược lại có $g=\gcd(a,m)$ nghiệm modulo m.

$\phi(p^k)=p^k-p^{k-1}$. $\phi(ab)=\phi(a)\phi(b)$ nếu gcd=1.

$\sum_{d|n}\phi(d)=n$. $\sum_{d|n}\mu(d)=[n=1]$.

$\tau(p^a)=a+1$. $\sigma(p^a)=\frac{p^{a+1}-1}{p-1}$.

$v_p(n!)=\sum_{j\geq 1}\lfloor n/p^j\rfloor$.

Nếu $n=\prod p_i^{a_i}$: $\tau(n)=\prod(a_i+1)$, $\sigma(n)=\prod\sigma(p_i^{a_i})$.

Math formula bank - tổ hợp

$C(n,k)=C(n,n-k)$. $C(n,k)=C(n-1,k)+C(n-1,k-1)$.

$kC(n,k)=nC(n-1,k-1)$.

$\sum_k C(n,k)=2^n$. $\sum_k kC(n,k)=n2^{n-1}$.

Vandermonde: $\sum_k C(r,k)C(s,n-k)=C(r+s,n)$.

Hockey-stick: $\sum_{i=r}^{n}C(i,r)=C(n+1,r+1)$.

Catalan: $Cat_n=\frac{1}{n+1}C(2n,n)$.

Derangement: $D_n=(n-1)(D_{n-1}+D_{n-2})$.

Stars-bars $x_i\geq 0$: $C(n+k-1,k-1)$.

Cayley labeled trees: n^{n-2}.

Math formula bank - tổng

$\sum_{i=1}^{n}i=\frac{n(n+1)}{2}$.

$\sum i^2=\frac{n(n+1)(2n+1)}{6}$.

$\sum i^3=(\frac{n(n+1)}{2})^2$.

$\sum_{i=0}^{n}r^i=(r^{n+1}-1)/(r-1)$.

$\sum_{i=1}^{n}\lfloor n/i\rfloor=\sum_{d=1}^{n}\tau(d)$.

$\sum_{d=1}^{n}d\lfloor n/d\rfloor=\sum_{i=1}^{n}\sigma(i)$.

Harmonic $H_n=\ln n+\gamma+O(1/n)$.

Math formula bank - xác suất

$E[X]=\sum_x xP(X=x)$. $E[X]=\sum_{k\geq 1}P(X\geq k)$ cho integer nonnegative.

Linearity: $E[\sum X_i]=\sum E[X_i]$.

$Var(X)=E[X^2]-E[X]^2$.

Independent: $Var(X+Y)=Var(X)+Var(Y)$.

Binomial: $E=np$, $Var=np(1-p)$.

Geometric waiting time success prob p: $E=1/p$.

Coupon collector: nH_n.

Math formula bank - geometry

Triangle area: $|cross(B-A,C-A)|/2$.

Heron: $\sqrt{s(s-a)(s-b)(s-c)}$.
Inradius $r = A/s$; circumradius $R = abc/(4A)$.
Shoelace: $2A = |\sum cross(P_i, P_{i+1})|$.
Pick: $A = I + B/2 - 1$.
Circle chord $= 2R\sin(\theta/2)$; arc $= R\theta$; sector $= \frac{1}{2}R^2\theta$.
Regular n-gon area $= \frac{n}{2}R^2\sin(2\pi/n)$.
Point-line distance $= |cross(b-a, p-a)|/|b-a|$.

Digits / logarithm tricks

```
// digits of positive n in base 10
int digits=(int)floor(log10((ld)n))+1;
// digits of n!
ld lg=0;
for(int i=2;i<=n;i++)lg+=log10((ld)i);
int digitsFact=(int)floor(lg)+1;
// leading k digits of huge X with log10(X)=y
ld frac=y-floor(y);
ll lead=powl(10,frac+k-1);
```

Với factorial rất lớn, dùng $\log\Gamma(n+1)$ / lgammal để lấy log nhanh.

Base conversion / digit sum

```
vector<int> digitsBase(ll x,int b){
  if(x==0)return {0};
  vector<int>d;
  while(x)d.pb(x%b),x/=b;
  reverse(all(d));return d;
}
```

$n \equiv$ digit sum $\pmod{b-1}$. Alternating digit sum cho base 10 liên quan mod 11.

Perfect power detection

```
bool eqPow(ll a,int k,ll n){
  int128 p=1;
  for(int i=0;i<k;i++){
    p*=a;
    if(p>n)return false;
  }
  return p==n;
}
bool perfectPower(ll n){
  if(n<=1)return true;
  for(int k=2;k<=60;k++){
    ll r=kthRoot(n,k);
    for(ll x=max(2LL,r-1);x<=r+1;x++)
      if(eqPow(x,k,n))return true;
  }
  return false;
}
```

K tối đa khoảng 60 cho số số nguyên có dấu 64-bit. Helper eqPow dùng __int128 để tránh sai số của pow.

Mod exponent tower - tư duy

Để tính a^{b^c} mod m composite: factor m thành prime powers, giải từng modulus rồi CRT.

Nếu gcd(a,m)=1 có thể giảm exponent modulo $\lambda(m)$/$\phi(m)$. Nếu không coprime, cần theo dõi exponent có "đủ lớn" để xử lý p-adic exponent; không giảm mù.

Counting multiples / harmonic loops

```
for(int d=1;d<=N;d++)
  for(int x=d;x<=N;x+=d){
    // process d | x
  }
// total iterations ~ N log N
```

Sieve-style loops là pattern nền của divisor frequency, Möbius, phi, tau, sigma, gcd pair counting.

Prime factor contribution

Khi answer multiplicative theo prime factors, đừng enumerate divisors nếu không cần. Factor $n = \prod p^e$ rồi xử lý contribution từng prime power.

Dấu hiệu: divisor count/sum/product, phi, Möbius, multiplicative order, gcd/lcm constraints.

LCM/GCD pair relation

$\gcd(a,b) \cdot lcm(a,b) = ab$.

Nếu biết gcd g, đặt $a = gx, b = gy, \gcd(x,y) = 1$. Rất nhiều bài "tìm pair với gcd/lcm cho trước" giảm về factor phần L/g và chia prime powers thành hai phía.

Count solutions $xy = n$

Số ordered positive pairs (x,y) với $xy = n$ là $\tau(n)$.

Số unordered pairs là $\lceil \tau(n)/2 \rceil$; nếu n square, cặp $\sqrt{n}, \sqrt{n}$ đứng riêng.

Count rectangles / gcd lattice

Primitive direction vector (dx, dy) iff $\gcd(|dx|, |dy|) = 1$.

Số lattice points giữa hai điểm nguyên (không tính endpoints) = $\gcd(|dx|, |dy|) - 1$.

Modular fraction

```
ll fracMod(ll a,ll b,ll mod){
  ll x,y;ll g=extgcd(b,mod,x,y);
  if(g!=1)throw; // no unique inverse
  x=(x%mod+mod)%mod;
  return (__int128)(a%mod+mod)%mod*x%mod;
}
```

Nếu denominator không khả nghịch, "phân số modulo" không có nghĩa duy nhất; đôi khi phải giải congruence thay vì chia.

Randomized number theory

Pollard-Rho phụ thuộc random polynomial; nếu loop d==n thì restart seed/c.

Miller-Rabin 64-bit dùng deterministic bases, không cần random.

Hash/treap có thể randomize để tránh adversarial cases; seed từ steady_clock.

ICPC classic math map - level 1

Dạng	Template
$a^b \bmod m$	binpow
gcd/lcm	Euclid
prime/factor nhỏ	sieve/SPF
nCk mod prime	fact/invfact
Fibonacci n lớn	fast doubling
linear recurrence	matrix/Kitamasa
subset sum n 40	MITM
valid parentheses	Catalan/ballot

ICPC classic math map - level 2

Dạng	Template
64-bit prime/factor	Miller + Pollard
gcd exactly d	Möbius/inversion
many divisor queries	sieve multiples
symmetry coloring	Burnside
huge nCk, small prime	Lucas
discrete logarithm	BSGS
sqrt modulo prime	Tonelli
convolution	NTT

ICPC classic math map - level 3

Dạng	Template
unknown recurrence	BM + Kitamasa
linear system	Gaussian
count spanning trees	Matrix Tree
XOR vector space	Linear Basis
submask transforms	SOS/FWT
affine floor sums	floor_sum
polynomial values	Lagrange
Pell/rational approx	continued fraction

Các trường hợp biên trong toán học

$n = 0, 1$ trong prime/factor. Negative modulo. Duplicate primes in PIE. Overflow khi lcm/product. $n \geq MOD$ với factorial mod prime. Exponent = 0. Base divisible by MOD. Circle/triangle degenerate. Gaussian singular/infinite solutions. Burnside fixed points phụ thuộc parity.

Theorem bank - number theory

Bezout: $ax + by$ nhận đúng các bội của $\gcd(a,b)$.
Fermat: prime p, $a^p \equiv a \pmod p$.
Euler: $\gcd(a,m) = 1 \Rightarrow a^{\phi(m)} \equiv 1$.
CRT: hệ modulo đôi một coprime có nghiệm duy nhất modulo tích; general CRT cần residues tương thích modulo gcd.
Wilson: p prime iff $(p-1)! \equiv -1 \pmod p$.
Lucas: $C(n,k)$ mod p tách theo digits base p.
Kummer: $v_p(C(n,k))$ = số carry khi $k + (n-k)$ base p.
Möbius inversion: đảo tổng trên divisor bằng μ.
Euler product: multiplicative function thường xử lý độc lập trên prime powers.
Chinese remainder + prime powers: bài modulo composite thường factor modulus trước rồi ghép lại.

Theorem bank - combinatorics

Pigeonhole: hơn n vật vào n hộp -> có hộp >=2.
Inclusion-Exclusion: $|\cup A_i| = \sum |A_i| - \sum |A_i \cap A_j| + \cdots$.
Burnside: orbits $= \frac{1}{|G|}\sum_{g\in G} |Fix(g)|$.
Cayley: labeled trees n^{n-2}.
Catalan: Dyck paths/BST shapes/noncrossing matchings.
Stirling-2: partition set into unlabeled nonempty groups.
Stirling-1: permutation by number of cycles.
Reflection principle: prefix/path constraints -> all paths minus reflected bad paths.
Linearity of expectation: sum indicators even without independence.

Useful asymptotic thresholds

Cỡ n	Nghĩ đến
$n \leq 20$	2^n, bitmask DP
$n \leq 25$	$2^{n/2}$ MITM rất nhẹ
$n \leq 40$	MITM $2^{n/2}$
$n \leq 200$	$O(n^3)$ thường ổn
$n \leq 2000$	$O(n^2)$ tùy TL
$n \leq 2e5$	$O(n \log n)$
$value \leq 1e6$	sieve/divisor loops
64-bit integer	Miller/Pollard nếu factor

Math debugging checklist

- Test $0, 1$, prime, square, prime power, product of two primes.
- Test gcd not 1 before modular inverse.
- Test exponent 0 and base 0.
- Test MOD divides numerator/denominator.
- Test overflow in $a * b$, lcm, cross product.
- Test negative residues after subtraction.
- Test floating degeneracy: collinear/tangent/coincident.
- For PIE, remove duplicate prime factors or understand duplicated sets.
- For Burnside, enumerate group actions carefully; reflection parity often differs.
- For recurrence, verify first $2k$ terms before trusting BM/Kitamasa.

Các phép dựng kinh điển

Primitive Pythagorean: $(m^2 - n^2, 2mn, m^2 + n^2)$.
Gray code: $g = i \oplus (i >> 1)$.
De Bruijn: Euler cycle trên states length $k-1$.
Prüfer: sequence length $n-2$ <-> labeled tree.
Regular polygon: $C + R(\cos\theta_i, \sin\theta_i)$, với $\theta_i = 2\pi i/n$.
Roots of unity: ω^i cho FFT/regular polygons.

Math patterns worth memorizing

1) Factor first, then multiply prime-power contributions.
2) Replace gcd-exact condition by divisor sums + Möbius/inversion.
3) Replace repeated floor values by quotient blocks.
4) Replace huge index recurrence by matrix/Kitamasa/BM.
5) Replace symmetry enumeration by Burnside.
6) Replace subset pair loops by MITM/convolution/FWT.
7) Replace "at least one" by complement/PIE.
8) Replace expected count by indicator variables.
9) Replace geometry slope by vector cross/dot.
10) Replace exact big arithmetic by modulo/log only if the statement permits it.

CHỌN THUẬT TOÁN - đường đi ngắn

Điều kiện	Dùng gì
Cạnh không trọng số	BFS, $O(V+E)$
Trọng số chỉ 0/1	0-1 BFS, deque, $O(V+E)$
Mọi cạnh ≥ 0	Dijkstra, $O(E \log V)$
Có cạnh âm, không cần APSP	Bellman-Ford; phát hiện negative cycle
DAG	Topological DP, kể cả cạnh âm
APSP, V nhỏ (~ 500)	Floyd-Warshall $O(V^3)$
k đường ngắn nhất	Dijkstra giữ tối đa k dist/node
Exact K edges	Matrix power / min-plus matrix

Điều kiện quan trọng: Dijkstra sai nếu có cạnh âm. BFS chỉ đúng khi mỗi bước có cùng cost. Nếu graph là DAG thì topo thường đơn giản hơn Bellman-Ford.

CHỌN THUẬT TOÁN - connectivity/MST

Dạng	Dùng gì / điều kiện
CC tĩnh	DFS/BFS
Union/query online	DSU
Xóa cạnh offline	Rollback DSU + time segment tree
MST sparse	Kruskal + DSU
MST dense/adj list tự nhiên	Prim
Second-best MST	MST + max-edge LCA
Minimax path	MST + max-edge path query
Directed components	SCC
Cầu/đỉnh khớp	Tarjan DFS, undirected

Kruskal/Prim yêu cầu graph vô hướng cho MST chuẩn. SCC condensation luôn là DAG - thường tiếp tục bằng topo/DP.

CHỌN DATA STRUCTURE

Nhu cầu	Dùng gì
Prefix/range sum + point update	Fenwick
Range add + range sum	2 Fenwick hoặc Lazy SegTree
Min/max/gcd tĩnh	Sparse Table
Aggregate associative động	Segment Tree
k-th theo frequency	Fenwick/SegTree descent
Order statistics set	PBDS
Range reverse/insert/delete sequence	Implicit Treap
Min/max của nhiều line tại x	Li Chao / CHT
Offline range query, add/del rẻ	Mo
Connectivity rollback	Rollback DSU

Điều kiện: Segment Tree cần phép gộp associative. Sparse Table $O(1)$ kiểu overlap cần idempotent như min/max/gcd. CHT monotone cần slope/query có thứ tự; không chắc thì Li Chao an toàn hơn.

CHỌN QHD

Dấu hiệu	Dùng gì
$n \leq 20$ set lựa chọn	Bitmask DP
$n \approx 40$ subset	Meet-in-the-middle
Đoạn $[l, r]$	Interval DP, len ở ngoài
Grid width nhỏ	Profile/Broken-profile DP
Số rất lớn theo chữ số	Digit DP
DAG state graph	Topological DP
Transition split có monotone opt	D&C DP optimization
Interval + quadrangle inequality	Knuth optimization
Transition dạng $m_i x + b_i$	CHT/Li Chao DP
Subset sum vừa phải	bitset DP

Trước khi tối ưu DP, viết recurrence đúng và xác định dependency. D&C/Knuth chỉ dùng khi điều kiện monotonicity/Monge thực sự đúng.

CHỌN STRING

Bài toán	Dùng gì
1 pattern trong text	KMP/Z
Prefix/border/period	prefix-function hoặc Z
Palindrome dài nhất	Manacher
Substring equality	rolling hash
Dictionary + prefix DP	Trie
Nhiều pattern cùng lúc	Aho-Corasick
Thứ tự suffix / LCP	Suffix Array + LCP
Append online / substring	Suffix Automaton
Distinct palindromes	Eertree
Min cyclic rotation	Booth

Hash có collision; khi correctness tuyệt đối quan trọng, ưu tiên suffix structures hoặc double/64-bit randomized hash.

Aho-Corasick FULL

```
const int AL=26;
int nxA[N][AL],linkA[N],outA[N],nodesA=1;
void acAdd(string s){
  int u=0;
  for(char ch:s){
    int c=ch-'a';
    if(!nxA[u][c])nxA[u][c]=nodesA++;
    u=nxA[u][c];
  }
  outA[u]++;
}
vector<int> bfsOrderA;
void acBuild(){
  queue<int>q;
  for(int c=0;c<AL;c++)
    if(nxA[0][c])q.push(nxA[0][c]);
  while(!q.empty()){
    int u=q.front();q.pop();
    bfsOrderA.pb(u);
    for(int c=0;c<AL;c++){
      int v=nxA[u][c];
      if(v){
        linkA[v]=nxA[linkA[u]][c];
        q.push(v);
      }else nxA[u][c]=nxA[linkA[u]][c];
    }
  }
}
ll occA[N];
void acScan(string t){
  int u=0;
  for(char ch:t){
    u=nxA[u][ch-'a'];
    occA[u]++;
  }
  for(int i=sz(bfsOrderA)-1;i>=0;i--){
    int u=bfsOrderA[i];
    occA[linkA[u]]+=occA[u];
  }
}
```

Dùng khi: nhiều pattern, tổng độ dài lớn. Build $O(\sum |p|\ AL)$ nếu fill transition như trên; scan $O(|T|)$. Để biết count từng pattern, lưu terminal node của pattern rồi đọc occA[node].

Suffix Array + LCP FULL

```
vector<int> suffixArray(string s){
  s+='$';int n=s.size();
  vector<int>p(n),c(n);
  vector<pair<char,int>> a(n);
  for(int i=0;i<n;i++)a[i]={s[i],i};
  sort(all(a));
  for(int i=0;i<n;i++)p[i]=a[i].second;
  c[p[0]]=0;
  for(int i=1;i<n;i++)
    c[p[i]]=c[p[i-1]]+(a[i].first!=a[i-1].first)
      ;
  for(int k=0;(1<<k)<n;k++){
    int d=1<<k;
    for(int i=0;i<n;i++)p[i]=(p[i]-d+n)%n;
    int C=*max_element(all(c))+1;
    vector<int>cnt(C),np(n);
    for(int x:c)cnt[x]++;
    for(int i=1;i<C;i++)cnt[i]+=cnt[i-1];
    for(int i=n-1;i>=0;i--)np[--cnt[c[p[i]]]]=p[
     i];
    p=np;
    vector<int>nc(n);nc[p[0]]=0;
    for(int i=1;i<n;i++){
      pair<int,int>cur={c[p[i]],c[(p[i]+d)%n]};
      pair<int,int>pre={c[p[i-1]],c[(p[i-1]+d)%n
     ]};
      nc[p[i]]=nc[p[i-1]]+(cur!=pre);
    }
    c=nc;
  }
  p.erase(p.begin());return p;
}
vector<int> kasai(string s,vector<int> sa){
  int n=s.size(),k=0;
  vector<int>r(n),lcp(max(0,n-1));
  for(int i=0;i<n;i++)r[sa[i]]=i;
  for(int i=0;i<n;i++){
    if(r[i]==n-1){k=0;continue;}
    int j=sa[r[i]+1];
    while(i+k<n&&j+k<n&&s[i+k]==s[j+k])k++;
    lcp[r[i]]=k;
    if(k)k--;
  }
  return lcp;
}
```

Dùng khi: cần thứ tự từ điển suffix, distinct substrings, repeating substring, substring order. Distinct substrings $= n(n+1)/2 - \sum LCP$. Build $O(N \log N)$.

Suffix Automaton FULL

```
struct SAMNode{
  int len=0,link=-1;
  map<char,int> next;
  ll occ=0;
};
vector<SAMNode> sam(1);
int lastS=0;
void samExtend(char c){
  int cur=sam.size();sam.pb({});
  sam[cur].len=sam[lastS].len+1;
  sam[cur].occ=1;
  int p=lastS;
  while(p!=-1&&!sam[p].next.count(c)){
    sam[p].next[c]=cur;p=sam[p].link;
  }
  if(p==-1)sam[cur].link=0;
  else{
    int q=sam[p].next[c];
    if(sam[p].len+1==sam[q].len)sam[cur].link=q;
    else{
      int clone=sam.size();sam.pb(sam[q]);
      sam[clone].len=sam[p].len+1;
      sam[clone].occ=0;
      while(p!=-1&&sam[p].next[c]==q){
        sam[p].next[c]=clone;p=sam[p].link;
      }
      sam[q].link=sam[cur].link=clone;
    }
  }
  lastS=cur;
}
ll distinctSAM(){
  ll ans=0;
  for(int v=1;v<sz(sam);v++)
    ans+=sam[v].len-sam[sam[v].link].len;
  return ans;
}
```

Dùng khi: build online, distinct substrings, occurrence/frequency, kth substring. Sau build, sort states theo len giảm và cộng occ[v] về suffix link để có occurrence. Có tối đa $2N-1$ states.

Eertree FULL - distinct palindromes

```
struct EN{int len,link,nx[26],occ;};
EN et[N];int ec=2,lastE=1;
string es="#";
void eInit(){
  et[0].len=-1;et[0].link=0;
  et[1].len=0; et[1].link=0;
}
int getLinkE(int v,int pos){
  while(pos-1-et[v].len<0||
        es[pos-1-et[v].len]!=es[pos])
    v=et[v].link;
  return v;
}
void eAdd(char ch){
  es+=ch;int pos=es.size()-1,c=ch-'a';
  int cur=getLinkE(lastE,pos);
  if(!et[cur].nx[c]){
    int nw=ec++;
    et[nw].len=et[cur].len+2;
    if(et[nw].len==1)et[nw].link=1;
    else{
      int q=getLinkE(et[cur].link,pos);
      et[nw].link=et[q].nx[c];
    }
    et[cur].nx[c]=nw;
  }
  lastE=et[cur].nx[c];
  et[lastE].occ++;
}
```

Dùng khi: cần mọi palindrome khác nhau, occurrence theo palindrome, palindrome online. Số node $\leq N+2$. Propagate occurrence từ palindrome dài xuống suffix link.

String advanced - conditions

SA vs SAM: SA tốt cho offline lexicographic/range LCP; SAM tốt cho append online và automaton substring.

Aho vs Trie: Trie chỉ chia sẻ prefix; Aho thêm failure links để scan text một lần.

Manacher vs Eertree: Manacher cho radius/longest trong $O(N)$; Eertree liệt kê distinct palindrome và quan hệ suffix.

Hash: dễ code nhất cho equality/LCP bằng binary search, nhưng collision là rủi ro.

Min-Cost Max-Flow - potentials + Dijkstra

Ngữ cảnh dùng: Không chỉ cần lượng flow tối đa mà còn tối thiểu hóa/tối đa hóa cost. Reduced costs phải không âm trước mỗi Dijkstra; cost âm ban đầu cần potential khởi tạo.

```
struct ME{int v,rev;ll cap,cost;};
vector<ME> mg[N];
void addM(int u,int v,ll cap,ll cost){
  ME a{v,(int)mg[v].size(),cap,cost};
  ME b{u,(int)mg[u].size(),0,-cost};
  mg[u].pb(a);mg[v].pb(b);
}
pair<ll,ll> mcmf(int s,int t,ll need=LINF){
  ll flow=0,cost=0;
  vector<ll> pot(n+1),d(n+1);
  vector<int> pv(n+1),pe(n+1);
  // If negative original costs exist,
     initialize pot by Bellman-Ford/SPFA.
  while(flow<need){
    fill(all(d),LINF);d[s]=0;
    priority_queue<pair<ll,int>,vector<pair<ll,
     int>>,
      greater<pair<ll,int>>>pq;
    pq.push({0,s});
    while(!pq.empty()){
      auto [du,u]=pq.top();pq.pop();
      if(du!=d[u])continue;
      for(int i=0;i<sz(mg[u]);i++){
        auto &e=mg[u][i];
        if(!e.cap)continue;
        ll nd=du+e.cost+pot[u]-pot[e.v];
        if(nd<d[e.v]){
          d[e.v]=nd;pv[e.v]=u;pe[e.v]=i;
          pq.push({nd,e.v});
        }
      }
    }
    if(d[t]==LINF)break;
    for(int v=1;v<=n;v++)if(d[v]<LINF)pot[v]+=d[
     v];
    ll add=need-flow;
    for(int v=t;v!=s;v=pv[v])
      add=min(add,mg[pv[v]][pe[v]].cap);
    for(int v=t;v!=s;v=pv[v]){
      auto &e=mg[pv[v]][pe[v]];
      cost+=add*e.cost;
      e.cap-=add;mg[v][e.rev].cap+=add;
    }
    flow+=add;
  }
  return {flow,cost};
}
```

Dùng khi: flow có cost trên mỗi đơn vị, assignment có capacity, min-cost disjoint paths. Reduced cost phải không âm để Dijkstra đúng; nếu ban đầu có cạnh cost âm, khởi tạo potential bằng shortest paths.

Flow/Matching - chọn gì

Mô hình	Thuật toán
Max flow / min cut	Dinic
Bipartite unweighted matching	Hopcroft-Karp
Assignment min-cost dense	Hungarian
Matching + capacities/cost	MCMF
Vertex-disjoint paths	split vertex + flow
Edge-disjoint paths	capacity edge = 1
Lower bound $L \le f \le R$	circulation + demands
Min path cover DAG	$n-$max bip matching

Nếu graph matching **không bipartite**, Hopcroft-Karp/Hungarian không áp dụng; general matching cần Blossom hoặc DP bitmask nếu n nhỏ.

König/Hall - điều kiện matching

König: trong bipartite graph, size maximum matching = size minimum vertex cover.

Hall: perfect matching từ Left sang Right tồn tại iff mọi subset S của Left có $|N(S)| \ge |S|$.

Contest thường không kiểm tra Hall trực tiếp; dùng max matching để quyết định feasibility.

Flow decomposition

```
// After integral max-flow, repeatedly DFS along
     edges carrying flow.
// Original edge flow = originalCap -
     residualCap.
// Extract s->t path, subtract bottleneck,
     repeat.
```

Dùng khi: đề không chỉ hỏi giá trị flow mà cần in từng route, như edge-disjoint paths. Với integer capacities, max-flow integral nên decomposition thành paths/cycles được.

Gomory-Hu - mọi cặp min-cut undirected

Dùng khi cần min-cut giữa nhiều cặp trên graph vô hướng có capacity. Gomory-Hu tree dùng $N-1$ lần max-flow; min-cut (u,v) bằng minimum edge trên path trong tree. Chỉ đáng giữ nếu N vừa và nhiều query min-cut.

Polynomial/FPS - điều kiện trước khi dùng

NTT: convolution modulo prime dạng $c \cdot 2^k + 1$ có primitive root, ví dụ 998244353.
FPS inverse: cần $f_0 \neq 0$.
FPS log: cần $f_0 = 1$.
FPS exp: cần $f_0 = 0$.
FPS sqrt bản đơn giản: thường giả sử $f_0 = 1$; general case cần modular sqrt + shift.

Newton doubling cho inverse/log/exp thường $O(N \log N)$ nếu convolution NTT.

Polynomial helpers

```
using Poly=vector<int>;
Poly cut(Poly a,int n){a.resize(n);return a;}
Poly derivP(const Poly&a){
  Poly r(max(0,sz(a)-1));
  for(int i=1;i<sz(a);i++)r[i-1]=(ll)a[i]*i%MODN
     ;
  return r;
}
Poly integP(const Poly&a){
  Poly r(sz(a)+1);r[0]=0;
  static vector<int> inv(2,1);
  while(sz(inv)<=sz(a)){
    int i=inv.size();
    inv.pb(MODN-(ll)(MODN/i)*inv[MODN%i]%MODN);
  }
  for(int i=0;i<sz(a);i++)r[i+1]=(ll)a[i]*inv[i
     +1]%MODN;
  return r;
}
void normP(Poly&a){
  while(!a.empty()&&a.back()==0)a.pop_back();
}
```

FPS inverse FULL

```
Poly invPoly(const Poly&f,int n){
  Poly g(1,binpow(f[0],MODN-2,MODN));
  for(int m=1;m<n;m<<=1){
    int need=min(n,m<<1);
    Poly fc(min(sz(f),need));
    for(int i=0;i<sz(fc);i++)fc[i]=f[i];
    Poly t=conv(fc,g);t.resize(need);
    for(int i=0;i<need;i++)
      t[i]=(MODN-t[i])%MODN;
    t[0]=(t[0]+2)%MODN;
    g=conv(g,t);g.resize(need);
  }
  g.resize(n);return g;
}
```

Điều kiện: $f[0] \neq 0$. Newton: $g \leftarrow g(2 - fg)$ modulo x^m.

FPS logarithm FULL

```
Poly logPoly(const Poly&f,int n){
  // require f[0] = 1
  Poly df=derivP(f);
  Poly invf=invPoly(f,n);
  Poly q=conv(df,invf);
  if(sz(q)>n-1)q.resize(n-1);
  Poly r=integP(q);
  r.resize(n);return r;
}
```

$\log f = \int f'/f$. **Điều kiện chuẩn:** $f_0 = 1$.

FPS exponential FULL

```
Poly expPoly(const Poly&f,int n){
  // require f[0] = 0
  Poly g(1,1);
  for(int m=1;m<n;m<<=1){
    int need=min(n,m<<1);
    Poly lg=logPoly(g,need);
    Poly diff(need);
    for(int i=0;i<need;i++){
      int fi=i<sz(f)?f[i]:0;
      int li=i<sz(lg)?lg[i]:0;
      diff[i]=(fi-li+MODN)%MODN;
    }
    diff[0]=(diff[0]+1)%MODN;
    g=conv(g,diff);g.resize(need);
  }
  g.resize(n);return g;
}
```

Điều kiện: $f_0 = 0$. Newton từ phương trình $\log g = f$.

FPS square root - $f_0 = 1$

```
Poly sqrtPoly1(const Poly&f,int n){
  Poly g(1,1);
  int inv2=(MODN+1)/2;
  for(int m=1;m<n;m<<=1){
    int need=min(n,m<<1);
    Poly fc(need);
    for(int i=0;i<need&&i<sz(f);i++)fc[i]=f[i];
    Poly ig=invPoly(g,need);
    Poly q=conv(fc,ig);q.resize(need);
    g.resize(need);
    for(int i=0;i<need;i++)
      g[i]=(ll)(g[i]+q[i])*inv2%MODN;
  }
  g.resize(n);return g;
}
```

General FPS sqrt cần tìm sqrt của constant term bằng Tonelli-Shanks và xử lý leading zeros. Template này chỉ dùng khi $f_0 = 1$.

Polynomial division/mod

Đa thức A/B: reverse coefficients, dùng FPS inverse để lấy quotient nhanh; remainder $R = A - BQ$. Với degree nhỏ, long division $O(nm)$ dễ code và ít bug hơn. Chỉ dùng fast division khi constraints thật sự lớn.

Multipoint evaluation/interpolation - khi nào

Nếu cần evaluate polynomial bậc N tại N điểm hoặc interpolate từ N điểm với N lớn, dùng product tree + remainder tree, khoảng $O(N \log^2 N)$ với NTT. Nếu chỉ một/vài điểm, Horner/Lagrange đơn giản hơn.

ICPC WF pattern: metric closure + bitmask DP

Dấu hiệu: input là grid/graph lớn nhưng chỉ có $K \le 20$ "điểm quan trọng" cần ghé.
Cách: BFS/Dijkstra từ mỗi special -> ma trận khoảng cách $K \times K$ -> bitmask DP/TSP.
Điều kiện: shortest path giữa special points độc lập với thứ tự ghé; nếu resource/state thay đổi theo đường đi thì metric closure có thể sai.

```
ll wfVisitAll(vector<vector<int>>&g, vector<int>
     sp){
  int n=sz(g),k=sz(sp);if(!k)return 0;
  vector<vector<int>> d(k,vector<int>(k));
  for(int i=0;i<k;i++){
    vector<int> dis(n,-1);queue<int> q;
    dis[sp[i]]=0;q.push(sp[i]);
    while(!q.empty()){
      int u=q.front();q.pop();
      for(int v:g[u])if(dis[v]<0)
        dis[v]=dis[u]+1,q.push(v);
    }
    for(int j=0;j<k;j++)d[i][j]=dis[sp[j]];
  }
  const ll BIG=(1LL<<60);
  vector<vector<ll>>dp(1<<k,vector<ll>(k,BIG));
  dp[1][0]=0; // sp[0] = start
  for(int m=1;m<(1<<k);m++)
    for(int u=0;u<k;u++)if(dp[m][u]<BIG)
      for(int v=0;v<k;v++)
        if(!(m>>v&1)&&d[u][v]>=0)
          dp[m|(1<<v)][v]=min(dp[m|(1<<v)][v],
                              dp[m][u]+d[u][v]);
  ll ans=*min_element(all(dp.back()));
  return ans==BIG?-1:ans;
}
```

Graph không trọng số; `sp[0]` là điểm bắt đầu, chỉ cần ghé đủ điểm (không bắt buộc quay về). Graph có trọng số không âm: thay BFS bằng Dijkstra.

ICPC WF pattern: binary search answer + checker

Dấu hiệu: hỏi min/max một giá trị và feasibility monotone.
Checker có thể là greedy, BFS connectivity, max-flow, matching, geometry, scheduling.
Invariant cần chứng minh: nếu X feasible thì mọi X' theo một phía cũng feasible.

```
bool wfCan(vector<ll>&a,ll cap,int k){
  int groups=1;ll sum=0;
  for(ll x:a){
    if(x>cap)return false;
    if(sum+x>cap){groups++;sum=0;}
    sum+=x;
  }
  return groups<=k;
}
ll wfMinLargestSum(vector<ll>a,int k){
  ll lo=-1,hi=0;
  for(ll x:a)hi+=x;
  while(hi-lo>1){
    ll md=lo+(hi-lo)/2;
    if(wfCan(a,md,k))hi=md;
    else lo=md;
  }
  return hi;
}
```

Ví dụ chia mảng **không âm** thành tối đa k đoạn liên tiếp, tối thiểu hóa tổng đoạn lớn nhất. Checker greedy $O(N)$; tổng $O(N \log S)$.

ICPC WF pattern: SCC -> DAG -> DP

Dấu hiệu: directed graph có cycle nhưng objective là max/min/count theo đường đi.
SCC gom các node "đi qua lại được", cộng weight/attribute theo component -> condensation DAG -> topo DP.
Điều kiện: contribution bên trong SCC phải gom được thành một state hợp lệ cho bài.

```
ll wfSccMaxPath(int n,int C,vector<int>&comp,
                vector<ll>&w,vector<pair<int,int
    >>&ed){
  vector<ll>cw(C),dp(C);
  vector<vector<int>>dag(C);
  vector<int>deg(C);
  for(int i=0;i<n;i++)cw[comp[i]]+=w[i];
  for(auto [u,v]:ed)if(comp[u]!=comp[v]){
    dag[comp[u]].pb(comp[v]);deg[comp[v]]++;
  }
  queue<int>q;
  for(int c=0;c<C;c++){
    dp[c]=cw[c];if(!deg[c])q.push(c);
  }
  ll ans=LLONG_MIN;
  while(!q.empty()){
    int u=q.front();q.pop();ans=max(ans,dp[u]);
    for(int v:dag[u]){
      dp[v]=max(dp[v],dp[u]+cw[v]);
      if(--deg[v]==0)q.push(v);
    }
  }
  return ans;
}
```

Cần có `comp[u]` từ Tarjan/Kosaraju (0-based). Mỗi SCC được tính tổng trọng số một lần và có thể chọn điểm bắt đầu bất kỳ. Duplicated DAG edges không ảnh hưởng đúng/sai.

ICPC WF pattern: offline sweep + DS

Sort query/events theo một coordinate/time/threshold, rồi add objects dần vào Fenwick/Segment Tree/DSU.
Ví dụ: query "bao nhiêu phần tử $\le X$ trong đoạn", connectivity với edges weight $\le X$, dominance counting.
Dấu hiệu: query có một tham số monotone cho phép reorder offline.

```
struct WFQ{int l,r,x,id;}; // 0-based [l,r]
vector<int> wfCountLE(vector<int>a,vector<WFQ>qs
    ){
  int n=sz(a);vector<int>ord(n),bit(n+1);
  iota(all(ord),0);
  sort(all(ord),[&](int i,int j){return a[i]<a[j
    ];});
  sort(all(qs),[](WFQ u,WFQ v){return u.x<v.x;})
    ;
  auto add=[&](int p){for(++p;p<=n;p+=p&-p)bit[p
    ]++;};
  auto sum=[&](int p){
    int r=0;for(++p;p>0;p-=p&-p)r+=bit[p];
    return r;
  };
  vector<int>ans(sz(qs));int j=0;
  for(auto q:qs){
    while(j<n&&a[ord[j]]<=q.x)add(ord[j++]);
    ans[q.id]=sum(q.r)-(q.l?sum(q.l-1):0);
  }
  return ans;
}
```

Offline query đếm $a[i] \le X$ trong $[l,r]$. Input `id=0..Q-1`. Sort theo ngưỡng, mỗi phần tử chỉ thêm một lần, Fenwick trả lời đoạn $O((N+Q)\log N)$.

ICPC WF pattern: divide & conquer on time + rollback

Dynamic add/delete edges + connectivity/query. Chuyển mỗi object thành interval thời gian tồn tại, gắn vào segment tree thời gian, DFS tree và rollback state.
Điều kiện: update có thể apply và undo rẻ; DSU rollback là ví dụ kinh điển.

```
// Need snap(),uniteRB(),findRB(),rollback().
// Each active edge: (l,r,u,v) on time [l,r).
vector<char> wfDynamic(int Q,vector<array<int
    ,4>>edge,
                vector<pair<int,int>>ask){
  using E=pair<int,int>;
  vector<vector<E>> seg(4*Q+4);
  auto add=[&](auto&&self,int id,int l,int r,
               int ql,int qr,E e)->void{
    if(qr<=l||r<=ql)return;
    if(ql<=l&&r<=qr){seg[id].pb(e);return;}
    int m=(l+r)/2;
    self(self,id*2,l,m,ql,qr,e);
    self(self,id*2+1,m,r,ql,qr,e);
  };
  for(auto [l,r,u,v]:edge)
    add(add,1,0,Q,l,r,{u,v});
  vector<char>ans(Q);
  auto dfs=[&](auto&&self,int id,int l,int r)->
     void{
    int checkpoint=snap();
    for(auto [u,v]:seg[id])uniteRB(u,v);
    if(r-l==1){
      auto [u,v]=ask[l];
      if(u>=0)ans[l]=(findRB(u)==findRB(v));
    }else{
      int m=(l+r)/2;
      self(self,id*2,l,m);self(self,id*2+1,m,r);
    }
    rollback(checkpoint);
  };
  if(Q)dfs(dfs,1,0,Q);
  return ans; // connectivity at time t
}
```

Graph vô hướng; cạnh tồn tại trên các interval $[l,r)$ và mỗi thời điểm có tối đa một query. Cần khởi tạo rollback DSU trước; nếu muốn trả kết quả ra ngoài, đổi hàm thành trả `vector<char>`.

ICPC WF pattern: small separator / meet-in-middle

Nếu brute force 2^N quá lớn nhưng cấu trúc chia thành hai nửa độc lập gần đúng: enumerate mỗi nửa $2^{N/2}$ rồi sort/hash/match.
Không chỉ subset sum: assignment nhỏ, path/state search, bidirectional BFS cũng cùng tư tưởng "gặp ở giữa".

```
ll wfCountSubsets(vector<ll>a,ll target){
  int n=sz(a),mid=n/2;
  vector<ll>L={0},R={0};
  for(int i=0;i<mid;i++){
    int m=sz(L);
    for(int j=0;j<m;j++)L.pb(L[j]+a[i]);
  }
  for(int i=mid;i<n;i++){
    int m=sz(R);
    for(int j=0;j<m;j++)R.pb(R[j]+a[i]);
  }
  sort(all(R));ll ans=0;
  for(ll s:L){
    auto [lo,hi]=equal_range(all(R),target-s);
    ans+=hi-lo;
  }
  return ans;
}
```

Đếm số tập con có tổng đúng `target`, kể cả tập rỗng. Dùng khi $N \approx 40$ nhưng 2^N quá lớn; xử lý được số âm và phần tử lặp.

ICPC WF pattern: state compression

State nhìn ban đầu có 3-5 tham số -> tìm tham số có thể suy ra từ các tham số còn lại và invariant (sum, area, popcount, processed index), rồi bỏ nó.
Đây thường là khác biệt giữa MLE/TLE và AC. Luôn hỏi: "biết các state variables này thì biến kia có recover được không?".

```
// 0/1 knapsack: dp[item][capacity] -> dp[
    capacity]
vector<ll> wfKnapsack(vector<pair<int,ll>>items,
    int W){
  const ll BAD=-(1LL<<60);
  vector<ll>dp(W+1,BAD);dp[0]=0;
  for(auto [weight,val]:items)
    for(int c=W;c>=weight;c--)
      if(dp[c-weight]!=BAD)
        dp[c]=max(dp[c],dp[c-weight]+val);
  return dp; // dp[c] = best with EXACT weight c
}
```

Giảm từ $O(NW)$ bộ nhớ xuống $O(W)$ vì hàng i chỉ phụ thuộc hàng $i-1$. **Bắt buộc quét capacity giảm** để không dùng một vật nhiều lần; giả sử weight dương.

ICPC WF pattern: exchange / greedy proof

Greedy thường đúng khi có exchange argument: từ một optimal solution bất kỳ, đổi lựa chọn đầu thành greedy choice mà không làm tệ hơn.
Dấu hiệu classic: interval scheduling, MST cut property, earliest deadline variants. Không code greedy chỉ vì "trông hợp lý" - cần invariant/exchange.

```
int wfMaxIntervals(vector<pair<ll,ll>>seg){
  sort(all(seg),[](auto a,auto b){
    if(a.second!=b.second)return a.second<b.
      second;
    return a.first<b.first;
  });
  ll finish=LLONG_MIN;int ans=0;
  for(auto [l,r]:seg)if(l>=finish){
    ans++;finish=r;
  }
  return ans;
}
```

Ví dụ interval scheduling: tối đa số đoạn **nửa mở** $[l,r)$ không chồng nhau. Exchange argument: chọn đoạn kết thúc sớm nhất không làm giảm số đoạn tối ưu.

ICPC WF pattern: min-cut modeling

Binary decision/choose one of two labels + pairwise penalty thường model thành s-t cut.
Unary cost -> source/node hoặc node/sink capacity; pairwise disagreement penalty -> edge giữa nodes.
Điều kiện: energy phải submodular cho binary graph-cut modeling chuẩn.

```
// Uses addEdge() and dinic() from Flow section.
// vertices 1..V, label 0=source-side.
ll wfBinaryEnergy(int V,vector<ll>&c0,vector<ll
    >&c1,
                  vector<tuple<int,int,ll>>&
    penalty){
  int S=V+1,T=V+2;
  for(int i=1;i<=V;i++){
    addEdge(S,i,c1[i]); // pay if label 1
    addEdge(i,T,c0[i]); // pay if label 0
  }
  for(auto [u,v,w]:penalty){
    addEdge(u,v,w);addEdge(v,u,w);
  }
  n=T; // global vertex count used by dinic()
  return dinic(S,T);
}
```

Energy $\sum_i c_i(label_i) + \sum_{uv} w_{uv}[label_u \ne label_v]$, với costs ≥ 0. Phải **reset residual graph** rồi gọi; cấu trúc Dinic hiện tại dùng biến đếm đỉnh toàn cục n.

ICPC WF pattern: geometry -> graph

Các object hình học trở thành vertices; edge tồn tại nếu giao nhau/nhìn thấy/khoảng cách $\le D$. Sau khi build graph, bài còn lại có thể là CC, shortest path, MST, matching.
Cẩn thận $O(N^2)$ pair tests; nếu N lớn cần sweep line/spatial hashing/convexity.

```
vector<int> wfCluster(vector<pair<ll,ll>>p,ll R)
    {
  int n=sz(p);vector<int>pa(n),siz(n,1);
  iota(all(pa),0);
  auto find=[&](int x){
    while(pa[x]!=x){pa[x]=pa[pa[x]];x=pa[x];}
    return x;
  };
  for(int i=0;i<n;i++)for(int j=i+1;j<n;j++){
    ll dx=p[i].first-p[j].first;
    ll dy=p[i].second-p[j].second;
    if((int128)dx*dx+(int128)dy*dy>(int128)R*R)
      continue;
    int a=find(i),b=find(j);
    if(a==b)continue;
    if(siz[a]<siz[b])swap(a,b);
    pa[b]=a;siz[a]+=siz[b];
  }
  for(int i=0;i<n;i++)pa[i]=find(i);
  return pa; // same representative = connected
}
```

Các điểm nối cạnh khi khoảng cách Euclid $\le R$, tìm connected components. Tránh `sqrt` và overflow bằng `int128`. $O(N^2)$ pair checks: chỉ dùng khi N vừa.

ICPC WF pattern: generating function / convolution

Nếu transition là "chia tổng thành i+j": $dp[k] = \sum_i A[i]B[k-i]$, nghĩ convolution/NTT.
Nếu lặp nhiều lần cùng distribution, polynomial exponentiation.
Điều kiện: operation thật sự là convolution theo index; nếu có min/max thì NTT không áp dụng trực tiếp.

```
// Uses conv() with mod 998244353 from NTT
     section.
vector<int> wfDiceCount(int dice,int maxSum){
  vector<int> base(7),res={1};
  for(int d=1;d<=6;d++)base[d]=1;
  while(dice){
    if(dice&1){
      res=conv(res,base);
      if(sz(res)>maxSum+1)res.resize(maxSum+1);
    }
    dice>>=1;
    if(dice){
      base=conv(base,base);
      if(sz(base)>maxSum+1)base.resize(maxSum+1)
      ;
    }
  }
  res.resize(maxSum+1);return res;
}
```

Hệ số bậc s của $(x+x^2+\cdots+x^6)^{dice}$ là số cách ra tổng s. Dùng NTT khi dice lớn và cần nhiều tổng; bài nhỏ dùng DP $O(6NS)$ đơn giản hơn.

ICPC WF pattern: randomized incremental

Một số geometry/DS có expected complexity tốt khi shuffle: Minimum Enclosing Circle, randomized treap, incremental constructions.
Điều kiện: random order phá adversarial structure; dùng seed ổn định nếu cần reproducible debugging.

```
// Needs Cir,PD,inside(),dist(),circumcenter().
Cir wfCircle3(PD a,PD b,PD c){
  vector<PD>p={a,b,c};
  Cir best{{0,0},1e100L};
  for(int i=0;i<3;i++)for(int j=i+1;j<3;j++){
    Cir t{{(p[i].ft+p[j].ft)/2,
           (p[i].sd+p[j].sd)/2},dist(p[i],p[j])
     /2};
    if(t.r<best.r&&inside(t,a)&&inside(t,b)&&
      inside(t,c))
       best=t;
  }
  if(best.r<1e99L)return best;
  PD o;
  if(circumcenter(a,b,c,o))return {o,dist(o,a)};
  return best; // collinear: some diameter works
}
Cir wfMinCircle(vector<PD>p){
  mt19937 rng(712367);shuffle(all(p),rng);
  Cir C{{0,0},-1};
  for(int i=0;i<sz(p);i++)if(C.r<0||!inside(C,p[
     i])){
    C={p[i],0};
    for(int j=0;j<i;j++)if(!inside(C,p[j])){
      C={{(p[i].ft+p[j].ft)/2,
          (p[i].sd+p[j].sd)/2},dist(p[i],p[j])
     /2};
      for(int k=0;k<j;k++)if(!inside(C,p[k])){
        C=wfCircle3(p[i],p[j],p[k]);
      }
    }
  }
  return C;
}
```

Bài minimum enclosing circle phủ **tất cả** điểm. Random shuffle để có expected $O(N)$. Hàm ba điểm xử lý tam giác tù hoặc thẳng hàng bằng diameter của một cặp khi phù hợp, rồi mới xét circumcircle.

ICPC WF pattern: optimize only after compression

Pipeline hay gặp: coordinate compress -> sort events -> prefix/Fenwick/SegTree. Nếu values đến 10^9 nhưng chỉ N values thực sự xuất hiện, không cấp array theo value range.

```
ll wfInversions(vector<ll>a){
  vector<ll>vals=a;
  sort(all(vals));
  vals.erase(unique(all(vals)),vals.end());
  int m=sz(vals);vector<int>bit(m+1);
  auto add=[&](int i){
    for(;i<=m;i+=i&-i)bit[i]++;
  };
  auto pref=[&](int i){
    int s=0;for(;i;i-=i&-i)s+=bit[i];
    return s;
  };
  ll ans=0;
  for(int i=0;i<sz(a);i++){
    int id=lower_bound(all(vals),a[i])-vals.
      begin()+1;
    ans+=i-pref(id); // earlier > a[i]
    add(id);
  }
  return ans;
}
```

Mảng giá trị lớn/âm/lặp: nén tọa độ rồi Fenwick đếm inversions trong $O(N \log N)$. Kỹ thuật này dùng lại được cho rank, k-th và bài sweep line.

Precondition cheat sheet

Thuật toán	Điều kiện bắt buộc / lưu ý
Dijkstra	edge weight ≥ 0
Topo DP	DAG
Kruskal/Prim	undirected weighted graph
Sparse Table $O(1)$	static + idempotent op
Binary Search Answer	feasibility monotone
Ternary/Golden	unimodal function
Knuth	quadrangle/Monge + opt monotone
D&C DP	opt index monotone
CHT monotone	slope/query order phù hợp
Li Chao	functions là lines; x-domain biết/quản lý được
Hopcroft-Karp	bipartite unweighted matching
Hungarian	bipartite assignment weighted
Aho	finite alphabet, many patterns
NTT	suitable modulus/root
FPS inverse	$f_0 \neq 0$
FPS log	$f_0 = 1$
FPS exp	$f_0 = 0$
Miller-Rabin bases	64-bit deterministic set
Lucas	modulus prime
Tonelli	modulus odd prime

Minimum Circle phủ ít nhất K điểm

Ngữ cảnh dùng: Cần bán kính nhỏ nhất phủ *ít nhất K* trong N điểm, tâm đặt tự do. Với $N \leq 300$, fix cặp điểm biên rồi sweep một tham số 1D cho mỗi cặp.

Dùng khi: N điểm 2D, tâm circle được đặt tự do, cần bán kính nhỏ nhất phủ ít nhất K điểm; có nhiều query K thì nên tiền xử lý cho mọi K.

Structural fact: circle tối ưu có ít nhất 2 điểm input trên biên. Nếu 0 điểm biên thì thu nhỏ được; nếu chỉ 1 điểm biên thì dịch tâm nhẹ về điểm đó rồi tiếp tục thu nhỏ, mâu thuẫn tối ưu.

Fix hai điểm biên A, B. Tâm nằm trên đường trung trực của AB. Đặt $D = B - A$, $V = (-D_y, D_x)$, $M = (A+B)/2$ và $C(z) = M + \frac{z}{2}V$.

Bán kính: $r(z) = \frac{1}{2}\sqrt{|D|^2(1+z^2)}$.

Điểm thứ ba trở thành 1D threshold

Với điểm P, đặt $Q = 2P - A - B$. Điều kiện P nằm trong circle qua A, B: $|P - C(z)|^2 \leq r(z)^2$.

Rút gọn: $2z(Q \cdot V) \geq |Q|^2 - |D|^2$.

Nếu $Q \cdot V > 0$: $z \geq t$ (mốc vào). Nếu $Q \cdot V < 0$: $z \leq t$ (mốc ra).

Trong đó $t = \dfrac{|Q|^2 - |D|^2}{2(Q \cdot V)}$.

Nếu $Q \cdot V = 0$: điểm collinear với AB; nếu $|Q|^2 \leq |D|^2$ thì nằm trong mọi circle qua A, B, ngược lại không bao giờ nằm trong.

Vì sao chỉ xét $z = 0$ và các mốc?

Nếu đường tròn tối ưu chỉ có đúng 2 điểm biên A, B, đoạn AB phải là một đường kính; do đó tâm là trung điểm và $z = 0$.

Nếu $z \neq 0$ ở optimum, phải có ít nhất một điểm biên thứ ba; khi đó z chính là threshold của điểm đó.

Vì vậy với mỗi pair A, B, chỉ cần xét $z = 0$ và $O(N)$ threshold, sort+sweep.

Cập nhật + dựng các mốc

```
struct P{ll x,y;};
struct Ev{ld z;int type;}; // +1 enter, -1 leave
struct Ans{ld x=0,y=0,r=1e100L;bool ok=0;};

vector<P> p;
vector<Ans> best;

void upd(int cnt,ld x,ld y,ld r){
  if(cnt<1||cnt>n)return;
  if(!best[cnt].ok||r<best[cnt].r){
    best[cnt]={x,y,r,1};
  }
}

void solvePair(int i,int j){
  ll ax=p[i].x, ay=p[i].y;
  ll bx=p[j].x, by=p[j].y;
  ll dx=bx-ax, dy=by-ay;
  ld d2=(ld)dx*dx+(ld)dy*dy;
  if(d2==0)return;

  ll vx=-dy, vy=dx;
  vector<Ev> ev;
  int base=0;

  for(int k=0;k<n;k++){
    ld qx=2.0L*p[k].x-ax-bx;
    ld qy=2.0L*p[k].y-ay-by;
    ld num=qx*qx+qy*qy-d2;
    ld s=qx*vx+qy*vy;

    if(fabsl(s)<EPS){
      if(num<=EPS)base++;
      continue;
    }

    ld z=num/(2.0L*s);
    ev.push_back({z,s>0?+1:-1});
  }

  // ... evaluate z=0 and sweep events ...
}
```

Evaluate $z = 0$ + sweep

```
void finishPair(ll ax,ll ay,ll bx,ll by,
                ll vx,ll vy,ld d2,
                int base,vector<Ev>&ev){
  int cnt0=base;
  for(auto e:ev){
    if(e.type==+1 && e.z<=EPS)cnt0++;
    if(e.type==-1 && e.z>=-EPS)cnt0++;
  }
  upd(cnt0,(ax+bx)/2.0L,(ay+by)/2.0L,sqrtl(d2)
     /2);

  sort(all(ev),[](Ev a,Ev b){return a.z<b.z;});

  int cur=base;
  for(auto e:ev)if(e.type==-1)cur++; // z=-
     infinity

  for(int l=0,m=sz(ev);l<m;){
    int r=l,enter=0,leave=0;
    ld z=ev[l].z;

    while(r<m && fabsl(ev[r].z-z)
          <=1e-13L*max((ld)1,fabsl(z))){
      if(ev[r].type==+1)enter++;
      else leave++;
      r++;
    }

    int cover=cur+enter; // ạti đúng z
    ld cx=(ax+bx+z*vx)/2.0L;
    ld cy=(ay+by+z*vy)/2.0L;
    ld rad=sqrtl(d2*(1+z*z))/2.0L;

    upd(cover,cx,cy,rad);

    cur+=enter-leave;
    l=r;
  }
}
```

Driver + nhiều query K

```
// bán kính 0 for duplicate coordinates
map<pair<ll,ll>,int> freq;
for(auto q:p)freq[{q.x,q.y}]++;
for(auto [pt,c]:freq)
  upd(c,(ld)pt.first,(ld)pt.second,0);

// all ặcp đểim biên
for(int i=0;i<n;i++)
  for(int j=i+1;j<n;j++)
    solvePair(i,j);

// >=K: circle covering more points is also
     valid
for(int k=n-1;k>=1;k--)
  if(best[k+1].ok &&
     (!best[k].ok||best[k+1].r<best[k].r))
    best[k]=best[k+1];

// query K -> best[K]
```

Complexity: $\binom{N}{2}$ cặp điểm biên. Mỗi cặp tạo $O(N)$ mốc và sort $O(N \log N)$, tổng $O(N^3 \log N)$; bộ nhớ phụ $O(N)$. Với $N \leq 300$ là phù hợp.

Trường hợp biên: điểm trùng nhau (bán kính 0), nhiều mốc cùng z phải gom cùng lúc, tránh -0.00, dùng long double. Nếu tọa độ lớn, phân tích số nguyên có thể cần __int128 trước khi ép kiểu.

Pattern cần nhớ: cố định 2 điểm biên $\rightarrow$ tham số hóa đường trung trực $\rightarrow$ mỗi điểm tạo ràng buộc một chiều $\rightarrow$ quét các mốc.

TRANG 25 — BIT / GNU C++ / MẪU THUẬT TOÁN HAY

Code mẫu ngắn + điều kiện sử dụng + bẫy thường gặp trong contest

1. Thao tác bit cơ bản (64-bit)

Ngữ cảnh dùng: Dùng khi trạng thái biểu diễn bằng mask: tập con, toggles, parity, bitmask DP, trạng thái đã thăm.

```
using ull=unsigned long long;
ull mask=0;
int i=3;
mask |= (1ULL<<i);      // set bit
mask &= ~(1ULL<<i);     // clear bit
mask ^= (1ULL<<i);      // toggle
bool on=(mask>>i)&1ULL; // test
ull low=mask & -mask;  // lowbit
mask &= (mask-1);       // remove lowbit
```

Điều kiện: $0 \le i < 64$. Với kiểu `int` chỉ shift dưới 31 bit dấu; ưu tiên `1ULL<<i`. `mask & (mask-1)` bỏ bit 1 thấp nhất; không dùng để tìm lowbit nếu cần giữ mask.

2. GNU builtins: đếm/vị trí bit

```
ull x=mask;
int cnt=__builtin_popcountll(x);
int parity=__builtin_parityll(x);
if(x){
  int lo=__builtin_ctzll(x);
  int hi=63-__builtin_clzll(x);
  int bits=64-__builtin_clzll(x);
}
```

Lưu ý: `ctz/clz(0)` không xác định; bắt buộc kiểm tra $x \neq 0$. `popcount(0)=0`. Các hàm `...ll` dùng cho 64-bit; GCC/Clang hỗ trợ.

3. Duyệt bit 1 và duyệt submask

```
for(ull x=mask;x;x&=x-1){
  int b=__builtin_ctzll(x);
  // process bit b (O(popcount))
}
for(int s=mask;s;s=(s-1)&mask){
  // non-empty submasks
}
// include empty submask:
for(int s=mask;;s=(s-1)&mask){
  // process s
  if(s==0)break;
}
```

Dùng khi: $N \lesssim 20$ và mask vừa bộ nhớ/thời gian. Tổng số cặp $(mask, submask)$ trên mọi mask là 3^N. Nếu chỉ cần tổng trên submask, SOS DP $O(N2^N)$ thường tốt hơn.

4. Gosper's Hack: chọn đúng k bit

Ngữ cảnh dùng: Liệt kê mọi tập con k phần tử trong n phần tử, nhanh hơn kiểm tra popcount của tất cả 2^n mask.

```
// 1<=k<=n<64; unsigned 64-bit
ull limit=1ULL<<n;
ull x=(1ULL<<k)-1;
while(x<limit){
  // process subset x
  ull c=x&-x, r=x+c;
  if(r>=limit || r==0)break;
  x=r|(((r^x)>>2)/c);
}
```

Điều kiện: $1 \le k \le n < 64$. Nếu $k = 0$, có đúng mask rỗng. Khi $n = 64$ phải xử lý giới hạn khác, không dịch `1ULL<<64`.

5. Bitset DP: tổng con (subset sum)

Ngữ cảnh dùng: Các phần tử không âm, tổng S không quá lớn; cần biết sum nào đạt được, đặc biệt khi knapsack Boolean truyền thống quá chậm.

```
const int S=200000;
bitset<S+1> can;
can[0]=1;
for(int v:a)if(0<=v && v<=S)
  can |= (can<<v);
// can[t] = reachable subset sum t
```

Cỡ bitset là hằng tại thời điểm biên dịch. Mỗi phép dịch/OR có cost xấp xỉ $O(S/64)$ machine words; dùng cho **0/1** subset sum, không hỗ trợ giá trị âm nếu không offset.

6. Bitset graph reachability

Ngữ cảnh dùng: N vừa và cần reachability giữa nhiều cặp; bitset giúp tăng tốc phép OR so với ma trận bool thông thường.

```
const int V=500;
bitset<V> reach[V];
for(int i=0;i<n;i++)reach[i][i]=1;
for(auto [u,v]:edges)reach[u][v]=1;
for(int k=0;k<n;k++)
  for(int i=0;i<n;i++)
    if(reach[i][k])reach[i]|=reach[k];
// reach[u][v] == 1 iff path u->v
```

Điều kiện: đồ thị tĩnh, $n \le V$, không cần độ dài đường đi. Kiểu phép toán Boolean. Độ phức tạp xấp xỉ $O(n^3/64)$ cho OR theo word và $O(n^2)$ kiểm tra bit.

7. GNU/STL tricks dùng thường xuyên

```
// kth element, 0-based, average O(n)
nth_element(a.begin(),a.begin()+k,a.end())
    ;
// remove duplicate after sorting
sort(all(a));
a.erase(unique(all(a)),a.end());
// avoid repeated reallocations
vector<int> v;v.reserve(n);
// count numbers in [L,R] of sorted a
auto cnt=upper_bound(all(a),R)
       -lower_bound(all(a),L);
```

Điều kiện: `lower/upper_bound` yêu cầu range đã sắp xếp. `nth_element` không sort toàn bộ mảng. `reserve` chỉ đổi capacity, không tăng size.

8. Tránh overflow / chia nguyên an toàn

```
using i128=__int128_t;
// requires 0<=a,b<m, m>0
ll mulmod(ll a,ll b,ll m){
  return (i128)a*b%m;
}
// ceil(a/b), a>=0,b>0
ll ceilDiv(ll a,ll b){
  return a/b+(a%b!=0);
}
// compare a*b with c safely
if((i128)a*b <= c){ /* ... */ }
```

Lưu ý: `(a+b-1)/b` có thể overflow dù đáp án vừa 64-bit. Với modulo composite, không chia theo kiểu số thực hay dùng Fermat inverse tùy tiện.

9. Tối ưu thực dụng + bẫy WA/TLE

Fast I/O: `ios::sync_with_stdio(false); cin.tie(nullptr);`; ưu tiên `'\n'` thay `endl` nếu không cần flush.

Bộ nhớ: mảng `long long` 10^6 phần tử ≈ 8 MiB; 2^{24} states ≈ 128 MiB.

DFS sâu: đường thẳng $2 \cdot 10^5$ đỉnh có thể tràn stack; cân nhắc iterative.

Kiểu số: kiểm tra `1<<i`, tích $a \cdot b$, `INF+weight`, chỉ số âm và mod âm.

Performance: giảm copies của vector lớn (dùng `const auto&`); preallocate khi biết size, tránh `map` nếu `vector` đủ.

Chọn bitset hay bitmask: bitset cho một *tập trạng thái rất rộng* xử lý song song; bitmask DP cho từng trạng thái con của *tập đối tượng nhỏ*.

Độ phức tạp: $O(2^N)$ thường chỉ với $N \lesssim$ 22–24 tùy máy, memory và số test; $O(3^N)$ nên nhỏ hơn. Đây là ngưỡng tham khảo, không phải điều kiện đủ.

10. Difference Array 2D (Imos)

Ngữ cảnh dùng: Cộng nhiều hình chữ nhật rồi lấy kết quả cuối (offline); query online dùng Fenwick 2D.

```
// 1-index; diff size (n+2)x(m+2)
void rectAdd(int x1,int y1,int x2,int y2,
     ll v){
  diff[x1][y1]+=v;
  diff[x2+1][y1]-=v;
  diff[x1][y2+1]-=v;
  diff[x2+1][y2+1]+=v;
}
void build2D(int n,int m){
  for(int i=1;i<=n;i++)for(int j=1;j<=m;j
     ++)
    diff[i][j]+=diff[i-1][j]+diff[i][j-1]
                -diff[i-1][j-1];
}
```

$O(1)$/update; $O(NM)$/build.

11. DSU parity - graph hai phía động

Ngữ cảnh dùng: Thêm dần cạnh vô hướng và phải kiểm tra graph có còn bipartite không. Mỗi cạnh (u, v) yêu cầu hai màu khác nhau (xor bằng 1).

```
int pp[N],ss[N],xr[N]; // init pp[i]=i, ss
     [i]=1
pair<int,int> findP(int u){
  if(pp[u]==u)return {u,0};
  auto [r,w]=findP(pp[u]);
  xr[u]^=w;pp[u]=r;
  return {r,xr[u]};
}
bool addParity(int u,int v,int w=1){
  auto [a,x]=findP(u);
  auto [b,y]=findP(v);
  if(a==b)return (x^y)==w;
  if(ss[a]<ss[b])swap(a,b);
  pp[b]=a;xr[b]=x^y^w;ss[a]+=ss[b];
  return true;
}
```

Trả false nếu mâu thuẫn (chu trình lẻ khi $w = 1$). Nếu cần xóa cạnh, dùng rollback/offline.

12. Largest Rectangle in Histogram

Ngữ cảnh dùng: Cho histogram có chiều cao không âm, tìm hình chữ nhật lớn nhất gồm các cột liên tiếp; mẫu kinh điển của monotonic stack.

```
ll maxHistogram(vector<ll>&h){
  int n=h.size();ll ans=0;
  vector<int> st;
  for(int i=0;i<=n;i++){
    ll cur=(i==n?0:h[i]);
    while(!st.empty()&&h[st.back()]>cur){
      ll height=h[st.back()];st.pop_back()
     ;
      int left=st.empty()?0:st.back()+1;
      ans=max(ans,height*(i-left));
    }
    if(i<n)st.push_back(i);
  }
  return ans;
}
```

$O(N)$ vì mỗi index push/pop tối đa một lần. Mở rộng: largest all-ones rectangle in binary matrix → xây histogram theo từng hàng.

13. DP minimum trong cửa sổ K

Ngữ cảnh dùng: $dp[i] = cost[i] + \min_{i-K \le j < i} dp[j]$ với $K \ge 1$. Thay quét K trạng thái bằng monotonic deque để được $O(N)$.

```
// 0-index dp, cost[1..n], K>=1
vector<ll> dp(n+1);
deque<int> q;q.push_back(0);
for(int i=1;i<=n;i++){
  while(!q.empty()&&q.front()<i-K)q.
     pop_front();
  dp[i]=cost[i]+dp[q.front()];
  while(!q.empty()&&dp[q.back()]>=dp[i])q.
     pop_back();
  q.push_back(i);
}
```

Điều kiện: giá trị cửa sổ chỉ phụ thuộc $dp[j]$; cửa sổ trượt tuần tự sang phải. Nếu transition có thêm cost phụ thuộc cả i, j, cần kiểm tra có dùng deque được không.

15. Chia nguyên âm - floor/ceil đúng

Ngữ cảnh dùng: Binary search/đếm nghiệm nguyên, cần $\lfloor a/b \rfloor$ hoặc $\lceil a/b \rceil$ khi a hoặc b âm. C++ mặc định chia lấy phần nguyên hướng về 0.

```
ll floorDiv(ll a,ll b){ // b!=0
  ll q=a/b,r=a%b;
  if(r&&((r<0)!=(b<0)))q--;
  return q;
}
ll ceilDiv(ll a,ll b){ // b!=0
  ll q=a/b,r=a%b;
  if(r&&((r<0)==(b<0)))q++;
  return q;
}
```

Không dùng `(a+b-1)/b` với số âm; trường hợp $a =$ `LLONG_MIN`, $b = -1$ vẫn tràn signed 64-bit.